TASK 1: Oracle VirtualBox Setup and Ubuntu

Date: 12-02-2026

Oracle VM VirtualBox is a virtualization application that enables users to run different operating systems on one computer. This involved first installing Ubuntu Linux inside the VirtualBox to create a virtual Linux environment. It is used for:

- Operating system and command line operation
- Create a secure and isolated environment for testing purposes

1. Installing Oracle VirtualBox

Steps:

1: Visit https://www.virtualbox.org
2: Download the installer for Windows hosts.
3: Open the installer file and click Next to start the setup process.
4: Keep the default configuration settings and install all the required components.
5: Allow the network driver installation.
6: Hit Finish to finalize the installation.

2. Download Ubuntu

Steps:

1: Go to https://ubuntu.com/download/desktop
2: Download the Ubuntu 22.04 LTS ISO file.

3. Creating Ubuntu Virtual Machine

Steps:

1: Open VirtualBox and click on New.
2: Enter the following information:

- **Name: Ubuntu**
- **Linux type**
- **Version: Ubuntu (64-bit)**

3: Allocate memory (RAM):

- **Minimum: 2048 MB**
- **Recommended: 4096 MB**

4: Set up a virtual hard disk:

- **Disk Type: VDI (VirtualBox Disk Image)**
- **Storage Allocation: Dynamically allocated**
- **Disk Size: Between 20–25 GB**

4. Installing Ubuntu

Steps:

1: Select the created virtual machine and click on Start.
2: Select the ISO with the download Ubuntu operating system.
3: Click Install Ubuntu.
4: Select the Normal installation option.
5: Select Erase disk and install Ubuntu (no changes will be made on your physical disk; alterations will be limited to the virtual disk only).
6: Provide a username and password.
7: When installation is done, the user should restart the virtual machine.

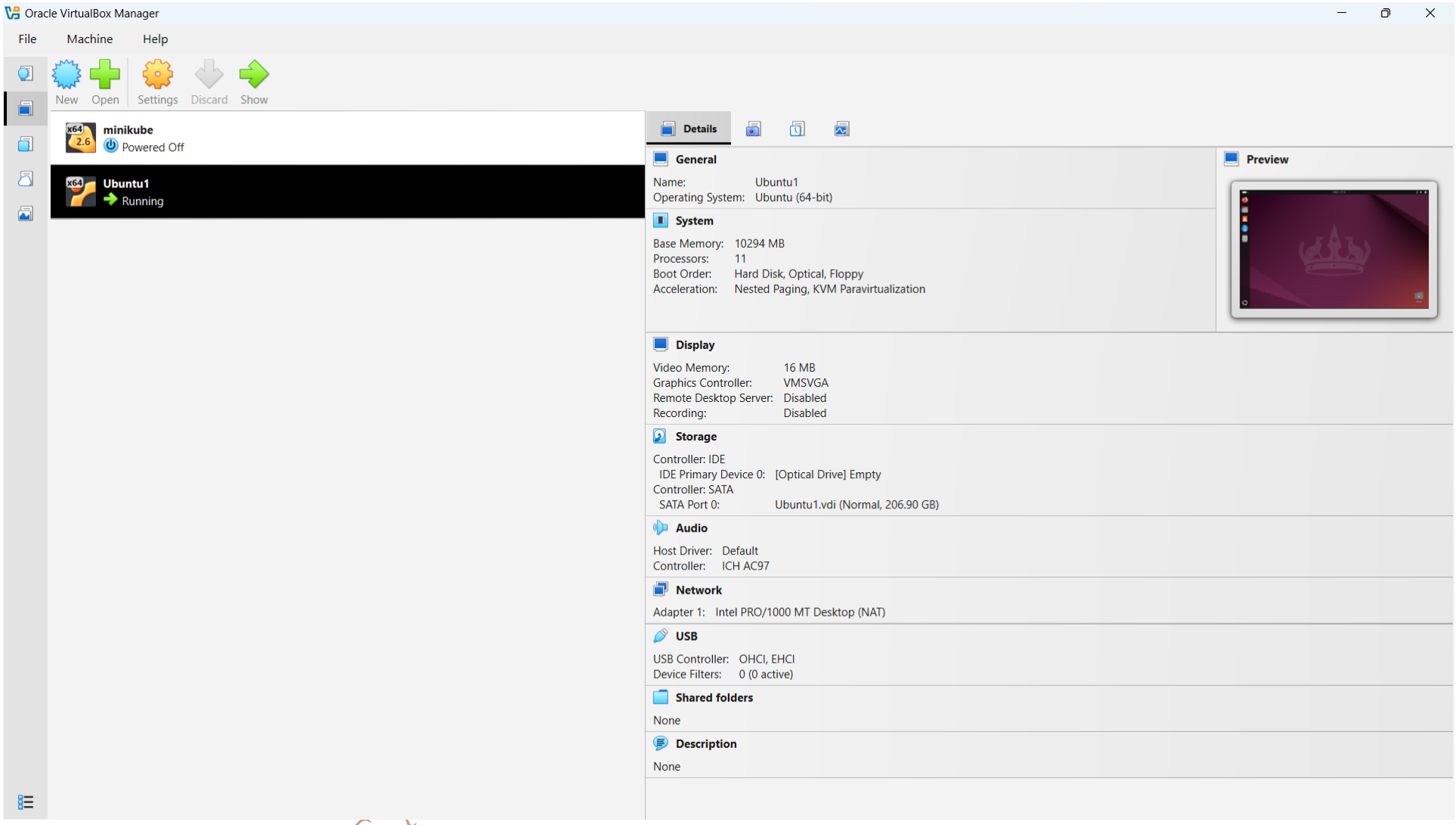


Figure 1: Ubuntu OS installed in a VM

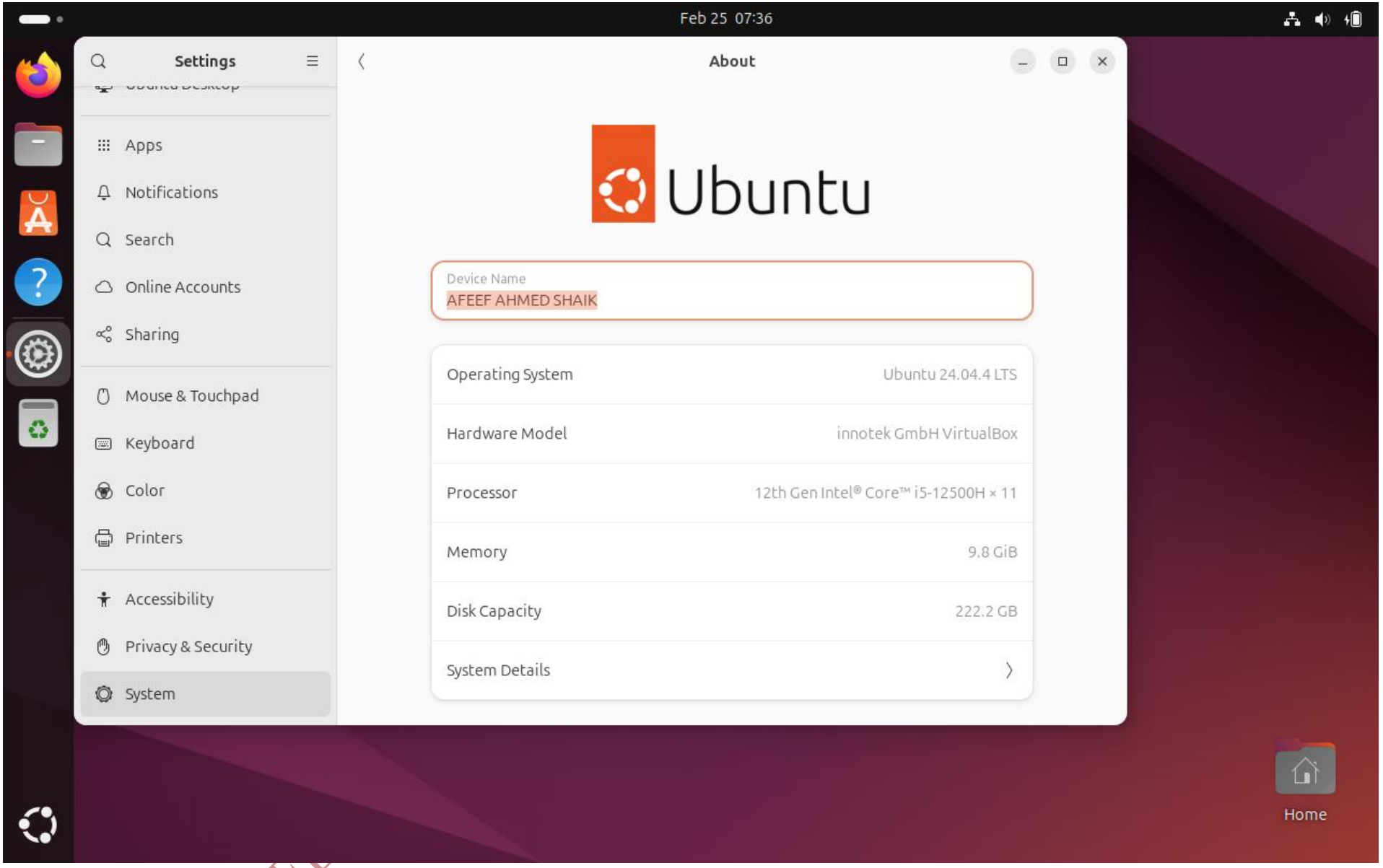


Figure 2: USER PROFILE

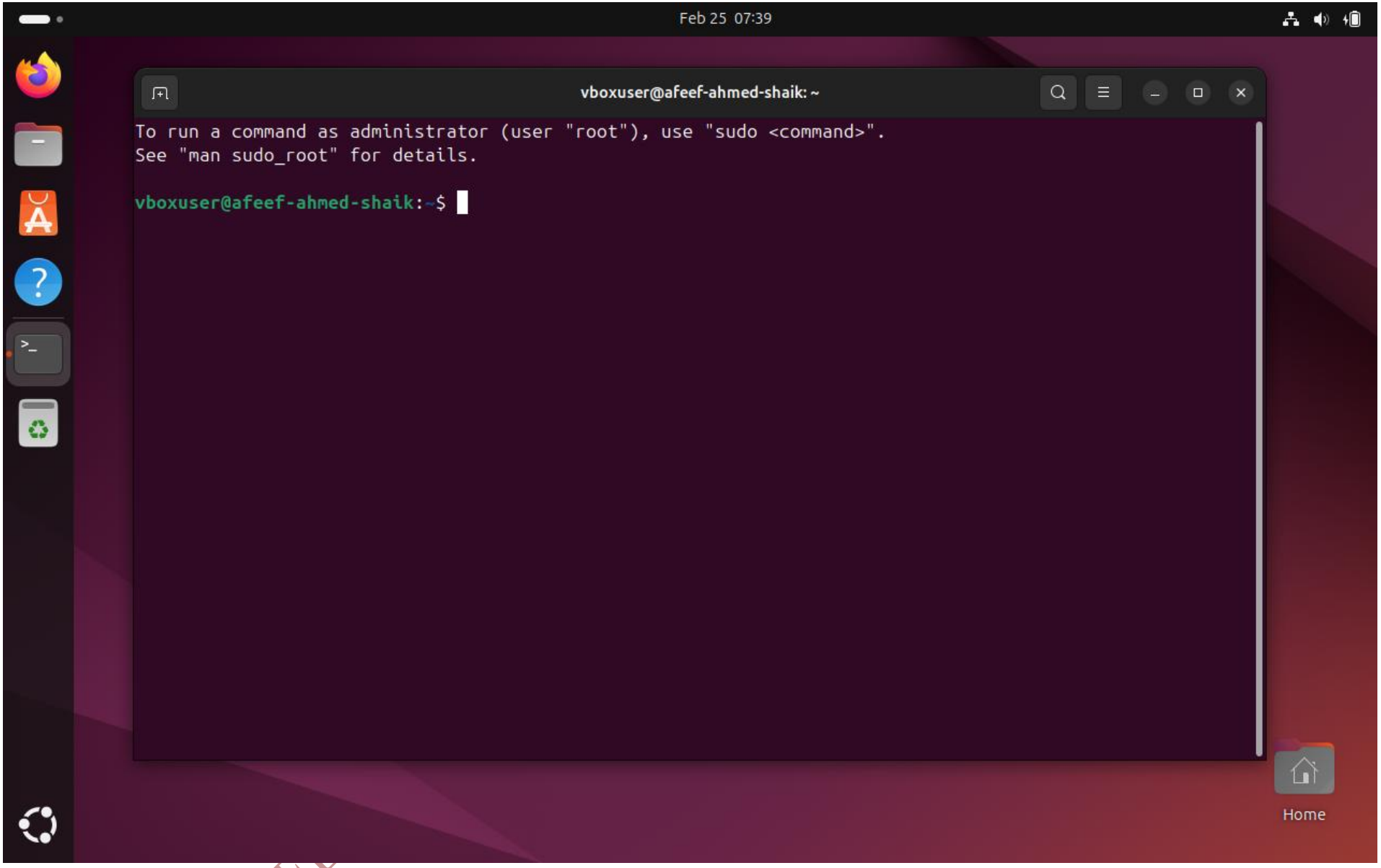


Figure 3: USER TERMINAL

Task 2 : Working Docker Images and NGC Container

Date: 18-02-2026

NVIDIA NGC Login and API Key Generation

To use GPU-enabled containers, which are available on the NVIDIA Container Registry, an **NVIDIA NGC** account was created. To ensure that the system can securely download the NGC containers, an API key has been generated and used to authenticate Docker.

Step 1: Create NVIDIA NGC Account

1. Launch a web browser.
2. Visit https://ngc.nvidia.com
3. Press **Sign In**.
4. Click on Create Account.
5. Fill out the registration form, and verify your email to activate your account.

Step 2: Generate NGC API Key

1. Log in to the NVIDIA NGC dashboard.
2. Click your User Profile icon.
3. Select **API Key** option.
4. Click Generate API Key.
5. Copy the generated key and save it somewhere secure for later use.

Step 3: Login to NGC Using Docker (Ubuntu Terminal)

In the Ubuntu terminal, run the following command: Code: docker login nvcr.io Input the following credentials when asked

- **Username: $oauthtoken**
- **Password: Paste the generated NGC API key**

With a successful login process, the terminal will display: The terminal after a login sequence is as follows: Login Succeeded.

This shows that the system is successfully authenticated and can now pull containers from the NVIDIA NGC registry.

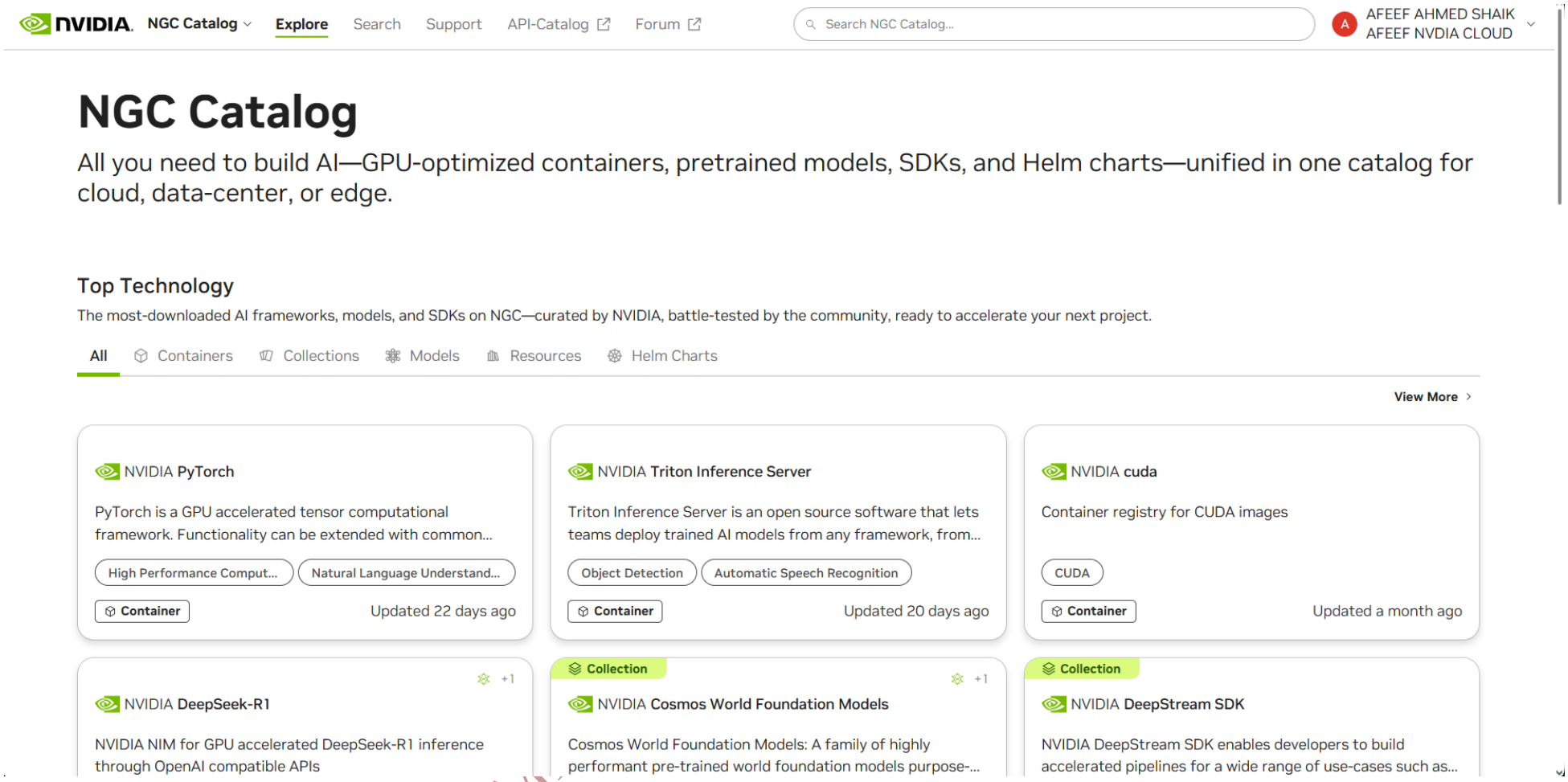


Figure 4: NVIDIA NGC USER DASHBOARD

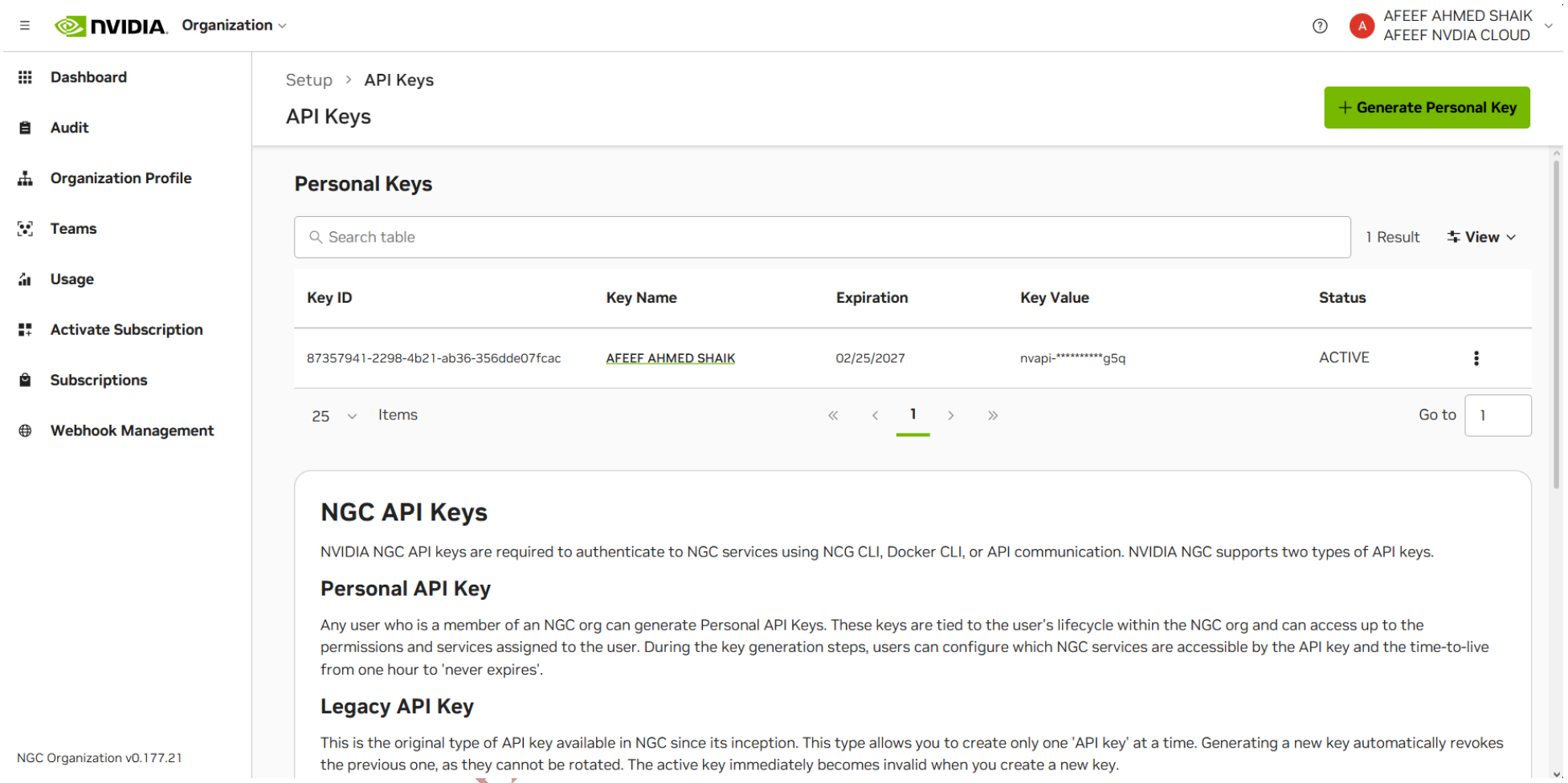


Figure 5: NVIDIA NGC API ORGANIZATION DASHBOARD

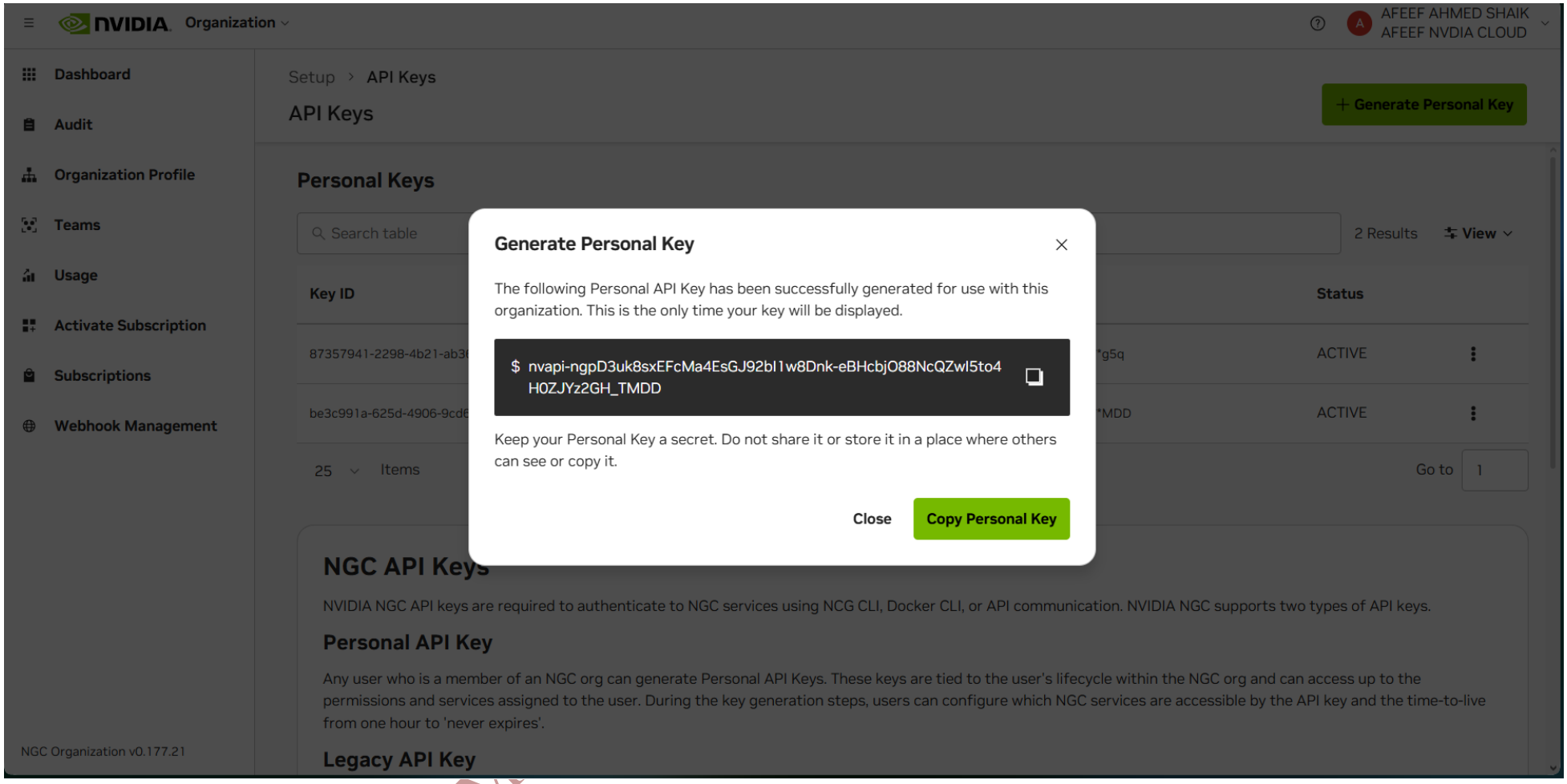


Figure 6: NVIDIA NGC API KEY GENERATION

Docker Images Before Pulling GitHub and Friend Images

The existing Docker images were checked first to ascertain the current status of the overall Docker trends before downloading any new images.

The below command was run:

```
docker images
```

This command lists out all the docker images stored locally in the system. These details include repository, tag, image ID, the date when the image was created, and the size of the image.

By running this command, verifying which images are already present and which have been downloaded later becomes much easier.

At this point, only previously pulled images (for example, NVIDIA container images) were in the system. Second, there were no custom images downloaded into the system created by the user or their peers.

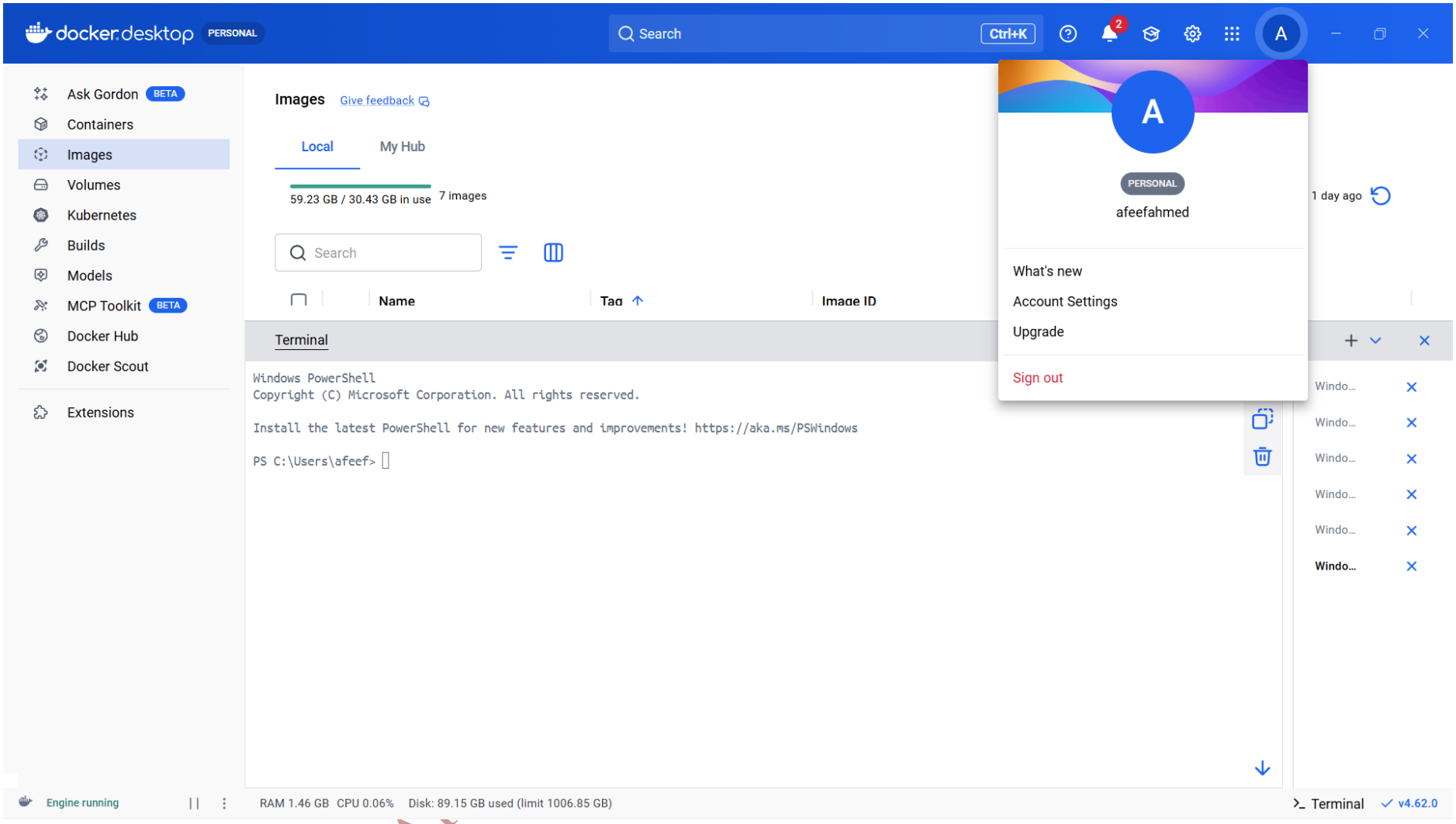


Figure 7: Docker Images Before Pulling GitHub and Friend Images

Pulling TensorFlow Container & Program Execution

The TensorFlow container was retrieved from the NVIDIA NGC registry to initiate a pre-configured deep learning Dockerized environment. This container has Python, TensorFlow, and all the necessary libraries needed in the process, hence, there is no need to install dependencies manually.

```
docker pull nvcr.io/nvidia/tensorflow:25.10-tf2-py3
```

After the downloading process had been completed, the container was then launched in interactive mode through:

```
docker run -it nvcr.io/nvidia/tensorflow:25.10-tf2-py3
```

This command started an interactive Python session inside the container.

To verify the TensorFlow installation, whether it was successful or not, and if the software behaved as required, the following commands were run in Python:

```
import tensorflow as tf
print("TensorFlow Version:", tf._version_)
print("GPU Available:", tf.config.list_physical_devices('GPU'))
```

The output included the currently installed tensorflow version and a statement that the container was running properly. Since the system does not have an NVIDIA GPU, the GPU list was empty, proving that the GPU has been disabled and that TensorFlow is working in CPU mode.

This verified that the TensorFlow container was successfully downloaded and run within the Docker environment.

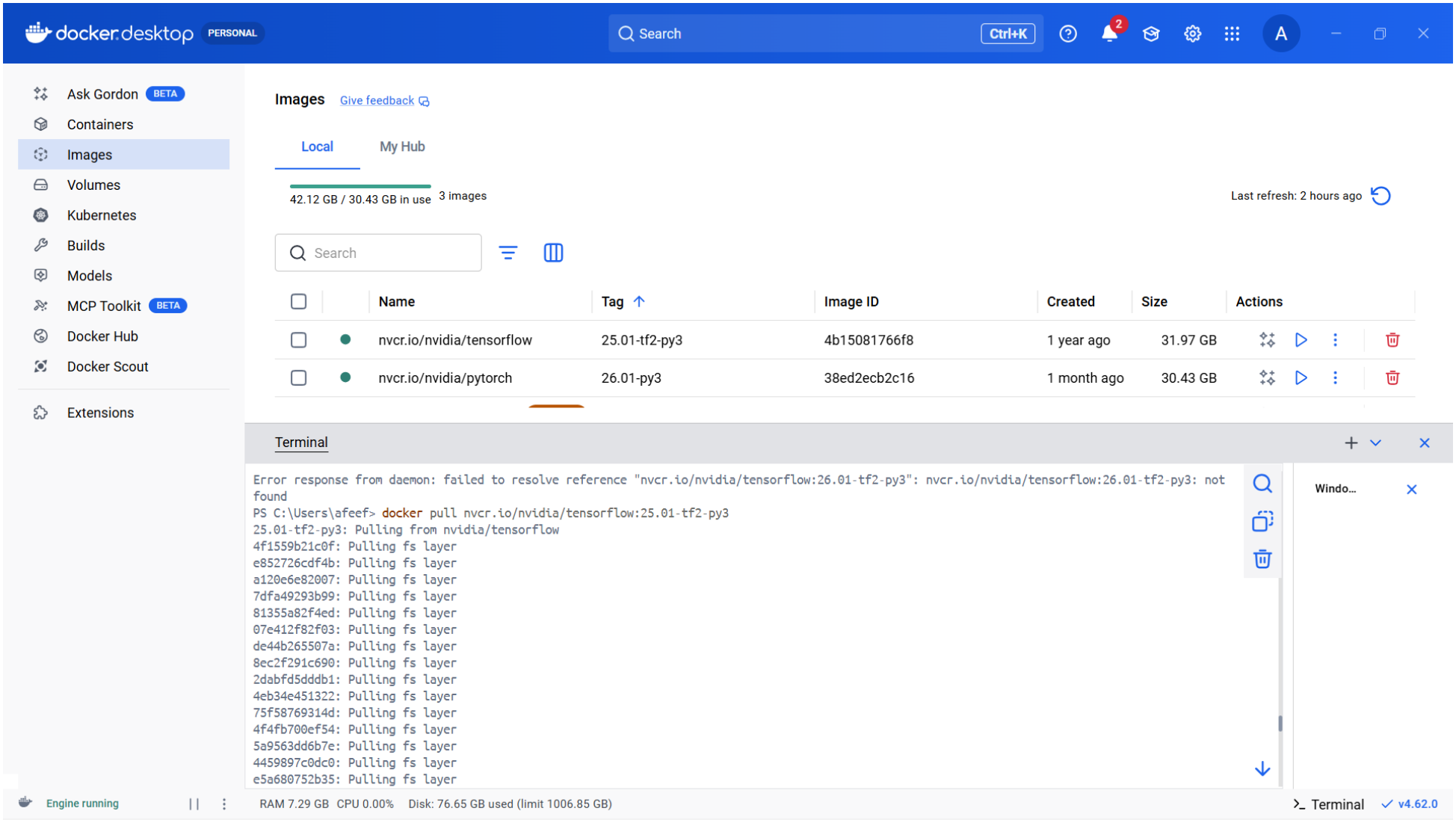


Figure 8: TensorFlow Container Pull

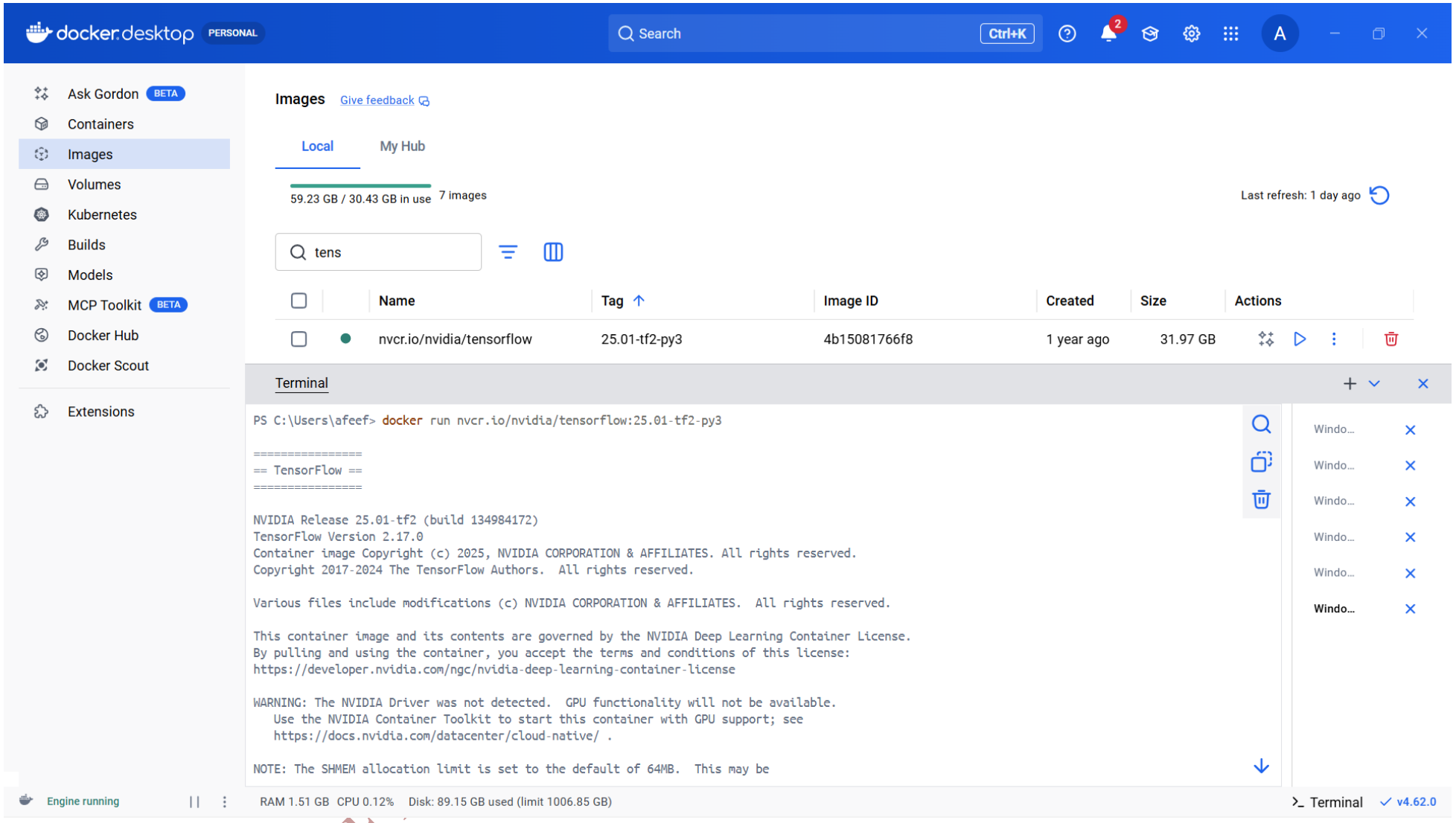


Figure 9: TensorFlow Container Run

PyTorch Container Pull & Program Execution

To have a ready deep learning environment inside Docker, the PyTorch container was downloaded from the NVIDIA NGC registry. Not only does this container require no manual configuration, but also it comes with Python, PyTorch, CUDA libraries (in case a compatible GPU is available), and all necessary dependencies.

The image was pulled using this command:

```
docker pull nvcr.io/nvidia/pytorch:26.10-py3
```

After the download was performed successfully, the container was launched in the interactive mode with:

```
docker run -it nvcr.io/nvidia/pytorch:26.10-py3
```

This command opened a Python terminal within the container environment.

To make sure that the PyTorch tool was correctly installed and working properly, the following Python commands were executed:

```
import torch
print("PyTorch Version:", torch._version_) print("GPU Available:",
torch.cuda.is_available())
```

The output showed the installed PyTorch version and the availability of GPU acceleration. Since the system does not an NVIDIA GPU, the result returned False, meaning that PyTorch was running on CPU.

This verified that the PyTorch container has been correctly downloaded and is running inside the Docker environment.

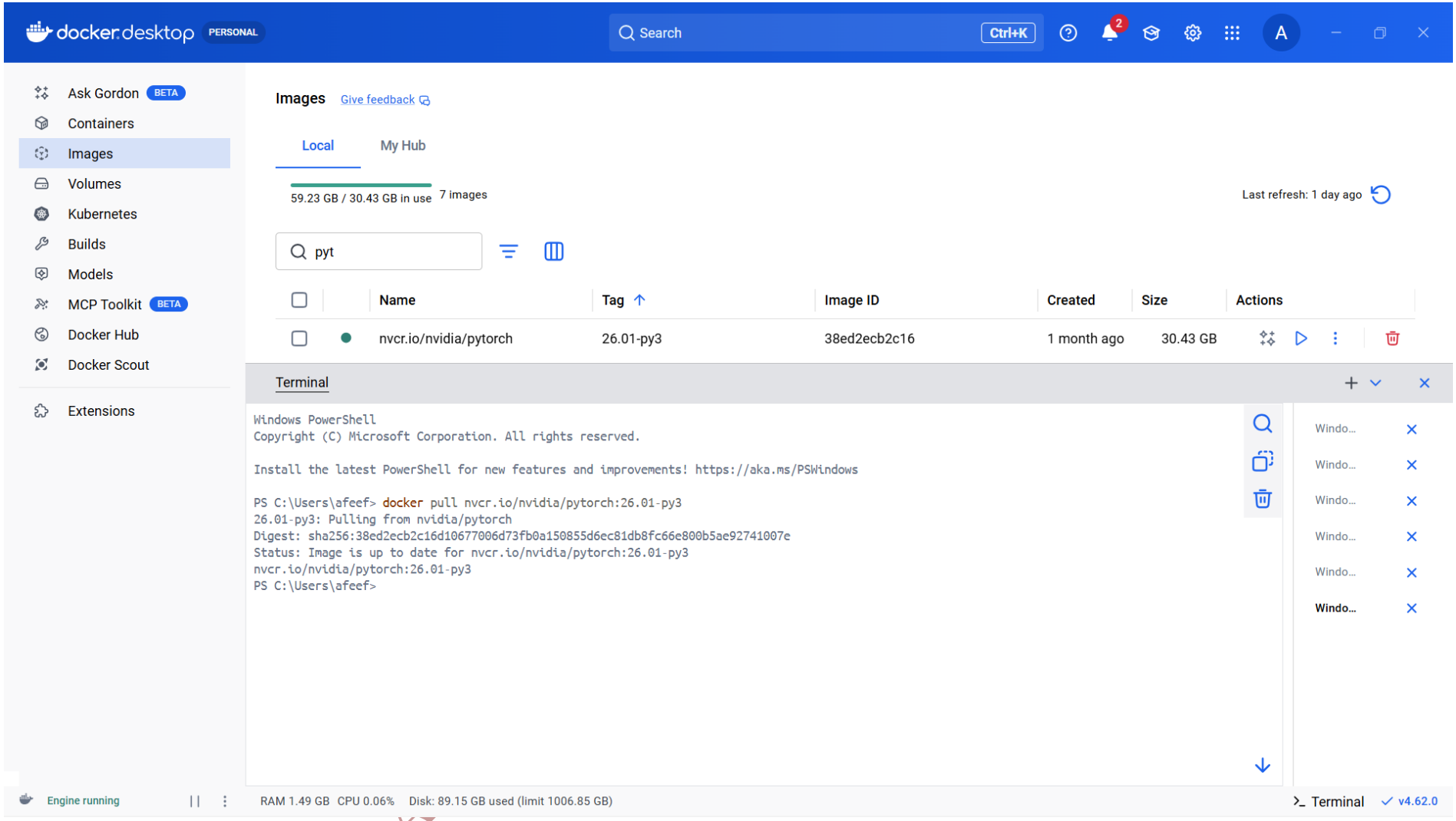


Figure 10: PyTorch Pull

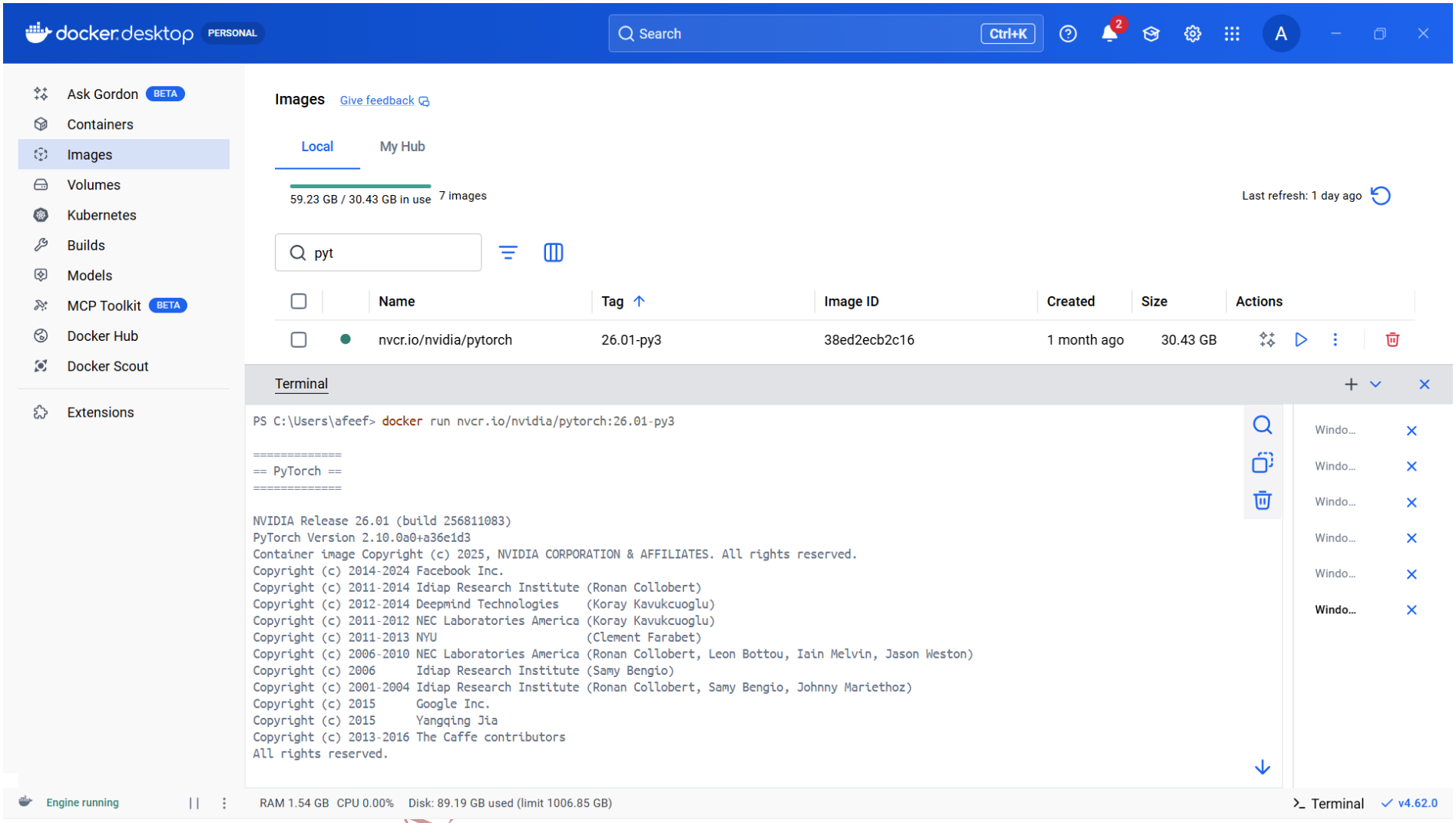


Figure 11: PyTorch Run

Use Case Container Setup Using NGC Pull and Run

As a practical example to show the downloading and execution processes of NGC containers with Docker, the Triton Inference Server container was used. It is built in with accelerated libraries for efficient, high-performance inference, and optimized to deploy and serve AI models in production settings.

The container image was pulled from the NVIDIA NGC registry using the below command; This is followed by pulling the specific container image from the NVIDIA NGC registry:

```
docker pull nvcr.io/nvidia/tritonserver:24.10-py3
```

After the image was downloaded successfully, the container was launched to confirm that it functions properly:

```
docker run nvcr.io/nvidia/tritonserver:24.10-py3
```

Once the container was started, initialization logs appeared on the terminal, implying that the Triton Inference Server was starting correctly. Thus, once the container started, initialization logs appeared on the terminal, and it was evident that the Triton Inference Server was starting successfully, and these logs verified that the container was functioning properly.

Triton Inference Server is common for deploying and managing trained machine learning models so that applications can send inference requests and get predictions in real time. The experiment has proven the capabilities of NGC container in real-world AI model serving and deployments.

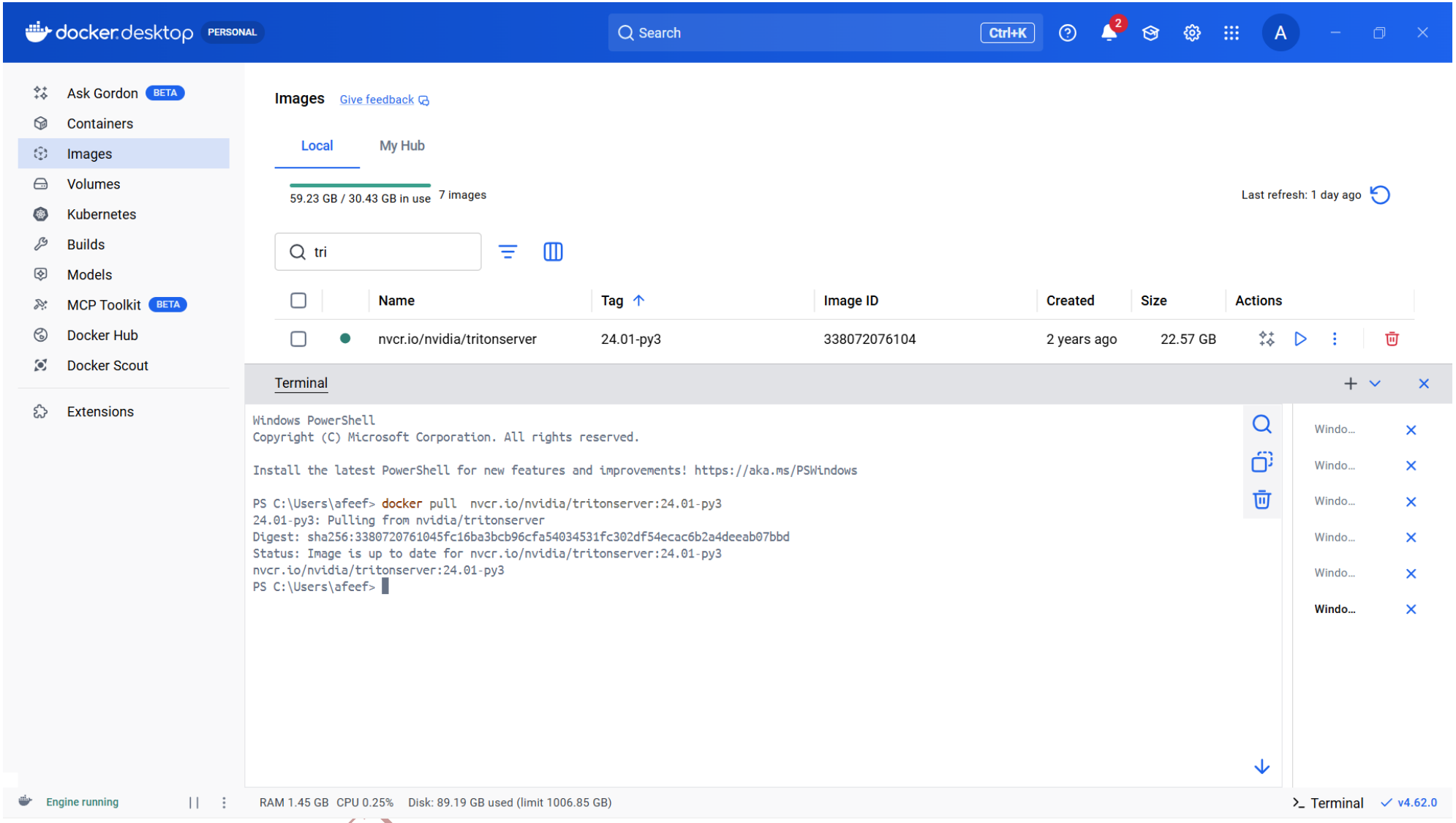


Figure 12: Use Case Container Pull

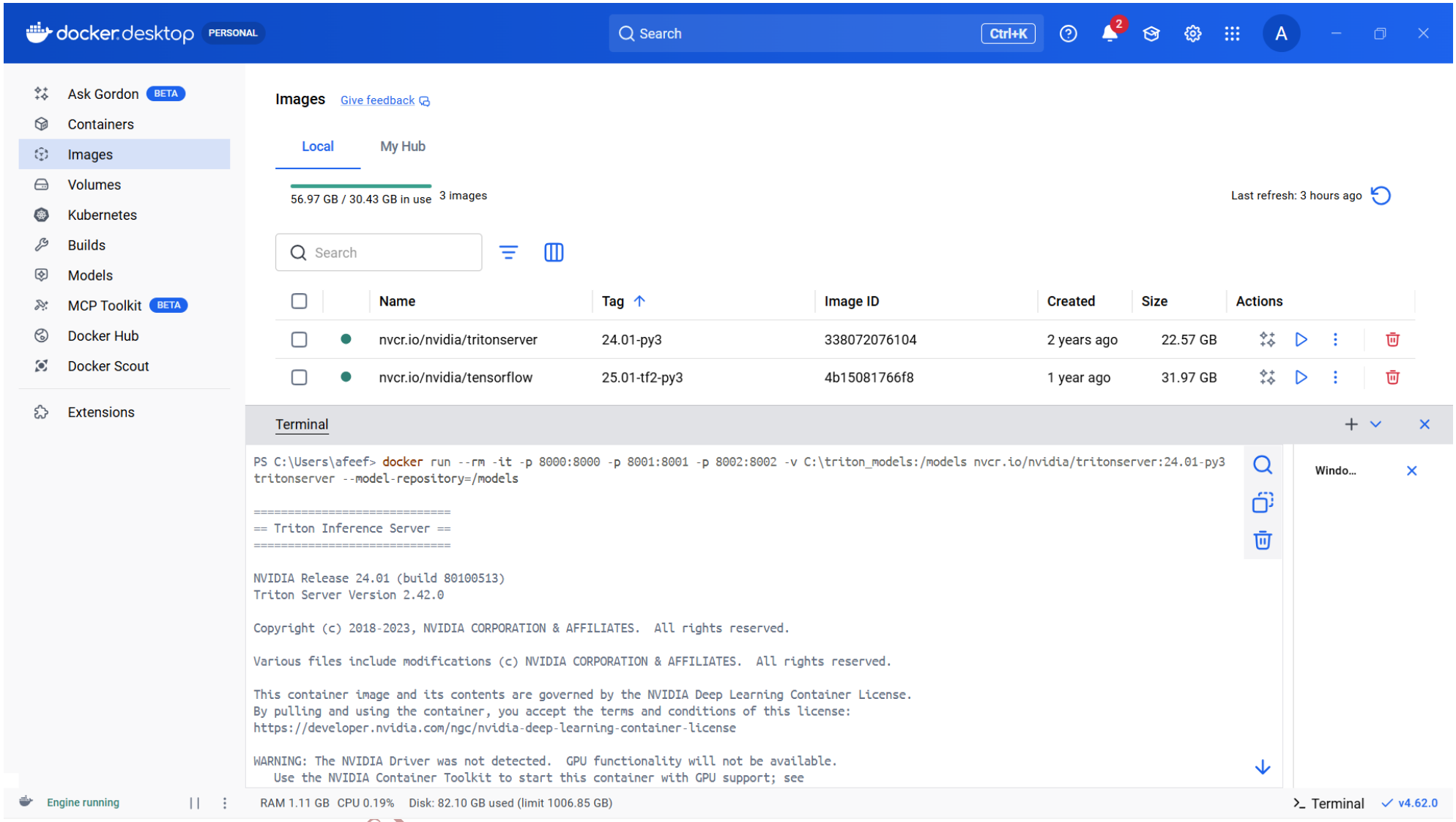


Figure 13: Use Case Container Run

Task 3 : Installition and Pulling Packages from DOCKER Hub

Date: 20-02-2026

Self Project Pushed in Docker Hub

The personal project was containerized and uploaded to Docker Hub, so that it can be shared, deployed, and run on any machine with Docker.

First, a Docker image was created from the project directory using a Dockerfile with this command:

```
build -t fitnessapp .
```

After the successful building of the image, it was tagged with the Docker hub username and version number, thus preparing the image for the upload:

```
docker tag fitnessapp afeefahmed/fitnessapp:v1
```

Next, the Docker Hub authentication was done by running:

```
docker login
```

Once logged in, the image was pushed to the Docker Hub repository using:

```
docker push afeefahmed/fitnessapp:v1
```

During the push operation, Docker uploaded the image layers to the Docker Hub repository. The digest message indicated that the upload was successfully completed.

This finalizes the assurance that the containerized project can be downloaded and run on any computer system, thus ensuring portability and deployment ease.

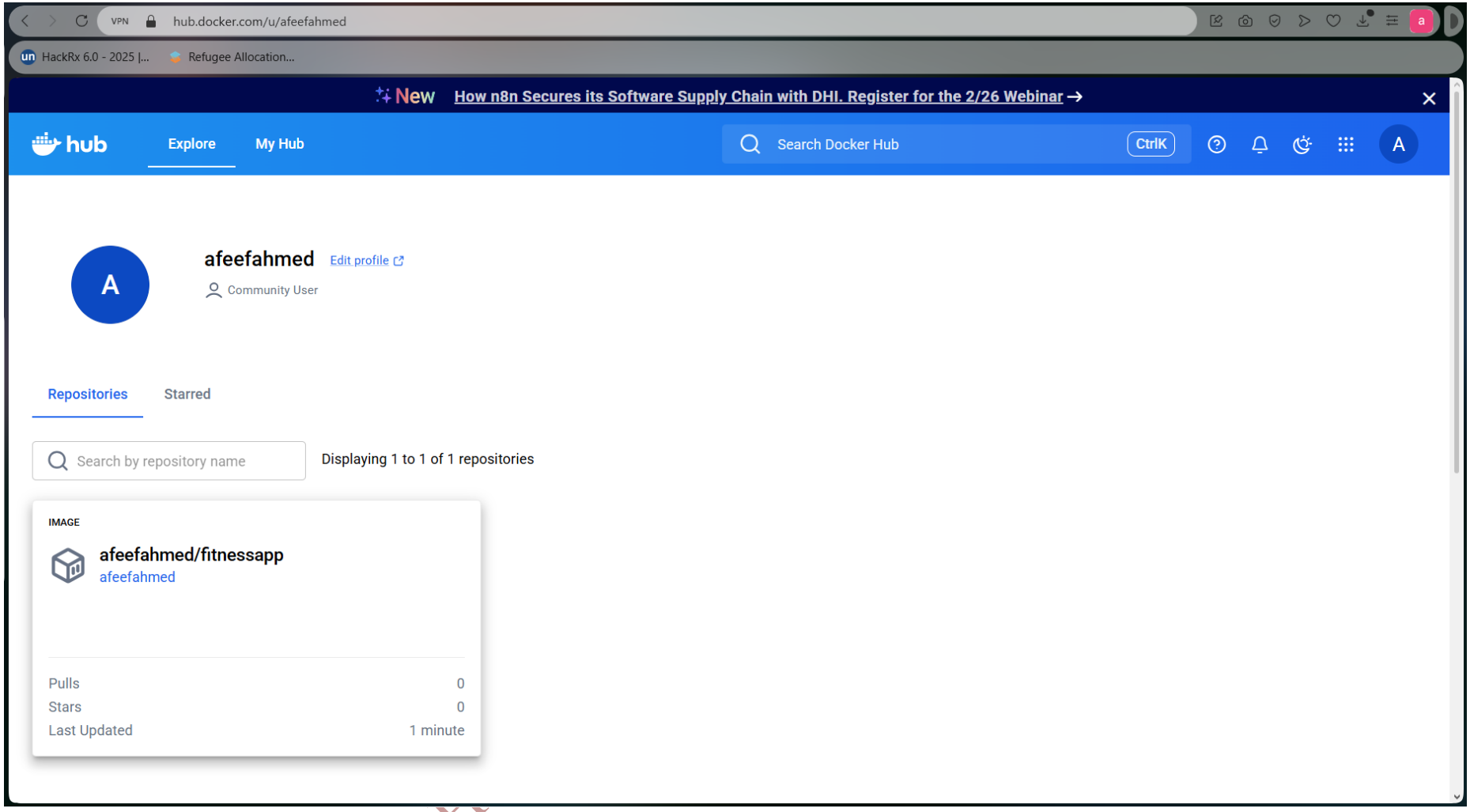


Figure 14: Self Project Pushed in Docker Hub

Pulling Self Image and Running Self Image

The project image that was previously uploaded to Docker Hub was pulled again to verify that it can be downloaded and run on any system. This is to ensure that the project image can be downloaded and run on any machine.

The image was downloaded via the below command:

```
Docker pull afeefahmed/fitnessapp:v1
```

After downloading, there was a message in Docker that the image was successfully received or was already available in the latest version.

Finally, the container was launched to run the application:

```
docker run afeefahmed/:v1
```

However, since the project is a web-based application, it was run with port mapping so that it could be accessed via a web browser:

```
docker run -p 8600:8600  afeefahmed/fitnessapp:v1
```

Once the container had started running, the application was available at: Once the container was running, the application was available under the following address: http://localhost:8600

This proved that the project image was indeed pulled from Docker Hub and ran successfully, proving container portability and Docker's ability to perform rapid deployment across systems.

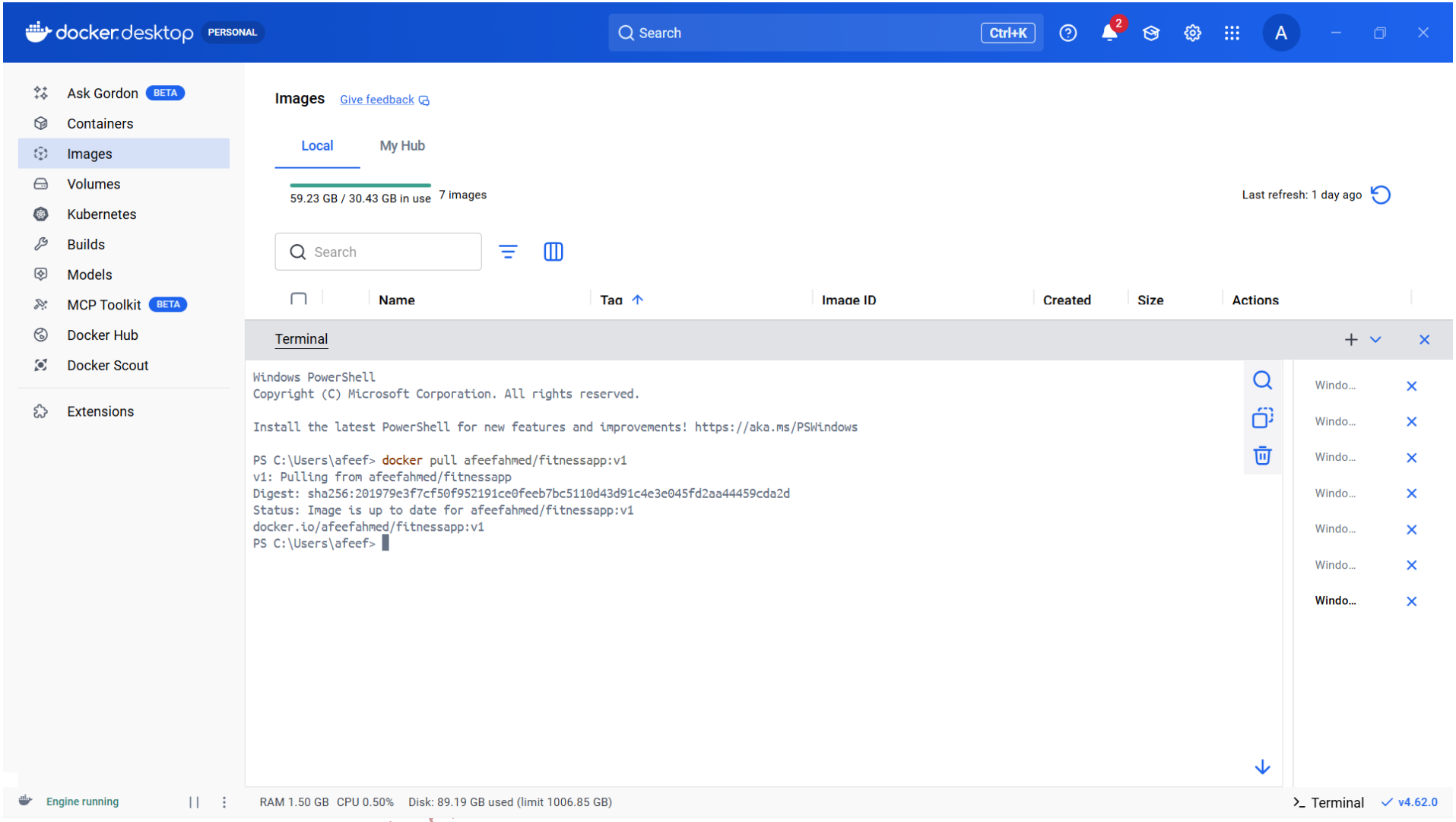


Figure 15: Pulling Self Image

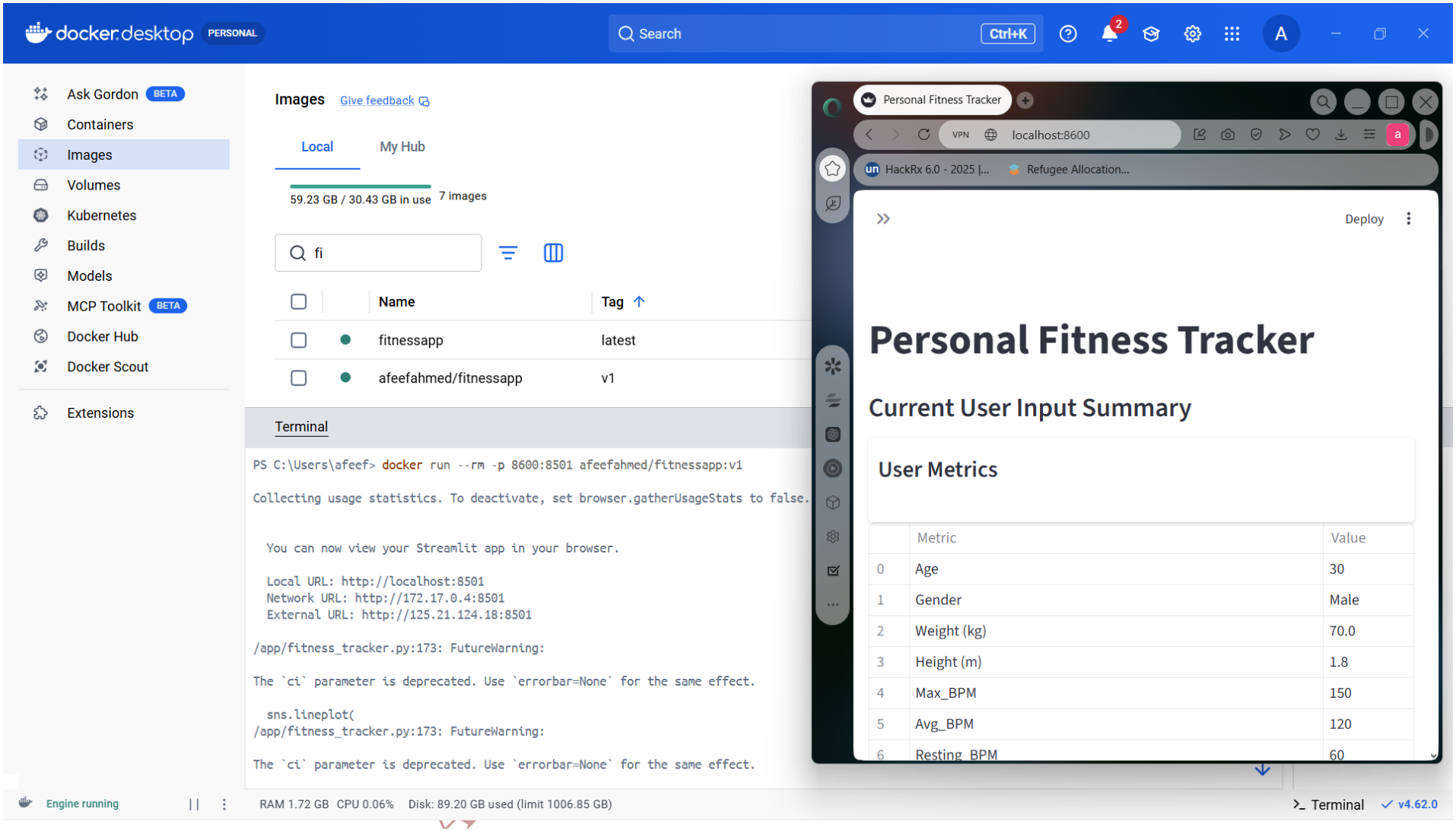


Figure 16: Running self-image

Pulling and Running Friend Image

The docker image shared by a friend was pulled from the docker hub to verify that containers published by other users can be successfully downloaded and executed. Pulling the image ensures that a similar version of the application can be created on other machines without any need for manual installation or setup.

The image was downloaded using the following command:

```
docker pull afeefahmed/fitnessapp:v1
```

After the process was complete, a confirmation message was displayed by Docker, confirming that the image had been successfully retrieved.

Next, the application was launched by executing the container:

```
docker run afeefahmed/fitnessapp:v1
```

If the application involves a web-based interface, it can be started by port mapping to enable browser access:

```
docker run -p 8600:8600 afeefahmed/fitnessapp:v1
```

However, the application worked fine after the container was put into operation. Once the container was started, the application was running successfully, which proved that containers created by other users can be pulled and executed, showing container portability and container sharing through Docker Hub.

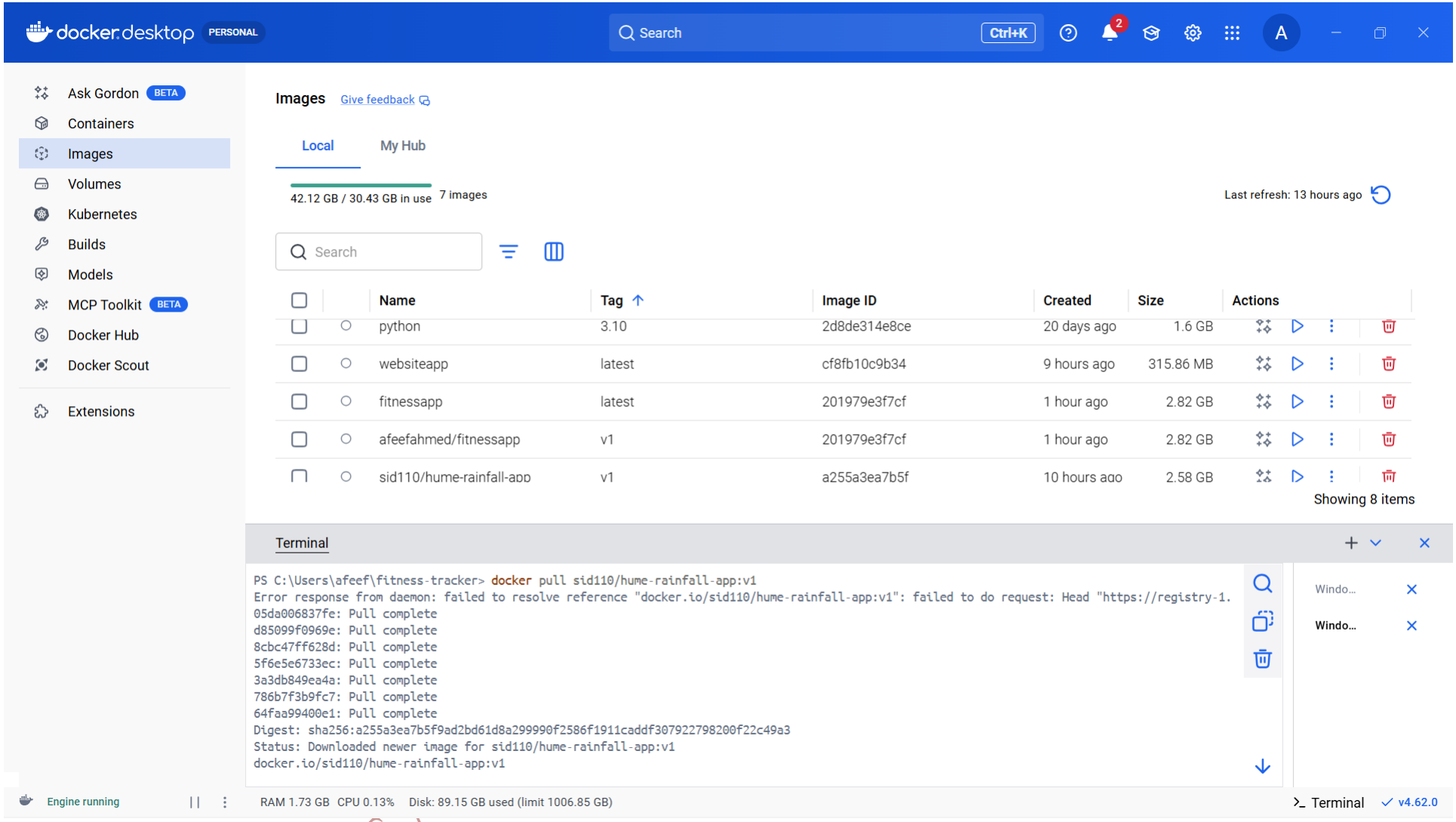


Figure 17: Pulling Friend Image

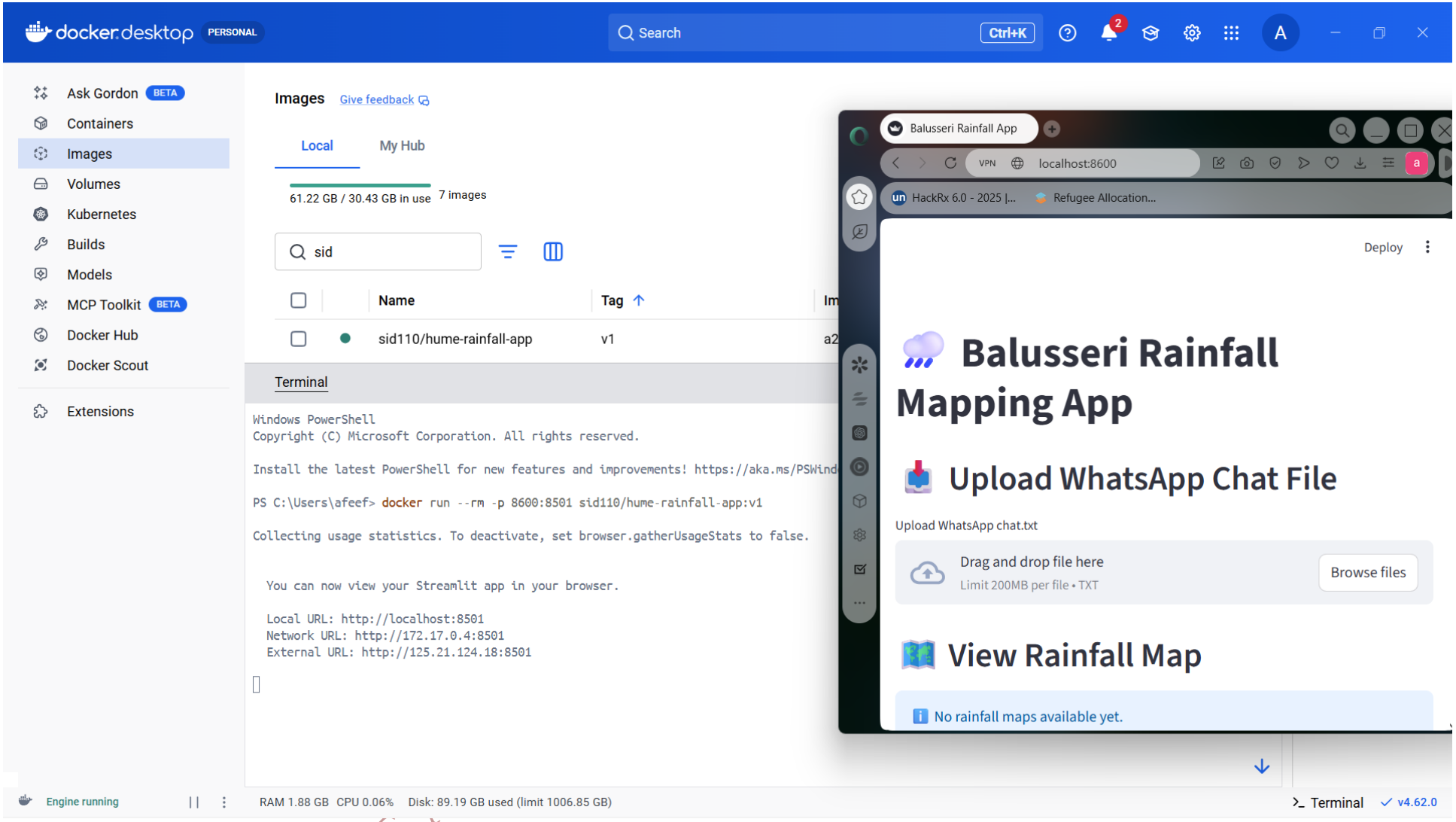


Figure 18: Running Friend Image

Docker Images After All Pulls

After downloading all the necessary containers, the list of Docker images was checked to ensure that every image was successfully pulled and stored locally. Finally, the list of docker images was checked to make sure that all images have been downloaded and stored locally, which includes tensorflow, pytorch, triton server images, self project image, and friend's image.

The below command was run:

```
docker images
```

This command prints all the locally saved Docker images alongside some major details, including:

- Repository
- Tag (version)
- Image ID
- Image creation date
- Size of the image

Output images included:

- NVIDIA TensorFlow container
- NVIDIA PyTorch container
- Triton Inference Server container
- A self project image retrieved from Docker Hub (ibid).
- A friend's shared Docker image

Finally, this verification step assured that:

- All of the required images were downloaded successfully
- The images are downloaded and saved locally (Docker environment)
- The containers are in place and can be run when necessary

Hence, this confirms successfully completing the image pulling process and correctly setting up the Docker environment.

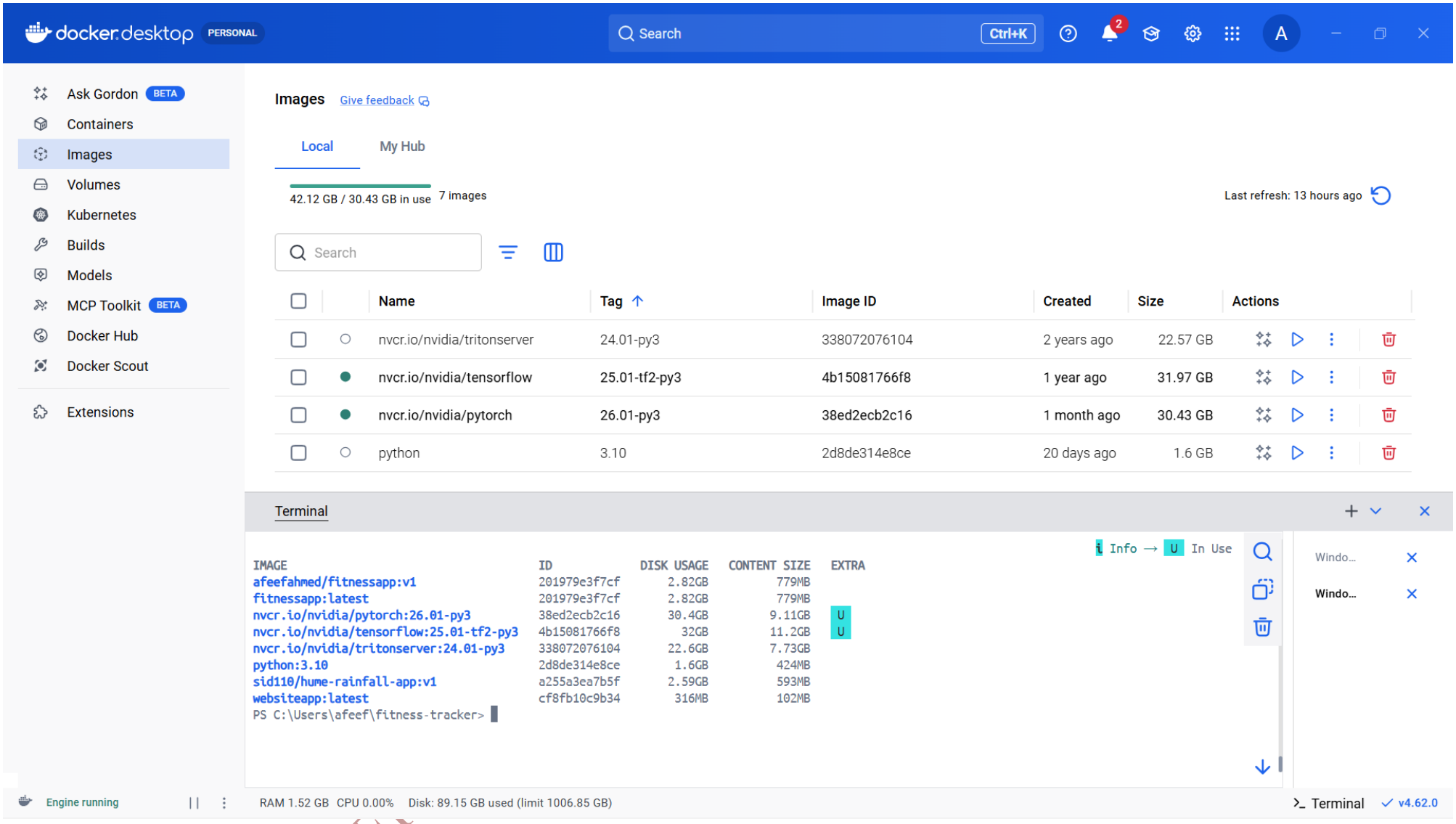


Figure 19: Docker Images After All Pulls

Task 4 : Implementing and executing Kubernetes

Date: 27-02-2026

CHECKING NODES & PODS AFTER INSTALLING MINIKUBE

After installing Minikube, a Kubernetes cluster was started to build a local single-node cluster for running containers.

The cluster was started through the command:

```
minikube start
```

This command executes the following functions:

- Creates a local Kubernetes cluster
- Sets up a virtual node using the Docker driver
- Configures kubectl communication with the cluster (ibid)

Once the startup procedure was complete without any errors, the cluster status was checked.

For checking whether the node was running correctly, the following command was fulfilled:

```
kubectl get nodes
```

The output showed the minikube node as Ready, indicating that the cluster was correctly initialized.

Next, all pods in the running status across the namespace are checked using:

```
kubectl get pods -A
```

This command outputted relevant system pods necessary for Kubernetes operations, including:

- kube-apiserver
- kube-controller-manager
- kube-scheduler
- Other core Kubernetes services

These verification steps proved that:

- Minikube was successfully started
- The Kubernetes node was in a Ready
- Verification that all system's components were in order

This made sure that the local kubernetes cluster was fully functional and ready to be used for deploying containerized applications.

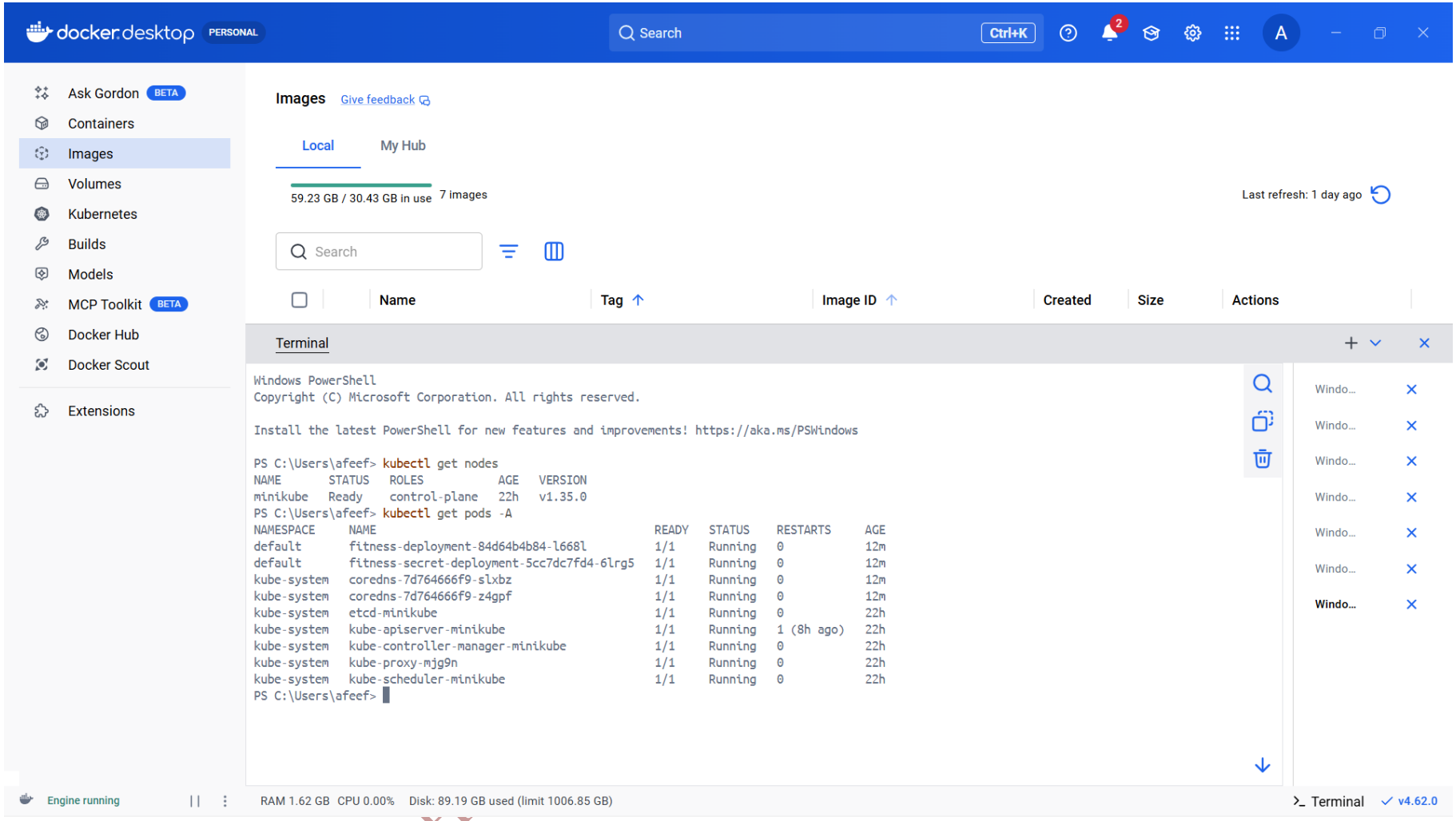


Figure 20: Checking Nodes & Pods After Installing

YAML DEFINING FOR SELF PULL

A Kubernetes deployment YAML file was created to deploy the self project image from Docker Hub into the Minikube cluster. This configuration file states the desired state of the application, and it includes details, such as the container image, number of replicas, and ports.

When this YAML file gets applied, Kubernetes, by default:

- Pulls the required container image from Docker Hub
- Sets up the pods to the number specified in the configuration
- Maintains the number of replicas as specified
- Guarantees that the application runs as per the configuration

The deployment configuration was saved in a file called:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: fitness-deployment
spec:
 replicas: 1
 selector:
  matchLabels:
   app: fitnessapp
 template:
  metadata:
   labels:
    app: fitnessapp
  spec:
   containers:
   - name: fitness-container
    image: afeefahmed/fitnessapp:v1
    ports:
    - containerPort: 8501
```

This configuration is meant to instruct Kubernetes to create a single replica of the container from self project image from Docker Hub. Upon the deployment being applied, Kubernetes downloads and executes it its cluster (on devices contained in the cluster) in case the image has not been previously downloaded. Thus, the application is run according to this configuration inside the Minikube (Kube) environment.

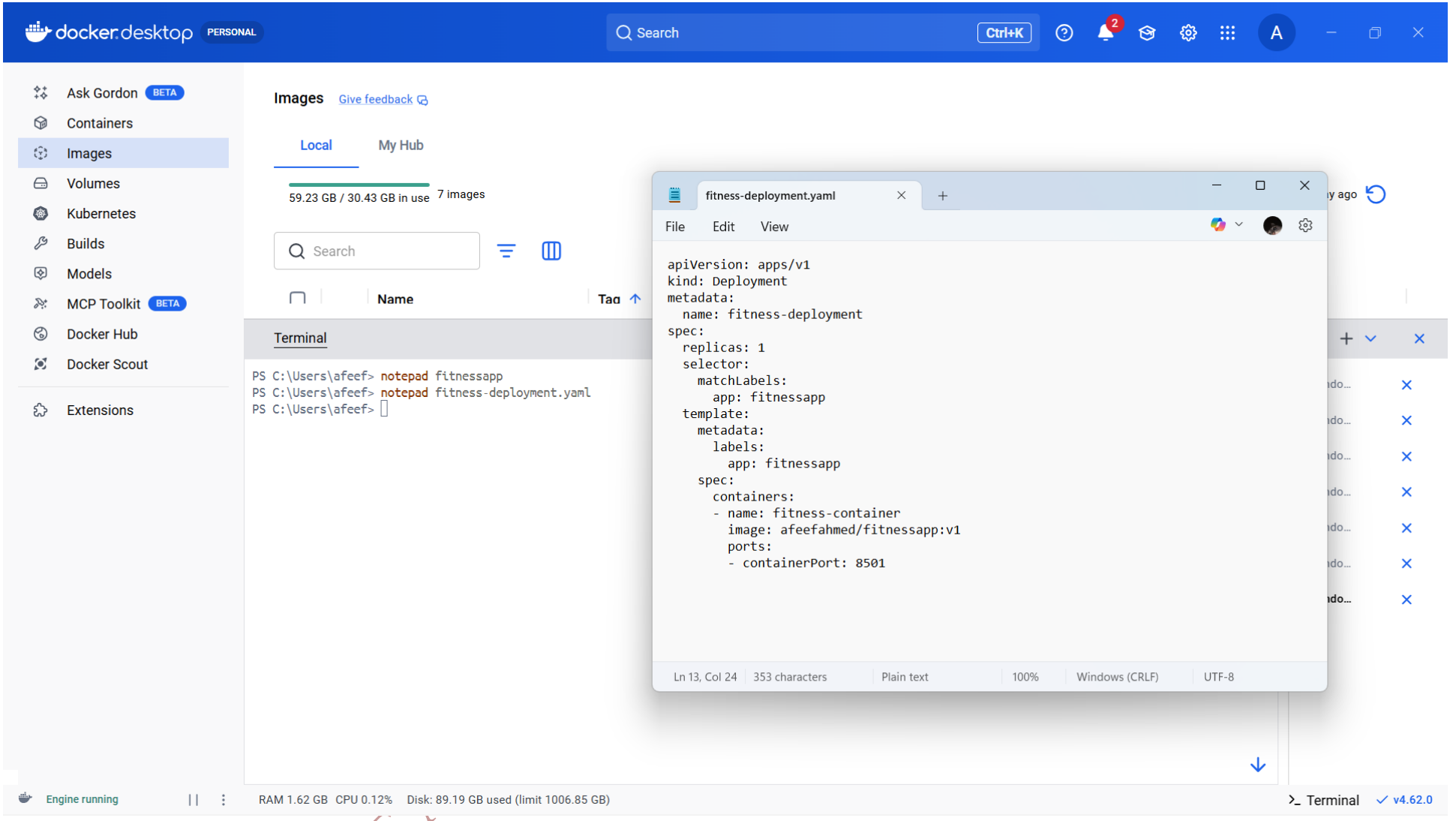


Figure 21: Yaml Defining For Self Pull

SELF PULL EXECUTION

After preparation of the deployment YAML file, it was deployed on the Minikube cluster to deploy the self project image. This step instructs the Kubernetes to create the pod by pulling the image from Docker Hub.

The deployment was done using the following command:

```
kubectl apply -f fitness-deployment.yaml
```

Below are the actions performed by Kubernetes upon the run of the above command:

- Created a deployment resource
- Pulled the container image from Docker Hub (if not already available)
- Started the container in the pod

To confirm that the pod was successfully created and running, the following command was applied:

```
kubectl get pods
```

The output showed one pod related to the deployment in a Running state.

This verified that the self project image was pulled successfully and then run using the Minikube Kubernetes cluster.

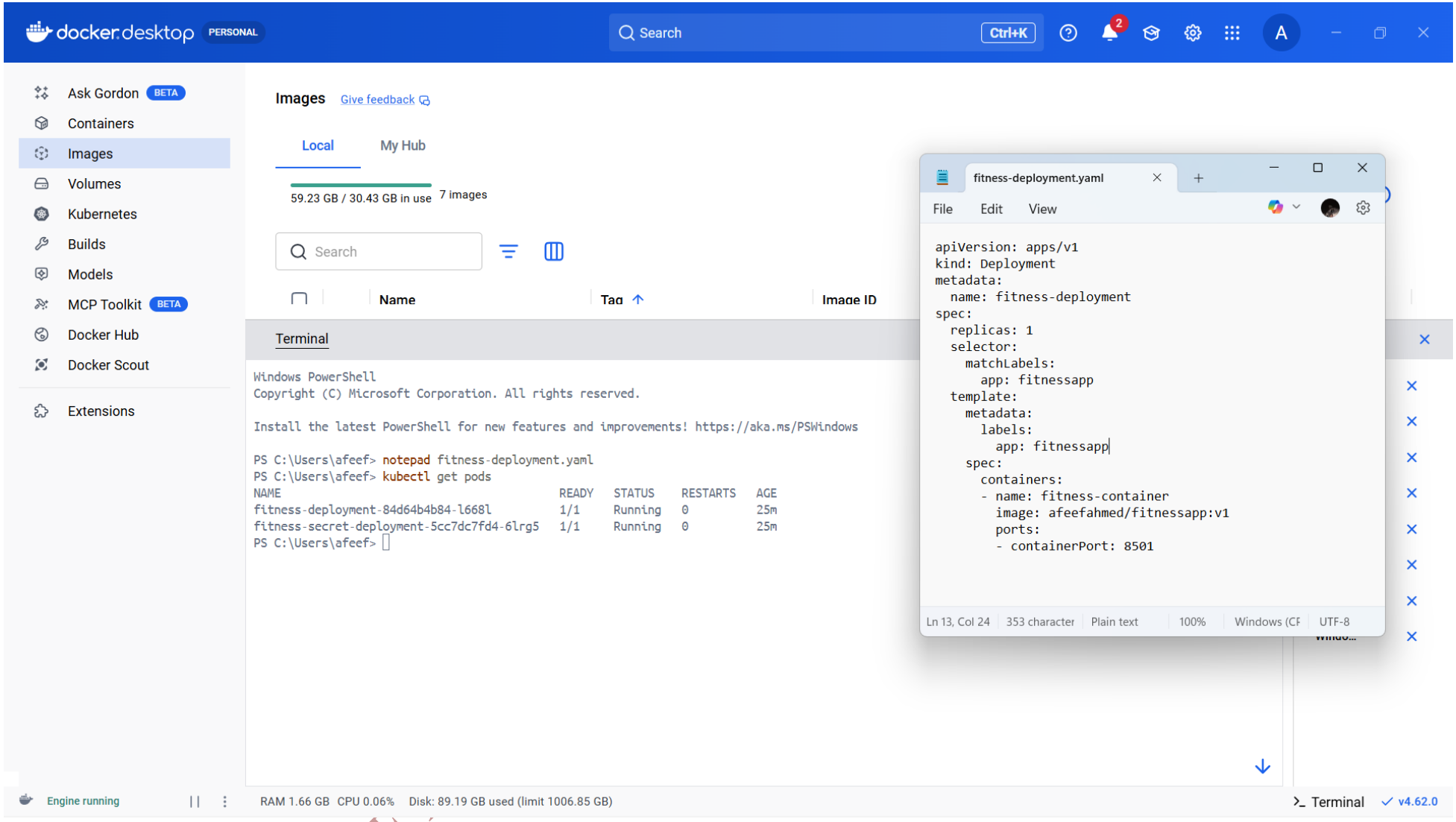


Figure 22: Self Pull Execution

UPSCALE

After deployment of the application in Minikube was successful, the deployment was scaled up to increase the number of running pods. Scaling enables Kubernetes to run multiple instances of the same application, which is useful for:

- Cater for more users
- Increase reliability through redundancy
- Enhance the availability of the application

The deployment was scaled by the following command:

kubectl scale deployment fitness-deployment --replicas=3

This command instructs Kubernetes to run three replicas of the container instead of

one. To scale out the container successfully, the following command was run to

ascertain the same:

kubectl get pods

The output showed three pods in the Running state, all linked to the deployment.

This verified that the application was scaled successfully and multiple instances were running within Minikube cluster.

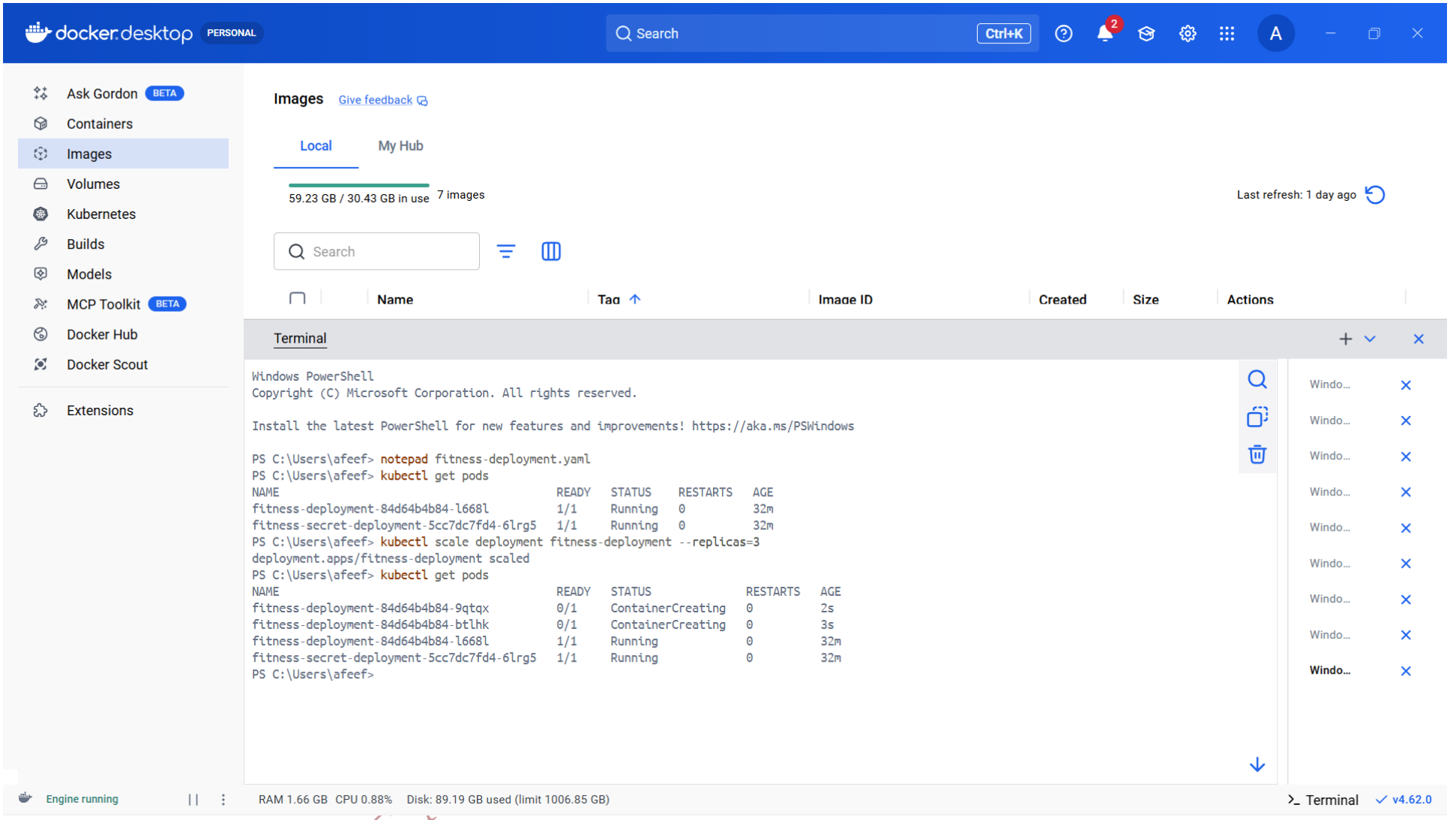


Figure 23: UPSCALE

DOWNSCALE

After increasing the number of replicas, the deployment was scaled down to demonstrate a Kubernetes downscaling. Downscaling also achieves a reduction in the number of active pods, thus releasing unnecessary system resources.

The deployment was downscaled by running the below command:

```
kubectl scale deployment fitness-deployment --replicas=1
```

This command tells Kubernetes to scale down the number of replicas to one.

To ensure that the downscaling operation was carried out successfully, the following command was run:

```
kubectl get pods
```

There was only one Running pod in the output, indicating that the deployment had been scaled down to one replica as intended.

This demonstrated Kubernetes' ability to dynamically create the number of instances of an application required at any given time.

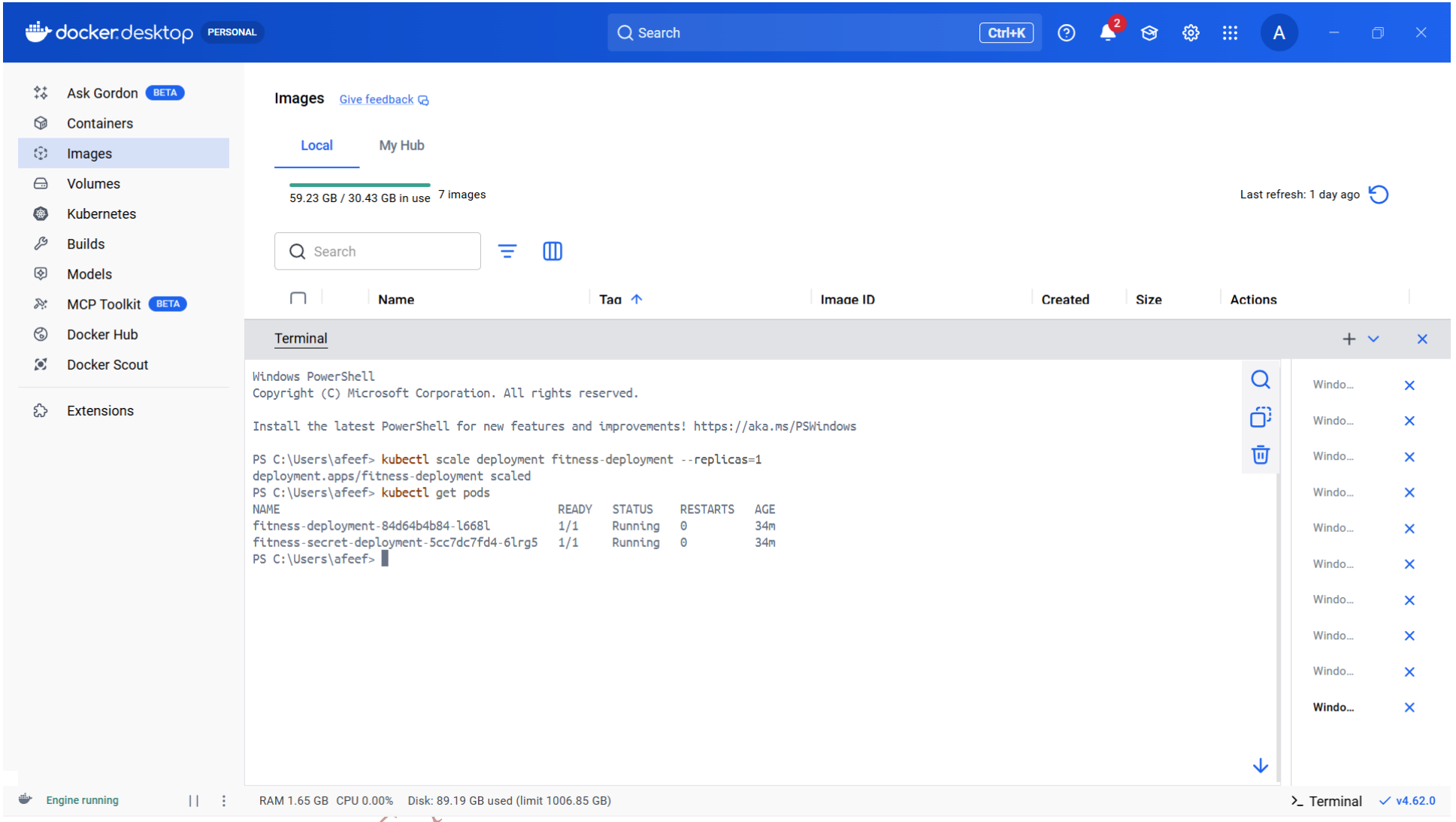


Figure 24: Downscale

SECRET YAML DEFINING

A Kubernetes Secret YAML file was created to hold the configuration data securely within the Minikube cluster. Secrets are useful to store passwords, API keys, or environmental variables without hardcoding them into the application.

The secret configuration was saved into a file called:

```
apiVersion: v1
kind: Secret
metadata:
 name: fitness-secret
type: Opaque
data:
 username: YWRtaW4=
 password: MTIzNA==
```

Here is the configuration:

- A Secret named app-secret is created
- The type Opaque defines a generic secret
- The key-value pair has APP_MODE set to production
- he stringData field accepts non-encoded data, which Kubernetes re-encodes using Base64 internally

This secret can later be referenced in a deployment file, usually as an environment variable in a container. This approach secures the sensitive configuration with a clear distinction from application code.

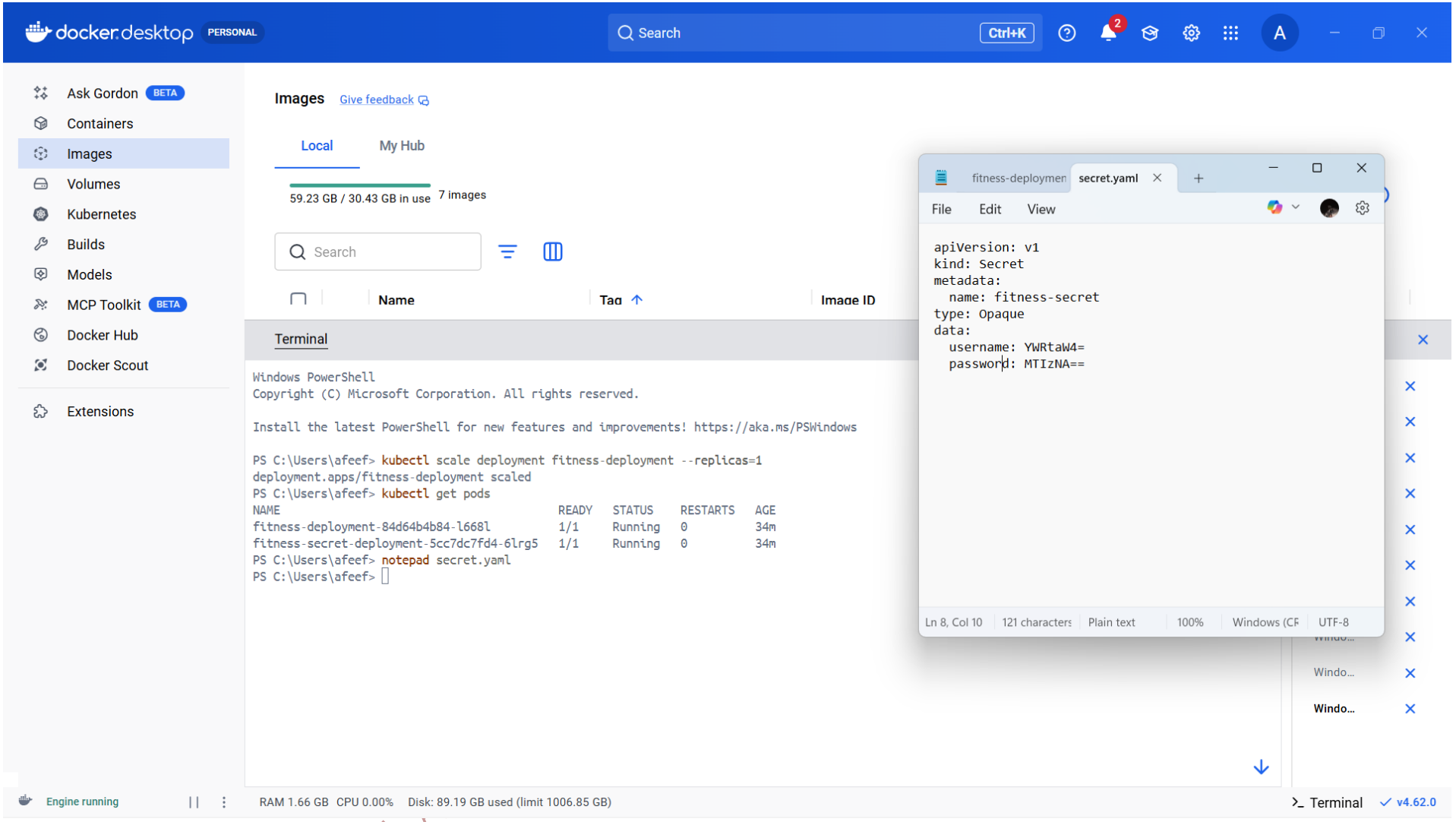


Figure 25: Secret Yaml Defining

NEW DEPLOYMENT DEFINING KIND SECRET

To show how a Secret can be passed into an app via environment variables, a new Kubernetes deployment configuration was done. This deployment injects Secret value into a running container instead of hardcoding it into an application.

The configuration was saved in a file called:

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: fitness-secret-deployment
spec:
 replicas: 1
 selector:
  matchLabels:
   app: fitnesssecret
 template:
  metadata:
   labels:
    app: fitnesssecret
  spec:
   containers:
   - name: fitness-container
    image: afeefahmed/fitnessapp:v1
    ports:
    - containerPort: 8501
    env:
    - name: USERNAME
     valueFrom:
      secretKeyRef:
       name: fitness-secret
       key: username
    - name: PASSWORD
     valueFrom:
      secretKeyRef:
       name: fitness-secret
       key: password
```


Explanation of the Configuration

- A deployment named fitness-secret-deployment is created
- t runs a single replica container based on the self project image from Docker Hub.
- The selector and labels are in place to ensure the proper identification and handling of the associated pods by Kubernetes.
- The vital part is the env section, where:
    - There is a set definition for the environment variable APP_MODE.
    - It is not specified directly in the file..
    - However, the value is not put directly in the file but retrieved securely from the Kubernetes Secret called app-secret.
    - The secretKeyRef defines the secret name and the particular key necessary for access.

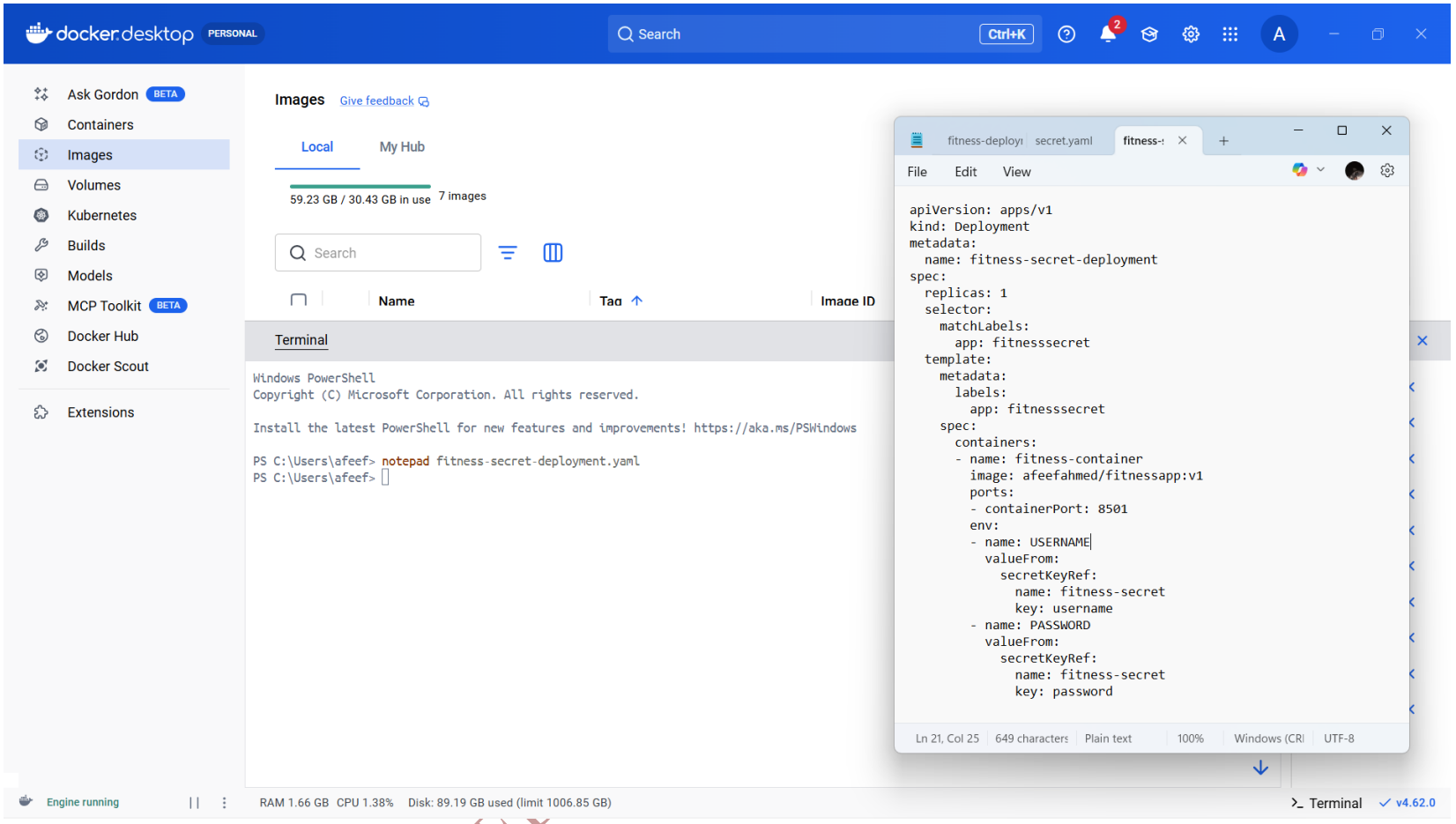


Figure 26: New Deployment Defining Kind Secret

GETTING SECRET FOR CHECK

Once the Kubernetes Secret had been generated, its presence and contents were validated by using various commands in order to prove that the Secret had been correctly stored within the Kubernetes cluster. Several commands can be used to inspect Kubernetes Secrets.

In order to list all the secrets stored within the cluster, the following command was run:

kubectl get secrets

As can be seen in the output above, there is indeed a secret called app-secret, which has been created.

In order to get additional information regarding the metadata associated with this secret, the following command was run:

kubectl describe secret app-secret

Additional metadata obtained from this secret include:

- Secret name
- The type of secret
- The number of keys stored in it

Finally, in order to validate that the entire content of this secret had been correctly stored, the following command was run:

kubectl get secret app-secret -o yaml

The stored values are shown in Base64 encoded format. In order to actually get the value of the APP_MODE field, the following command was used:

kubectl get secret app-secret -ojsonpath="{.data.APP_MODE}" | base64 -- decode

Conclusion

The verification steps shown in this output demonstrate that:

- The Kubernetes secret has been successfully created
- It has been securely stored within the Kubernetes cluster
- he values stored within it can be accessed when necessary

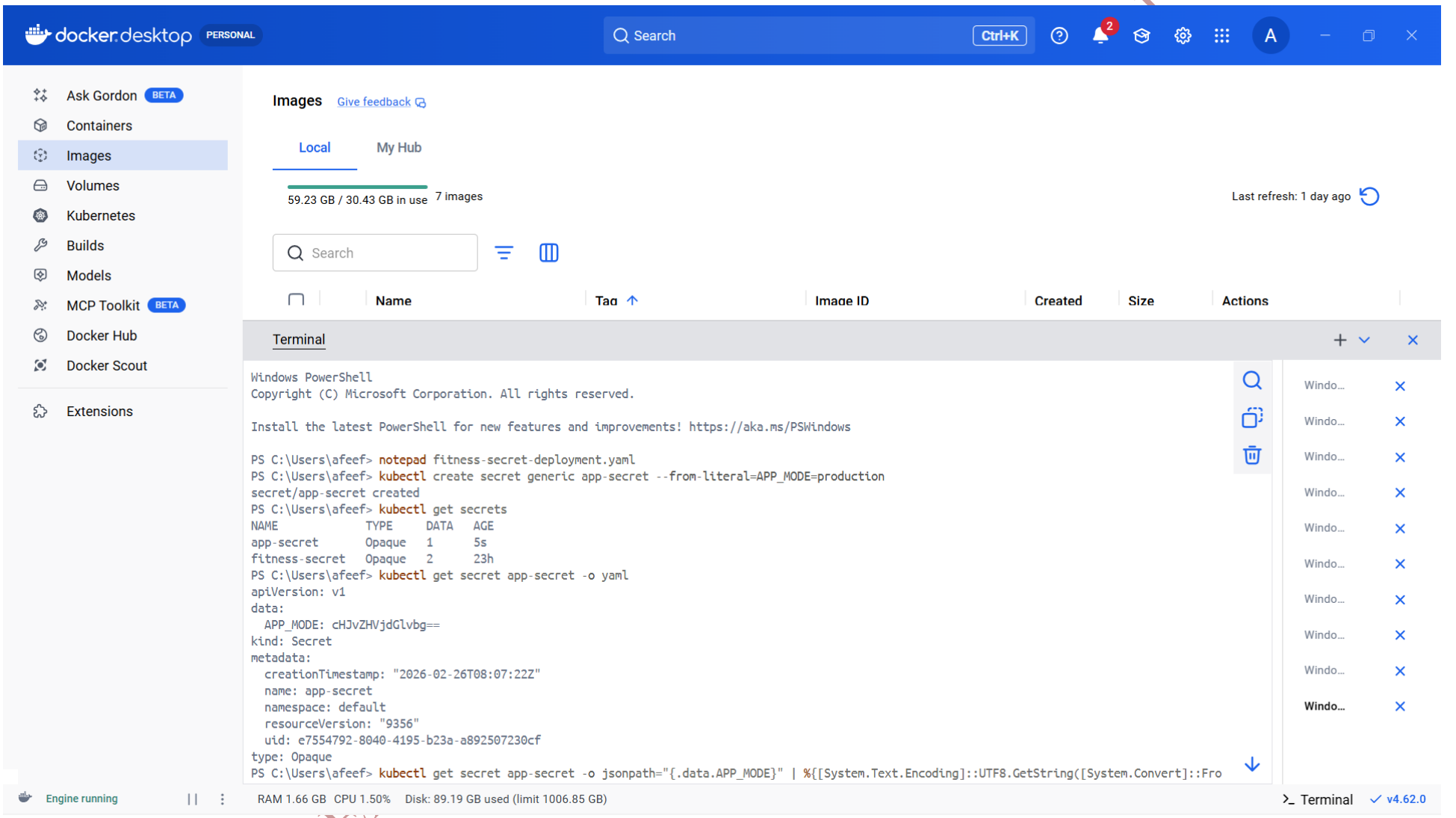


Figure 27: Getting Secret For Check

SECRET DEPLOYMENT

Once the Kubernetes secret had been created, its creation and storage were validated in order to demonstrate proper creation. There are many commands available within Kubernetes that could verify the secret in question.

Firstly, the following command was used in order to list all the secrets currently available within the Kubernetes cluster:

```
kubectl get secrets
```

It shows that there is a secret called app-secret available within the cluster.

The following command was used in order to display additional metadata:

```
kubectl describe secret app-secret
```

Additional metadata revealed by this secret include:

- Secret name
- Secret type (Opaque)
- Keys stored within the secret

In order to verify the data stored within the secret, the following command was used:

```
kubectl get secret app-secret -o yaml
```

Note that the stored values are in Base64 format, because Kubernetes encrypts the secret data automatically.

kubectl get secret app-secret -o jsonpath="{.data.APP_MODE}" | base64 -- decode:

```
kubectl get secret app-secret -o jsonpath="{.data.APP_MODE}" | base64 -- decode
```

Conclusion

The verification steps demonstrated above confirm that:

- The Kubernetes secret has been successfully created
- It has been securely stored within the Kubernetes cluster
- The values stored within it can be accessed whenever required

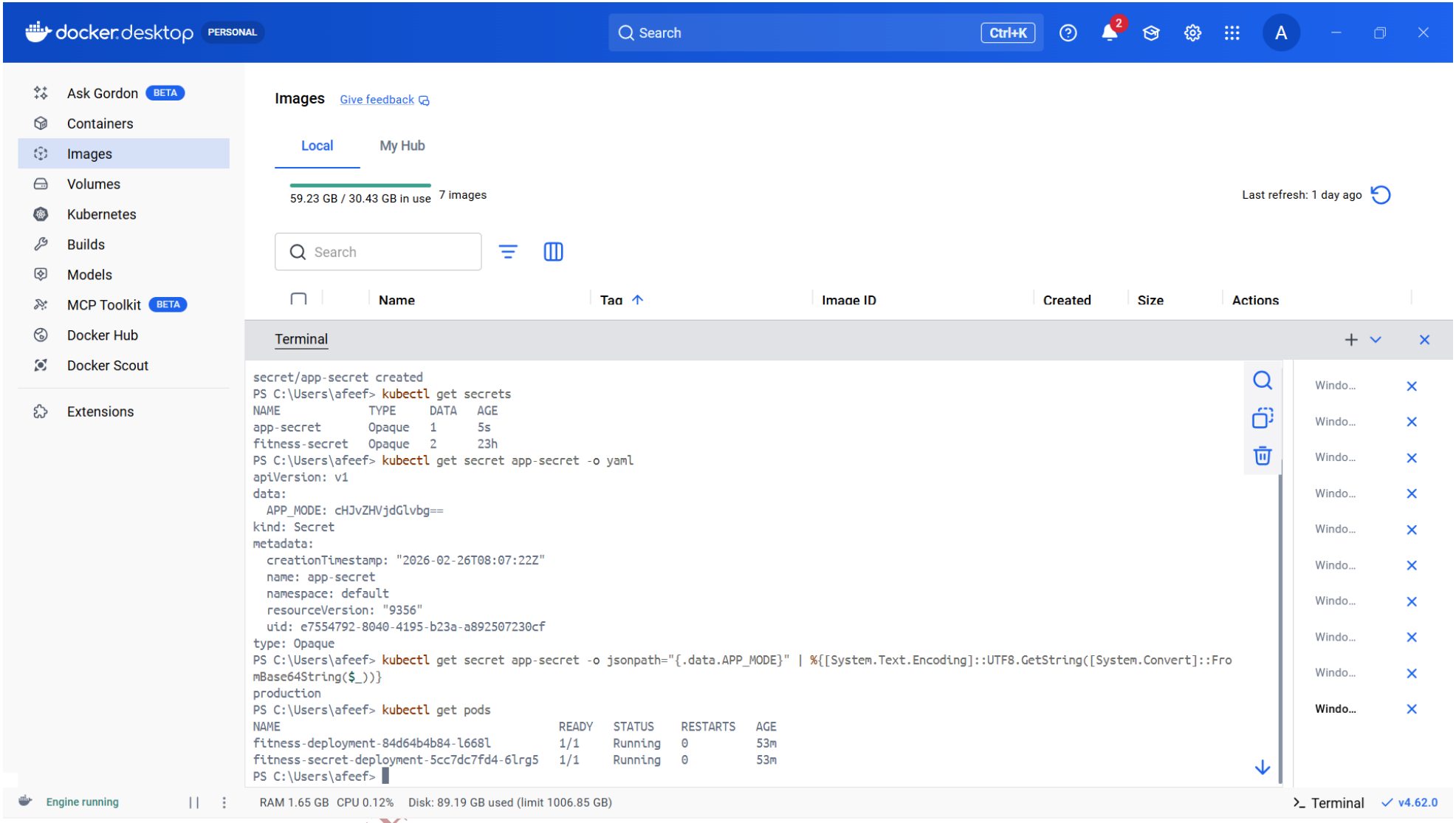


Figure 28: Secret Deployment

UPSCALE

In order to further demonstrate the capabilities of the application and the platform, the deployment was upscaled to increase the number of active pods. Upscaling makes the application more reliable and proves that the Kubernetes platform can handle increasing workloads.

The deployment is upscaled using the following command:

```
kubectl scale deployment fitness-secret-deployment --replicas=3
```

This instruction tells Kubernetes to make three copies of the pods related to this deployment.

To verify whether the deployment is upscaled successfully, the following command is executed:

```
kubectl get pods
```

As a result, three pods with the status of Running are observed with the name of fitness-secret-deployment.

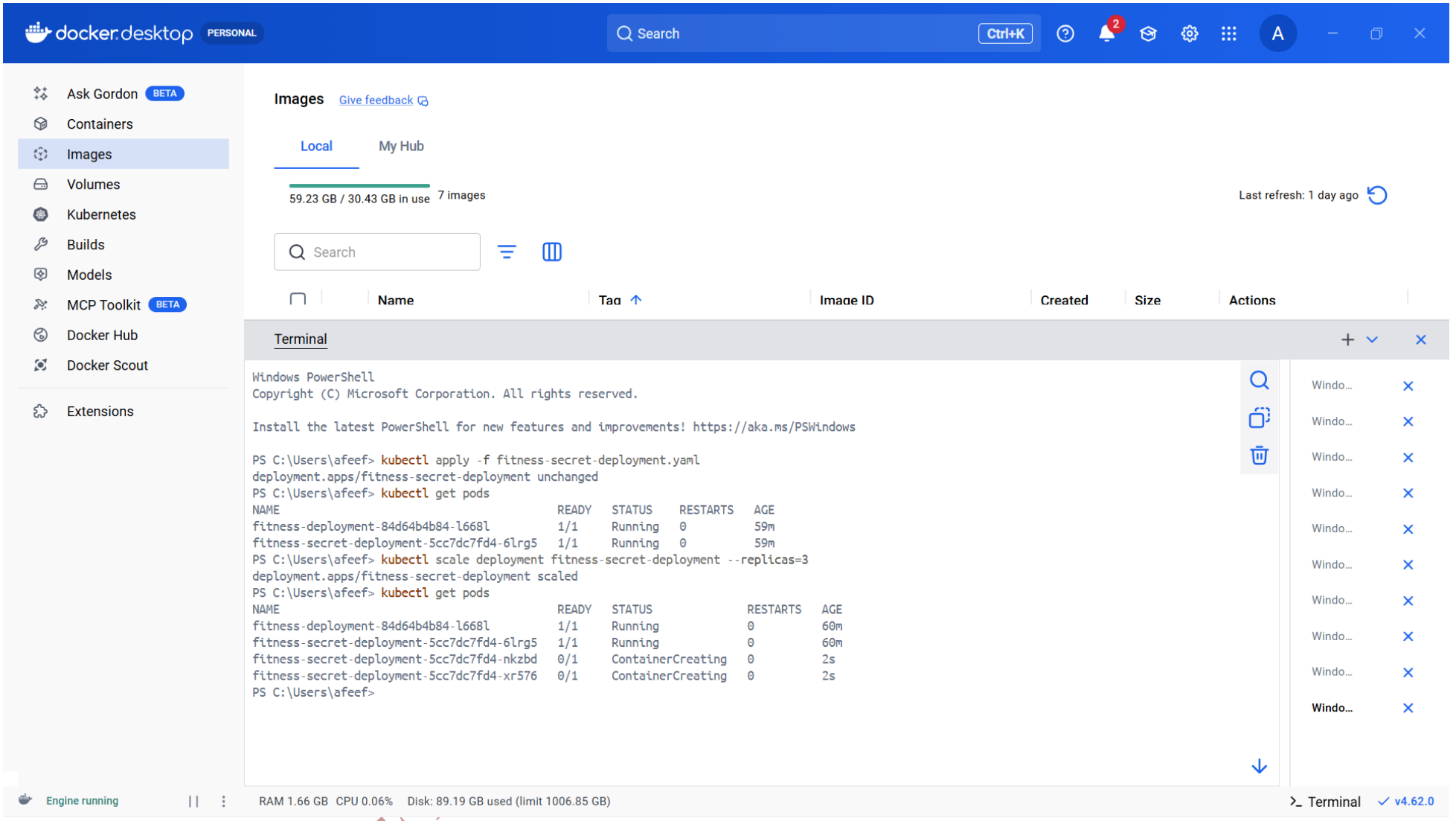


Figure 29: UPSCALE

DOWNSCALE

Having demonstrated the upscaled deployment, the deployment was downscaled, which means that fewer pods will be running.

Downscaling helps optimize the usage of resources and prove that Kubernetes can effectively adjust its capacity for an application.

The deployment is downscaled using the following command:

```
kubectl scale deployment fitness-secret-deployment --replicas=1
```

kubectl scale deployment fitness-secret-deployment --replicas=1

This command instructs Kubernetes to scale down to one copy of the deployment.

To verify whether the operation was successful, the following command is executed:

```
kubectl get pods
```

As a result, one pod with the status of Running is observed.

This demonstrates Kubernetes' ability to efficiently manage scaling operations while maintaining application stability.

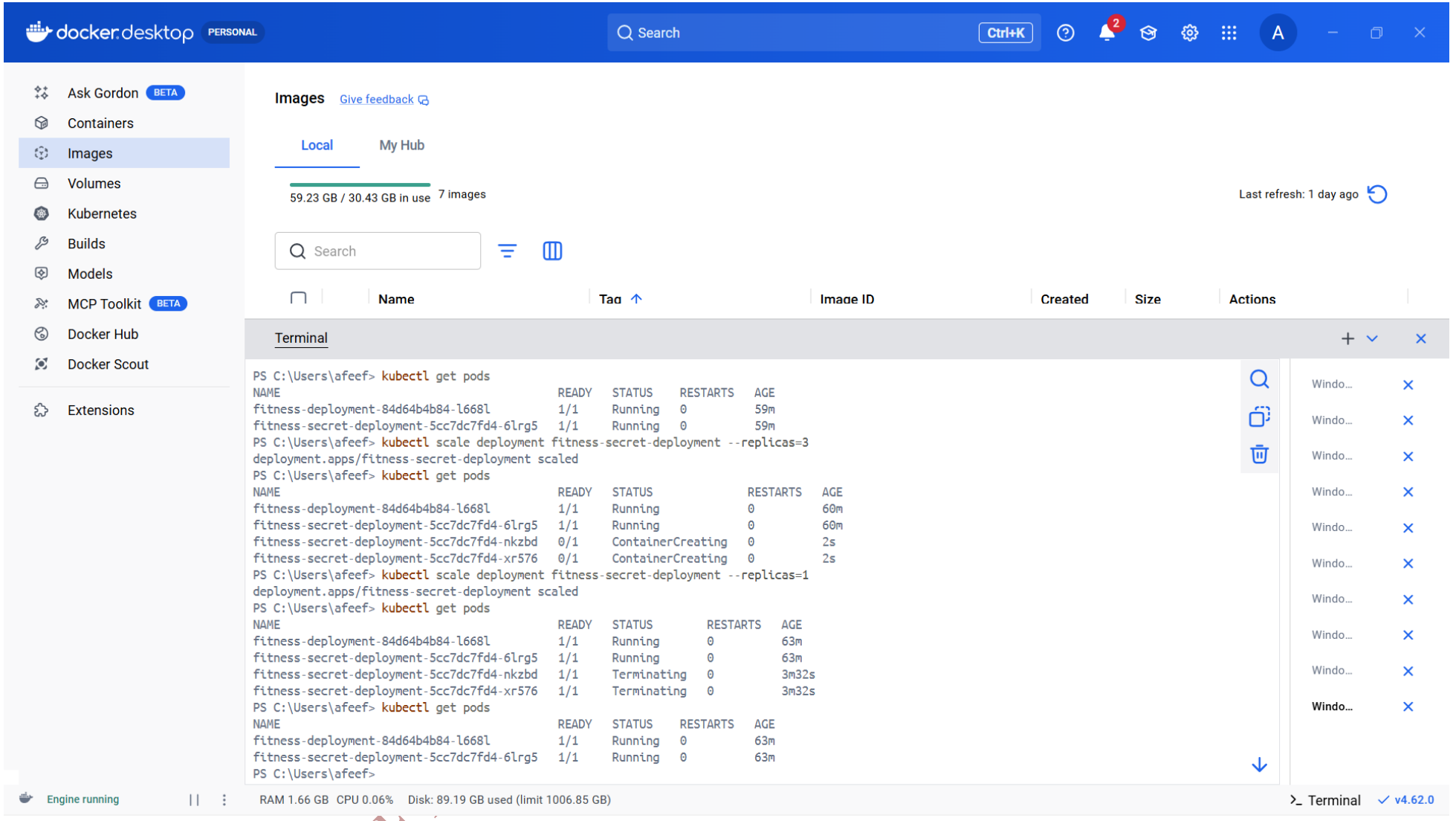


Figure 30: Downscaling

ALL FINAL DEPLOYMENT

Upon successfully completing the process of deployment, performing scaling operations, and adding secrets to the application, it is important to check all of the deployments within the cluster. This will help ensure that the applications are successfully deployed in the cluster.

The following command will show all of the deployments within the cluster:

```
kubectl get deployments
```

The output will show the following information regarding the deployment:

- Name of the deployment
- Desired number of pods
- Available pods
- status

There are deployments such as:

- **Fitness deployment**
- **Fitness secret deployment**
- **Hume rainfall deployment**

All of these deployments have the necessary number of pods running in the READY state.
To receive more information about any deployment, the following command may be executed:

```
kubectl describe deployment fitness-secret-deployment
```

The output will show:

- Information about the pod template
- List of replica sets
- Scaling events and updates.

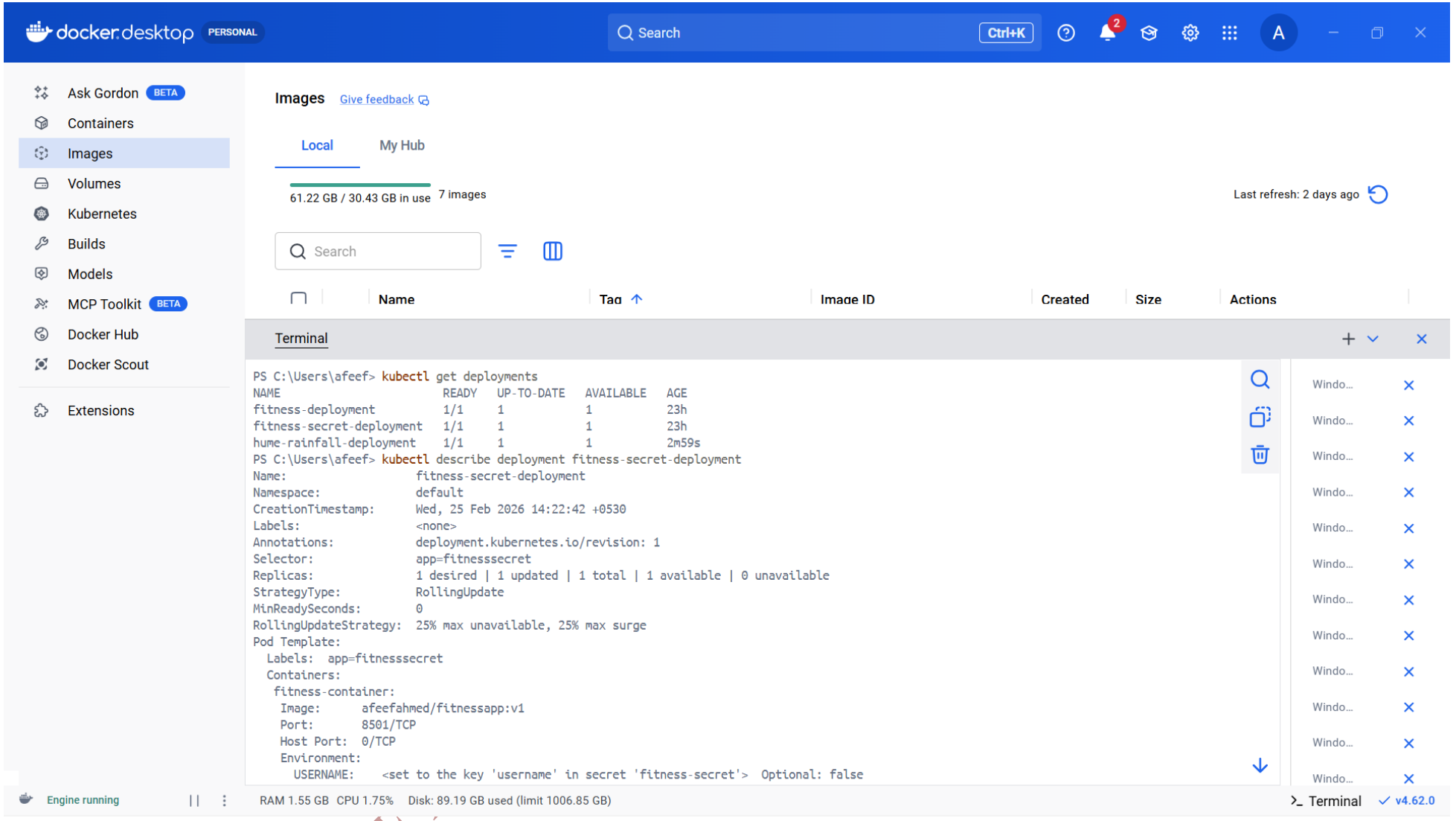


Figure 31: All Final Deployment

ALL PODS AT THE LAST

Once all deployment, scaling, and secret operations have been completed, the status of all pods in the Minikube cluster was then assessed to determine whether the applications have run smoothly and have been managed effectively.

Listing all running pods is made possible using the following command:

```
kubectl get pods
```

The following are some features displayed by the output of this command:

- Name of the Pod
- Readiness of the Pod
- Status of the Pod
- Number of restarts of the Pod
- Age of the Pod

Examples of pods include those associated with:

- **fitness-deployment**
- **fitness-secret-deployment**
- **hume-rainfall-deployment**

All the pods in this assessment had the status “Running,” with the correct number of running containers (e.g., 1/1).

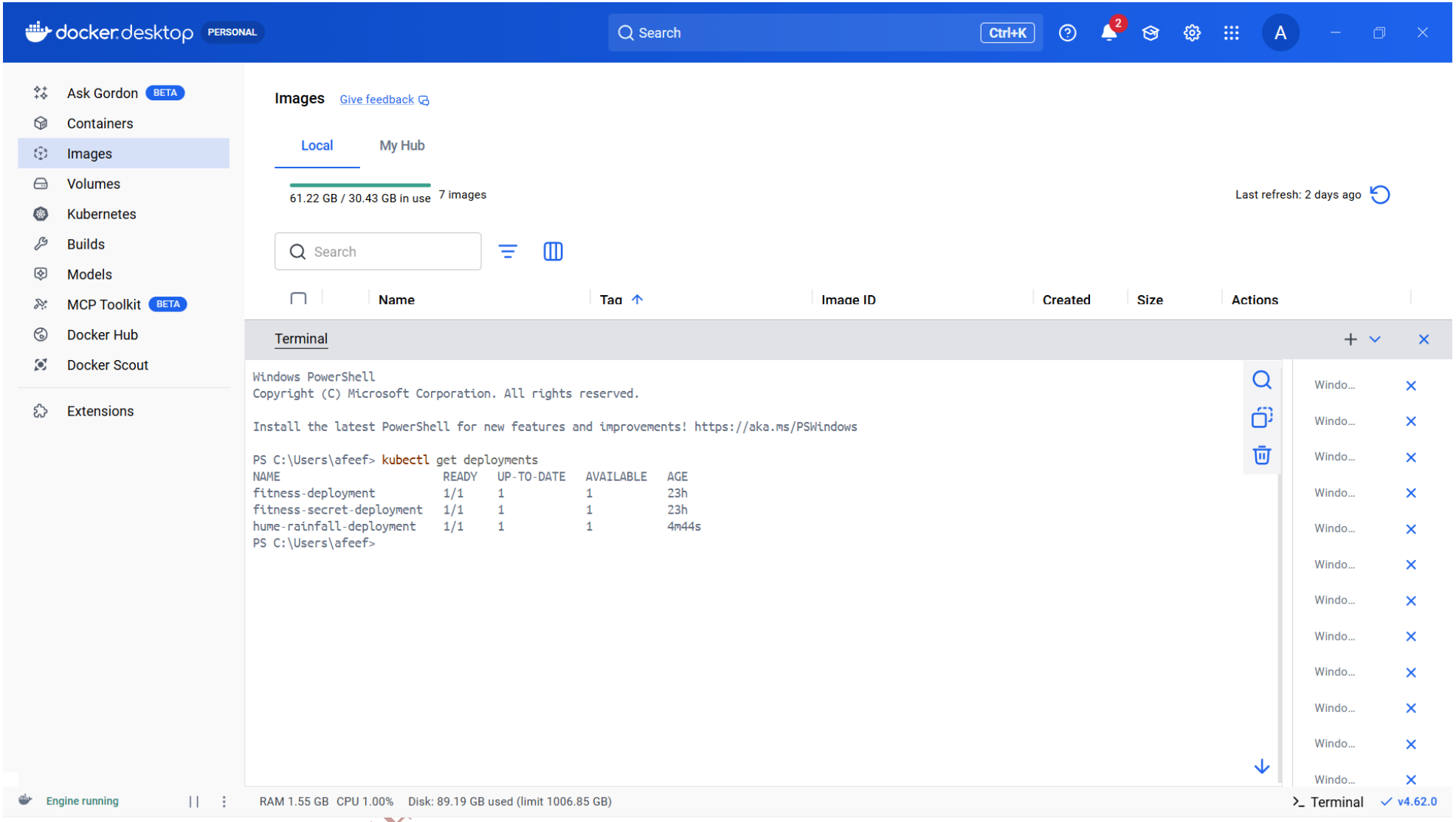


Figure 32: All Pods

Task 5 : Working with SLURM - Scripting & Kubeflow

Date: 27-03-2026

PART 1 — SLURM

Procedure

The purpose of this study was to analyze how job scheduling is performed using the SLURM workload manager in a container environment. To perform the experiment, the SLURM cluster was created based on the Docker platform that comprised a controller node as well as compute nodes. In order to test SLURM functionality, a batch job consisting of matrix addition implemented with Python was run.

Step-by-Step Explanation

The SLURM cluster setup was done using Docker-based cluster setup.

```
git clone https://github.com/giovtorres/slurm-docker-cluster.git
cd slurm-docker-cluster
docker compose up -d
```

These clusters running were checked by doing the following commands:

```
docker ps
docker exec -it slurmctld bash
sinfo
```

A simple Python script to add two matrices:

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

result = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]
print(result)
```

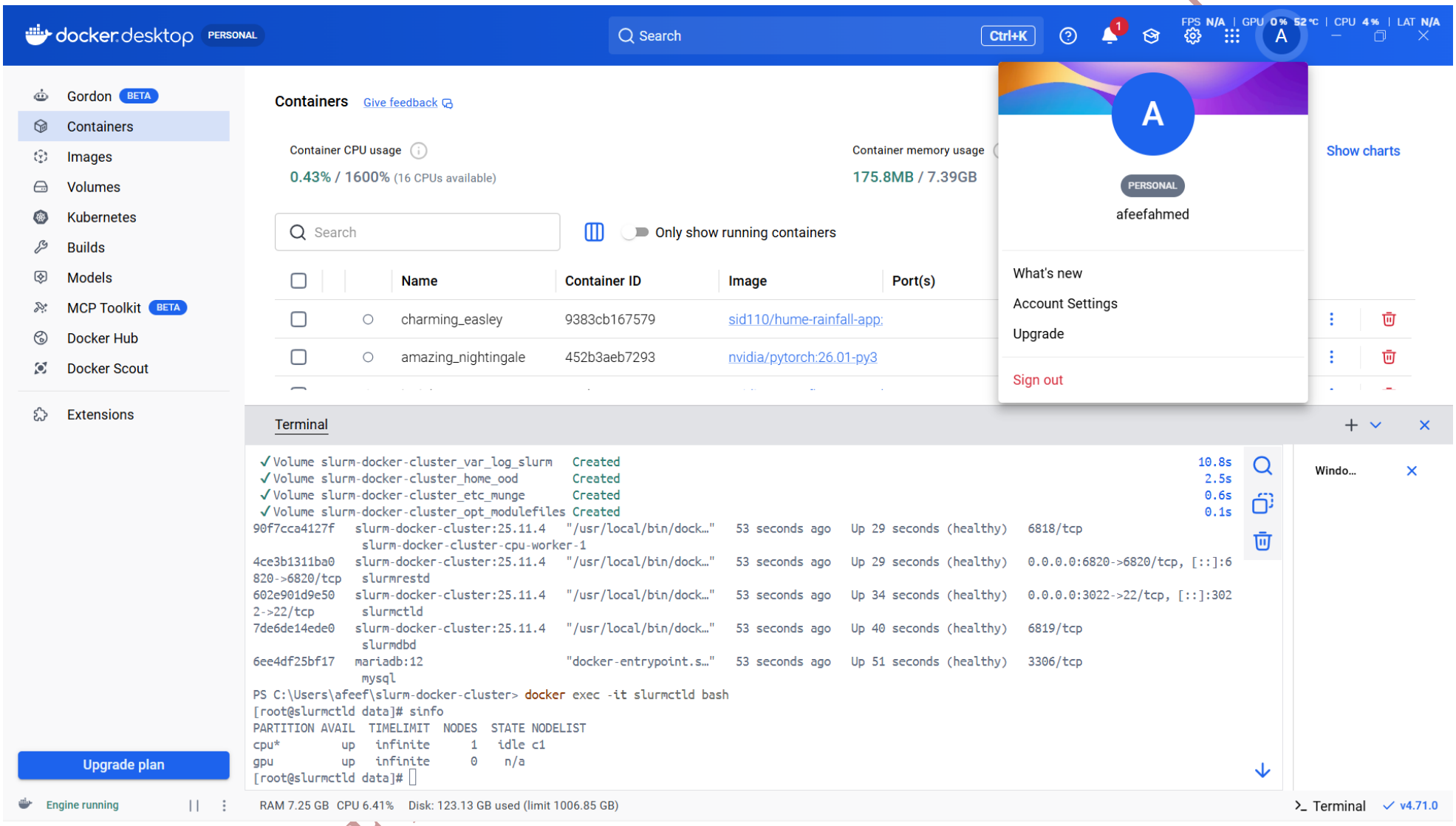


Fig 33: Docker Containers Running (SLURM Cluster)

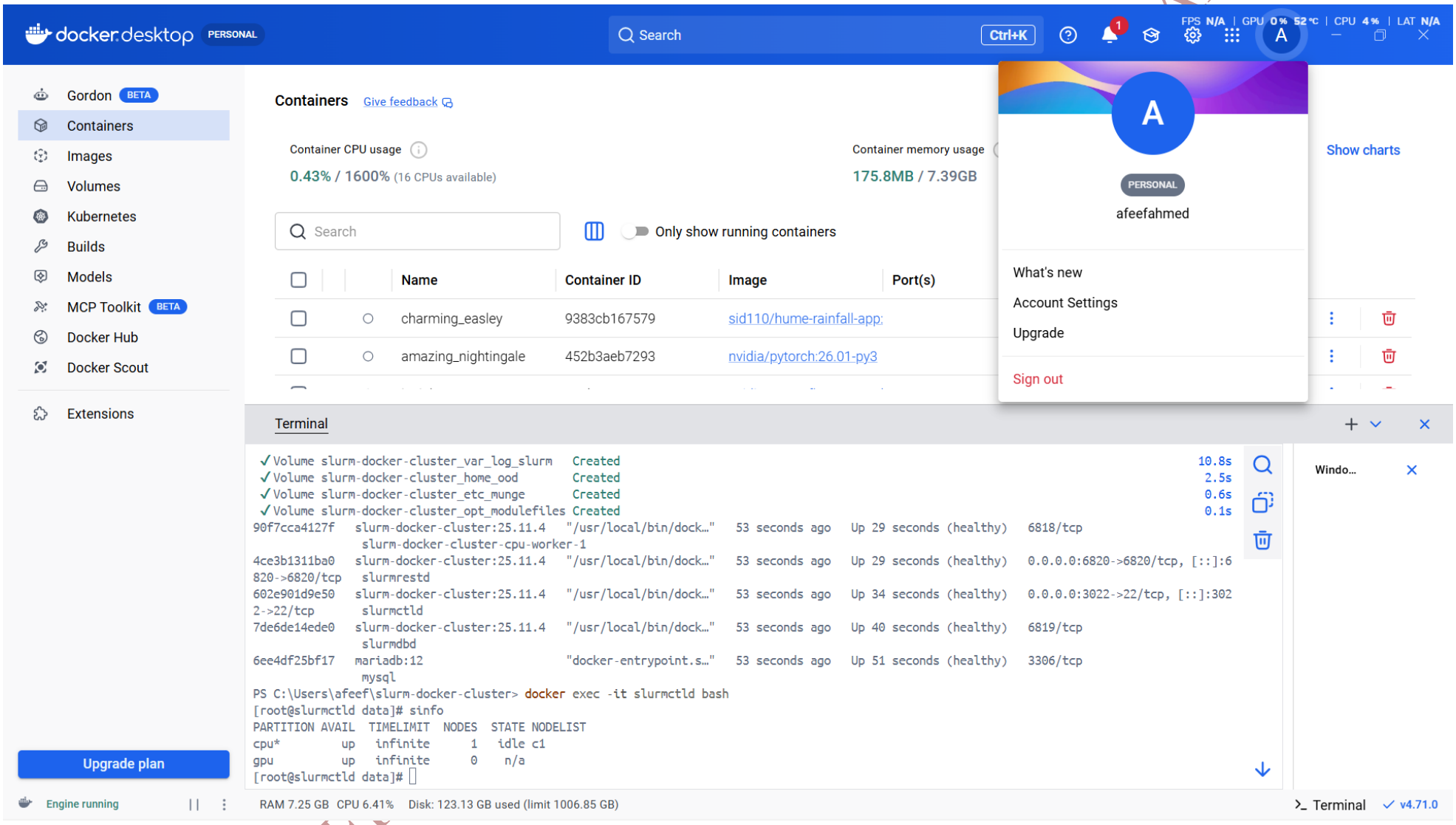


Fig 34: SLURM Cluster Status (sinfo)

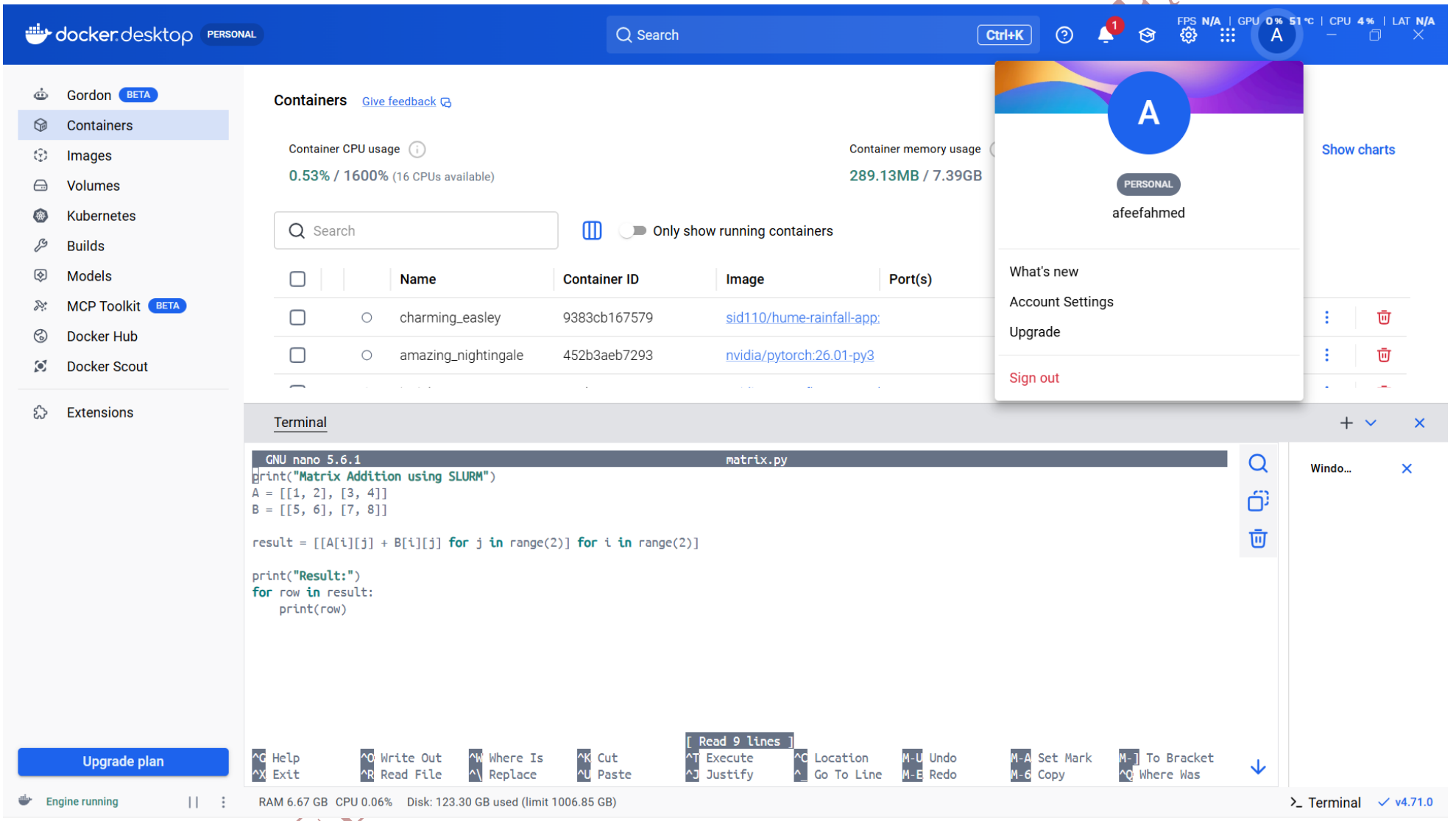


Fig 35: Matrix Program in SLURM

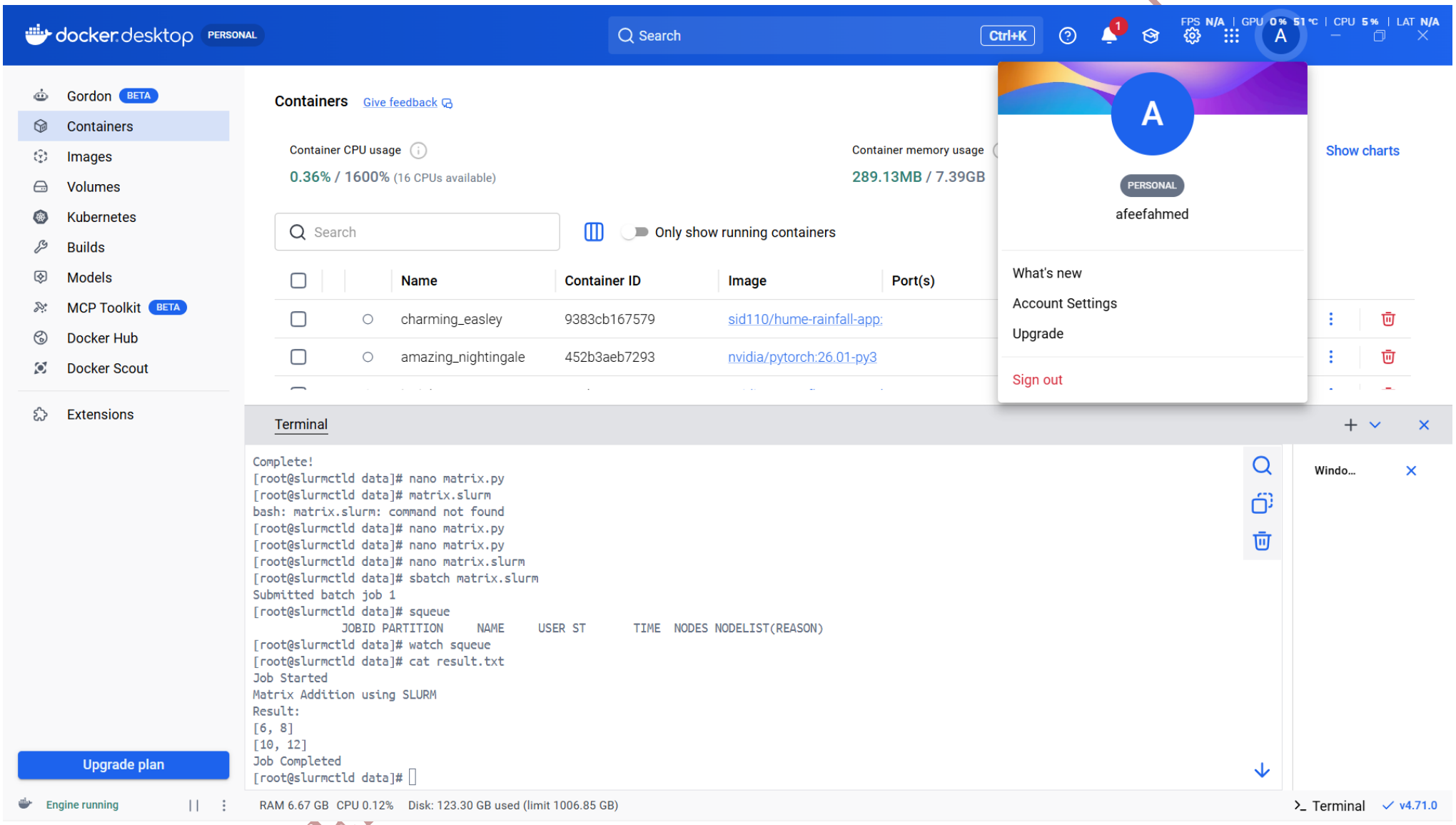


Fig 36: SLURM Job Output

A batch script with SLURM commands was developed as follows:

```
#!/bin/bash
#SBATCH --job-name=matrix_add
#SBATCH --output=result.txt
python3 matrix.py
```

The script was successfully run:

```
sbatch matrix.slurm
cat result.txt
```

As seen from the result, the experiment succeeded as SLURM was able to schedule and execute the command in question.

PART 2 — Kubeflow

Procedure

The goal of the experiment was workflow automation with Kubeflow Pipelines. Thus, a matrix addition component had to be created and compiled into a YAML file describing the workflow process.

Step-by-Step Explanation

Firstly, we installed Kubeflow Pipelines SDK to allow us to define our workflows:

```
pip install kfp
```

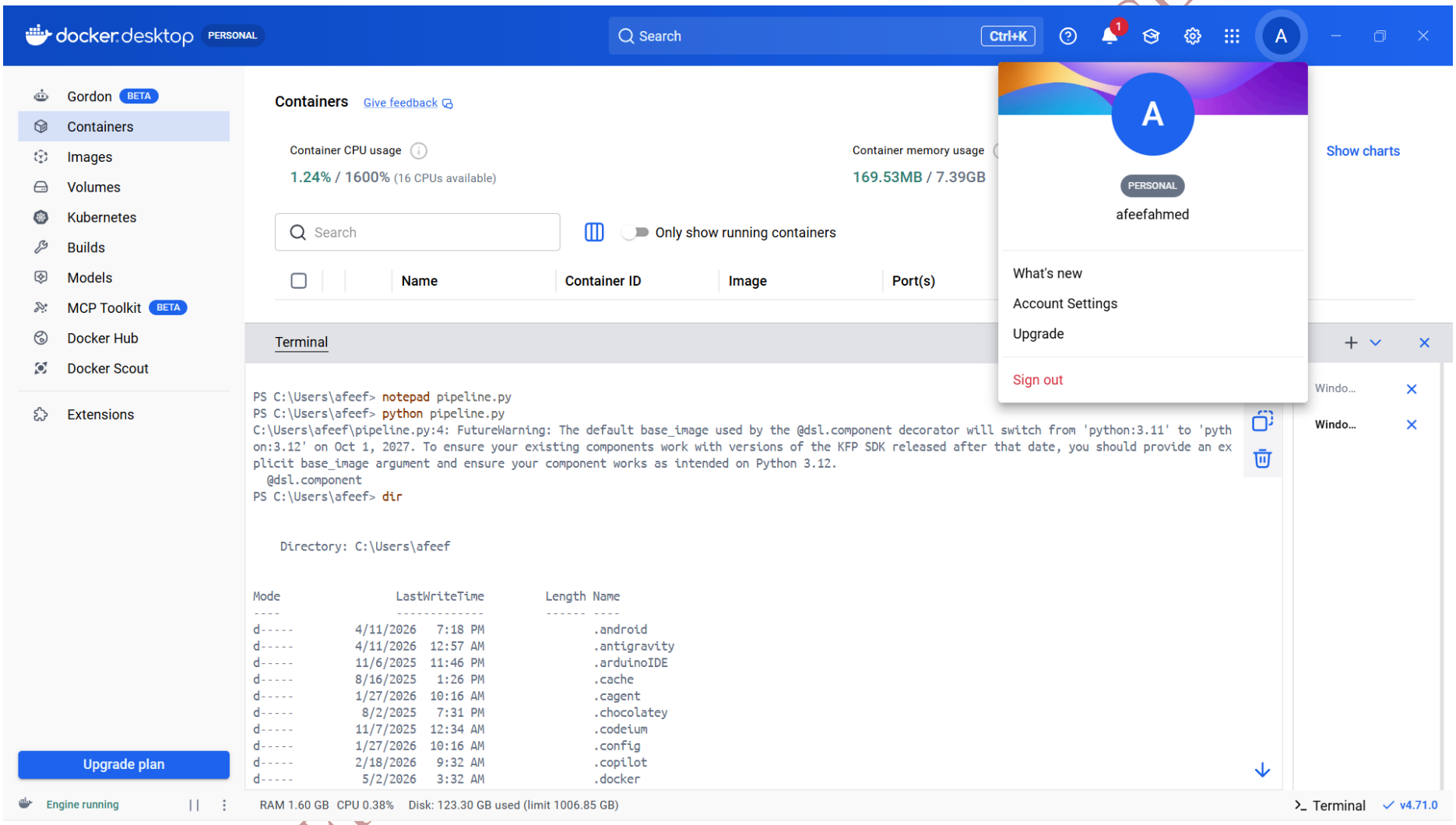


Fig 37:Kubeflow Pipeline Script

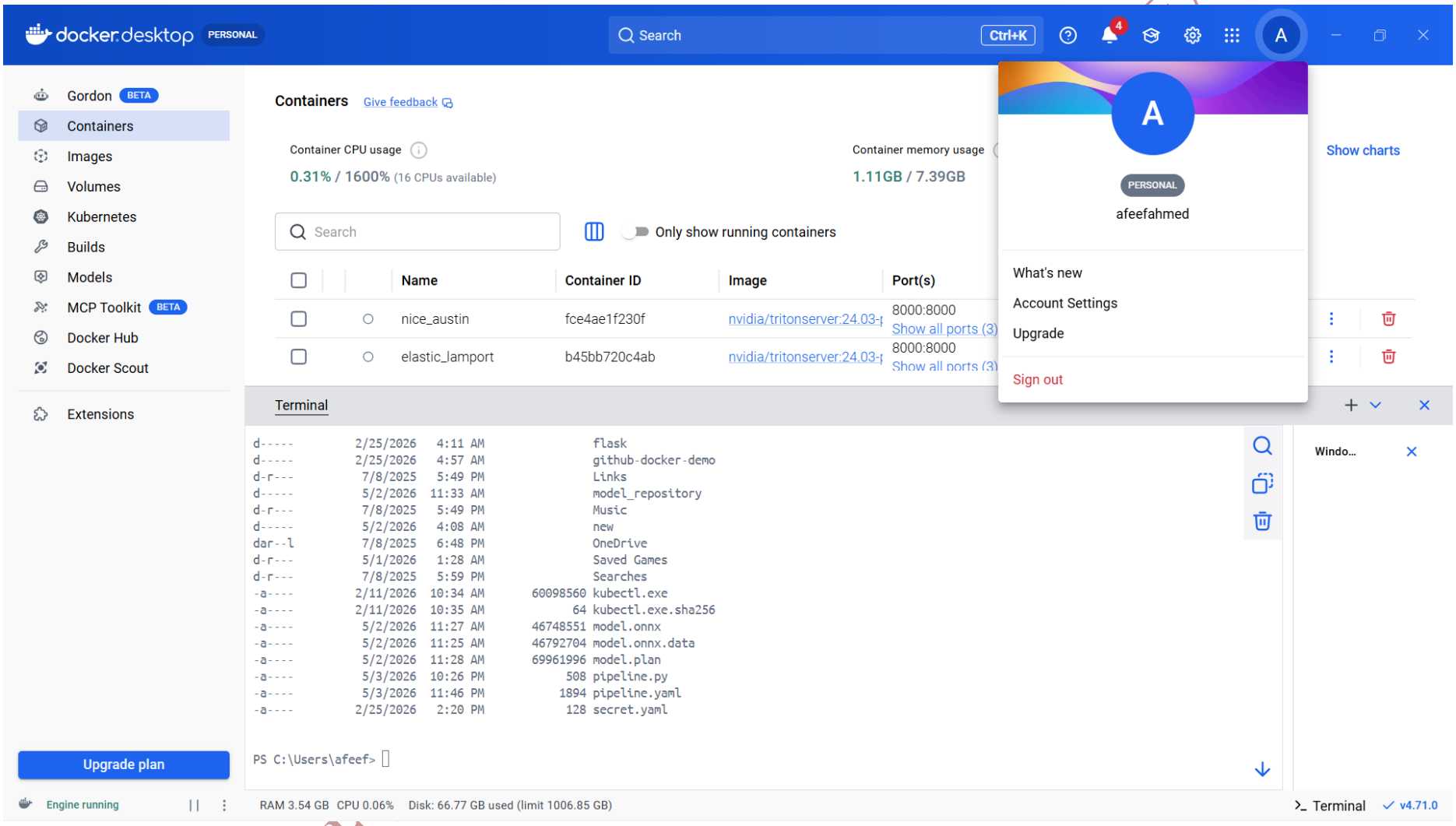


Fig 38:Kubeflow Pipeline YAML Listing

A script in Python was developed to implement a reusable module to perform matrix addition. This involved building logic for matrix addition:

```
from kfp import dsl
from kfp.compiler import Compiler

@dsl.component
def matrix_addition():
    print("Matrix Addition using Kubeflow")

    A = [[1, 2], [3, 4]]
    B = [[5, 6], [7, 8]]

    result = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]

    print("Result:")
    for row in result:
        print(row)

@dsl.pipeline(name="matrix-addition-pipeline")
def pipeline():
    matrix_addition()
```

Execution:

```
python pipeline.py
```

The resulting pipeline.yaml indicated that the pipeline was successfully created..

Task 6 : Using Jupyter Notebook and PyTorch for Tensor Operations and Data-Centric Application

Date: 08-04-2026

Experiment Procedure

This experiment was designed to understand tensor operations and create a data-centric application using Jupyter Notebook and PyTorch. The following steps were executed to complete the task.

Step-by-Step Explanation

Installation of required libraries and opening of notebook.
Performance of tensor operations using Python script.
Creation of data-centric application using linear regression model.
Experiment Steps Explanation

In order to perform the experiment, all the required libraries were installed and Jupyter Notebook was opened:

```
pip install notebook torch torchvision
jupyter notebook
```

After opening the notebook, two tensors were created and several operations like addition, subtraction, element-wise multiplication, and matrix multiplication were executed on them.

```
import torch

a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])

print("Addition:\n", a + b)
print("Subtraction:\n", a - b)
print("Element-wise Multiplication:\n", a * b)
print("Matrix Multiplication:\n", torch.matmul(a, b))
```

Further, the data-centric application was prepared by creating a dataset containing linearly related variables and defining a linear model using PyTorch.

```
import torch.nn as nn
import torch.optim as optim

x = torch.tensor([[1.0], [2.0], [3.0], [4.0]])
y = torch.tensor([[3.0], [5.0], [7.0], [9.0]])

model = nn.Linear(1, 1)
```

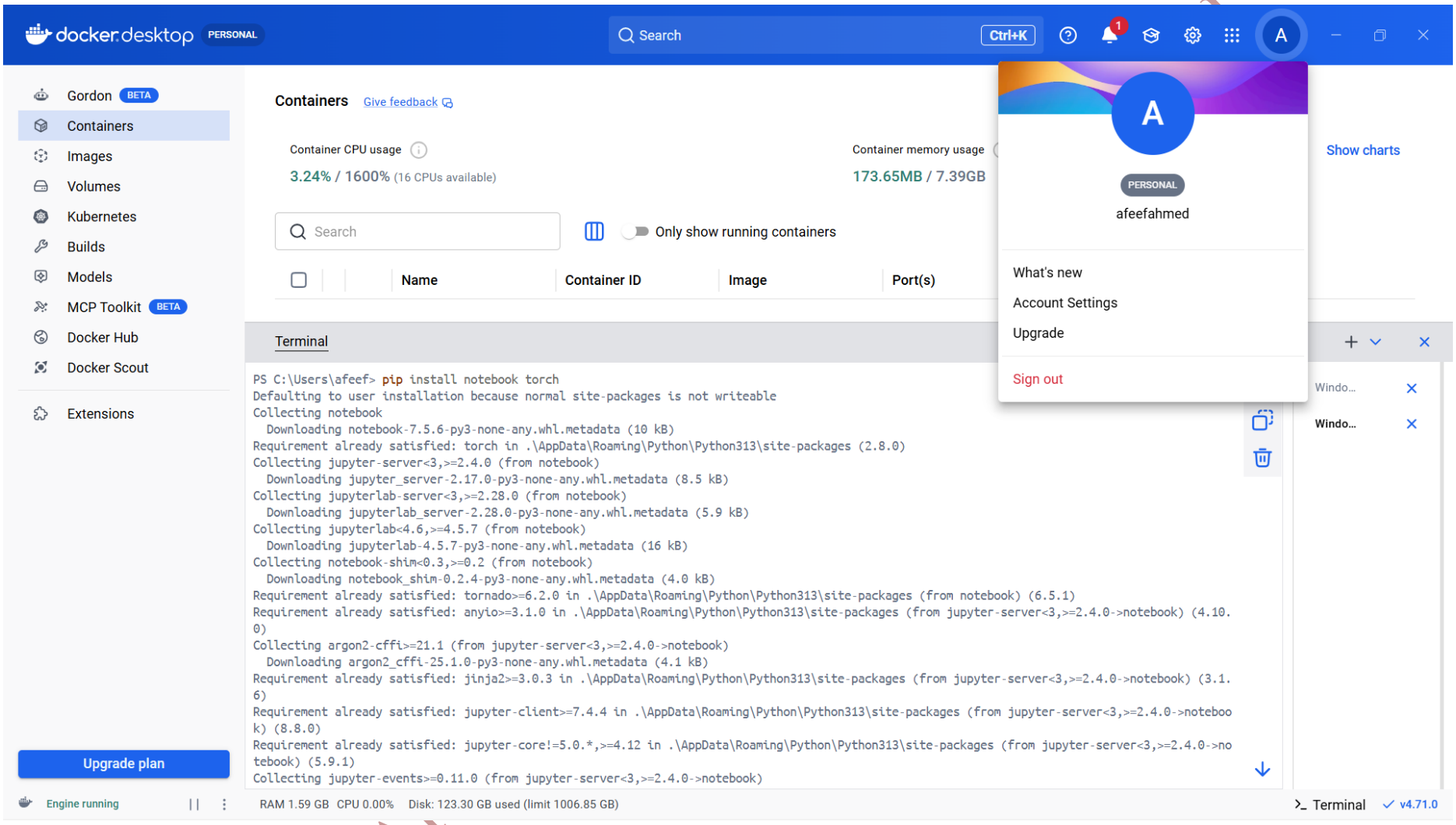


Fig 39:Jupyter Installation Process

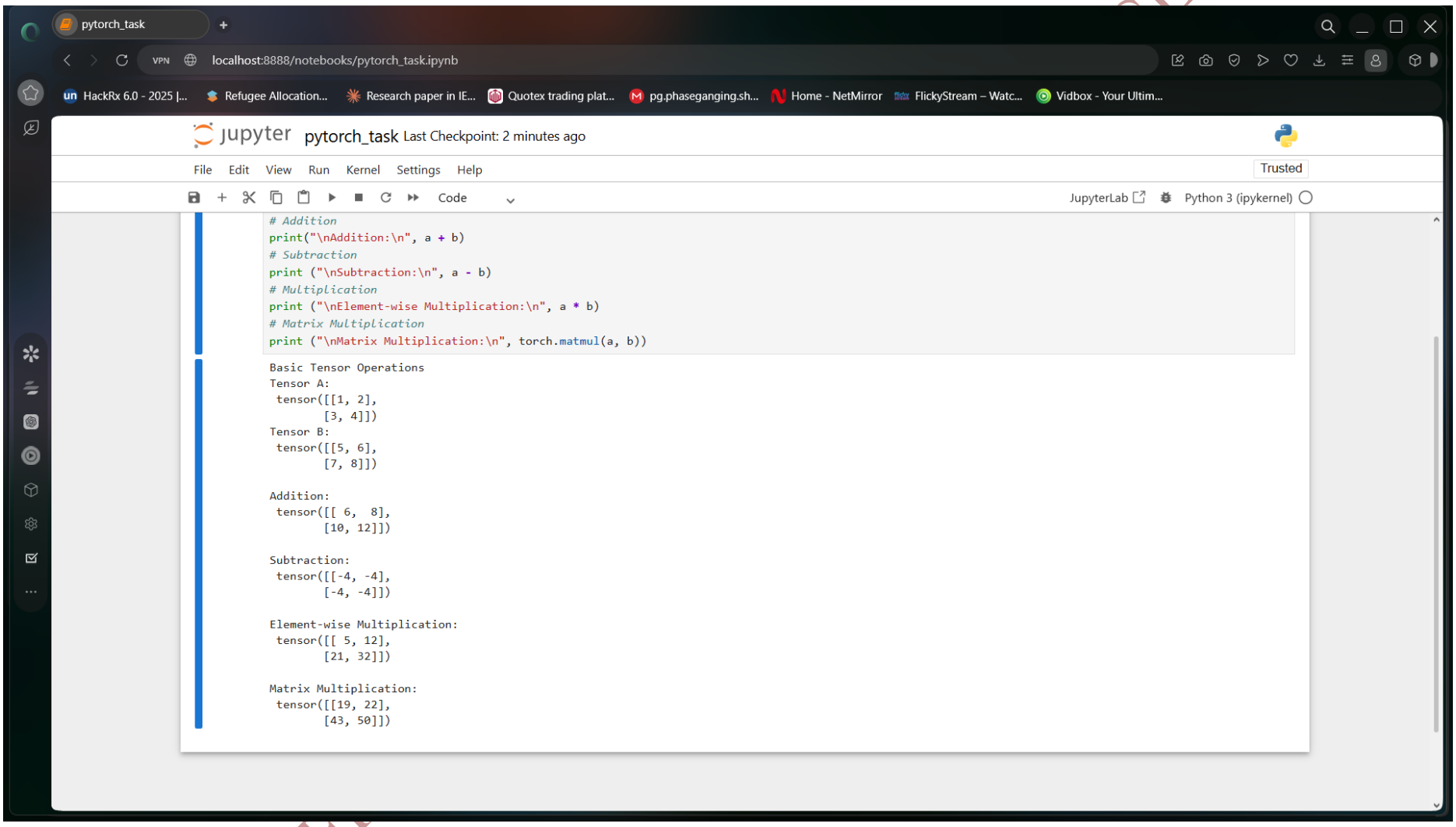


Fig 40:Jupyter Tensor Output

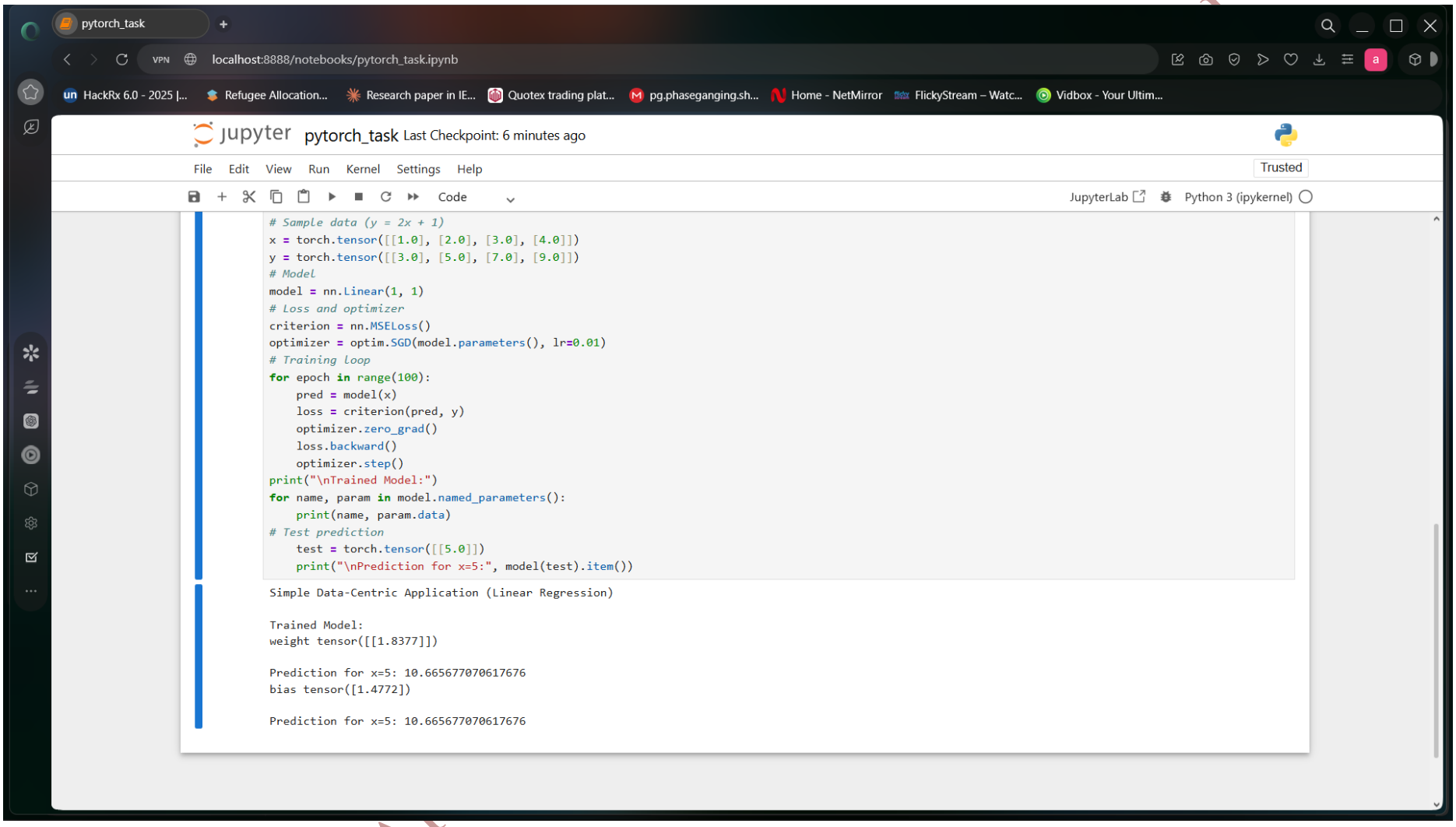


Fig 41:Linear Regression Output

After preparing the dataset and defining the model, the next step involved specifying a loss function and an optimizer in order to train the linear regression model iteratively.

```
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

for epoch in range(100):
    pred = model(x)
    loss = criterion(pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

After training the model using iterative epochs, the final step was to test the trained model using test inputs.

```
test = torch.tensor([[5.0]])
print("Prediction for x=5:", model(test).item())
```

Finally, we checked whether the newly trained linear model gave correct predictions for new inputs.

Task 7 : Testing the GPGPU Computing

Date: 21-04-2026

Procedure

In this experiment, analysis and comparison of CPU-based computation vs GPU-based computation was carried out using PyTorch. The objective was to show the improvements offered by the use of GPUs for parallel computing on large scale tasks.

Step-by-Step Explanation

In order to confirm the availability of a GPU, the following code snippet was executed.

```
import torch

print("GPU Available:", torch.cuda.is_available())
```

Firstly, a large dataset was created. Sorting operation was performed using CPU, and runtime was measured.

```
import time

data = torch.rand(10_000_000)

start = time.time()
torch.sort(data)
cpu_time = time.time() - start

print("CPU Time:", cpu_time)
```

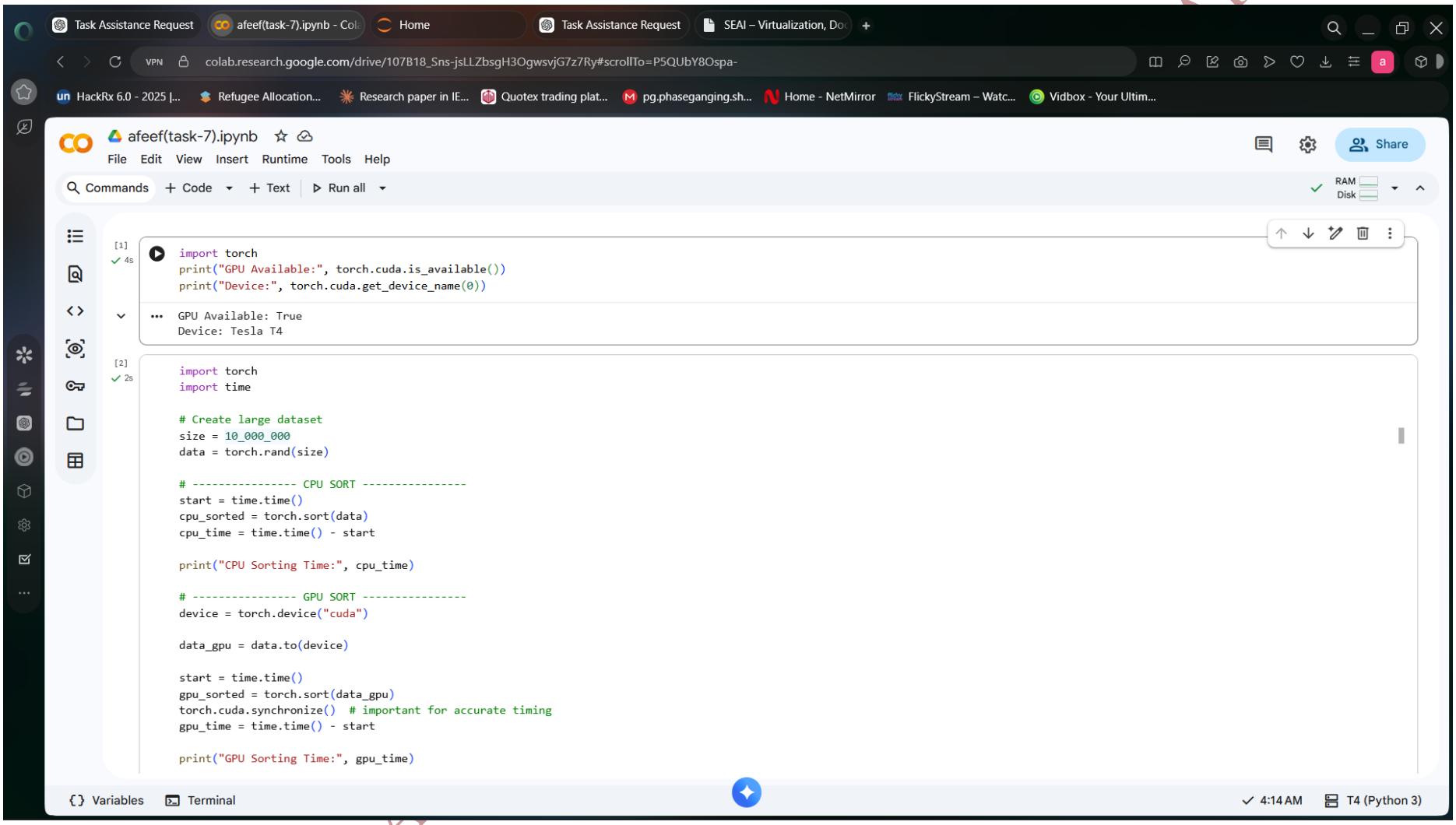


Fig 42:GPU Verification Output

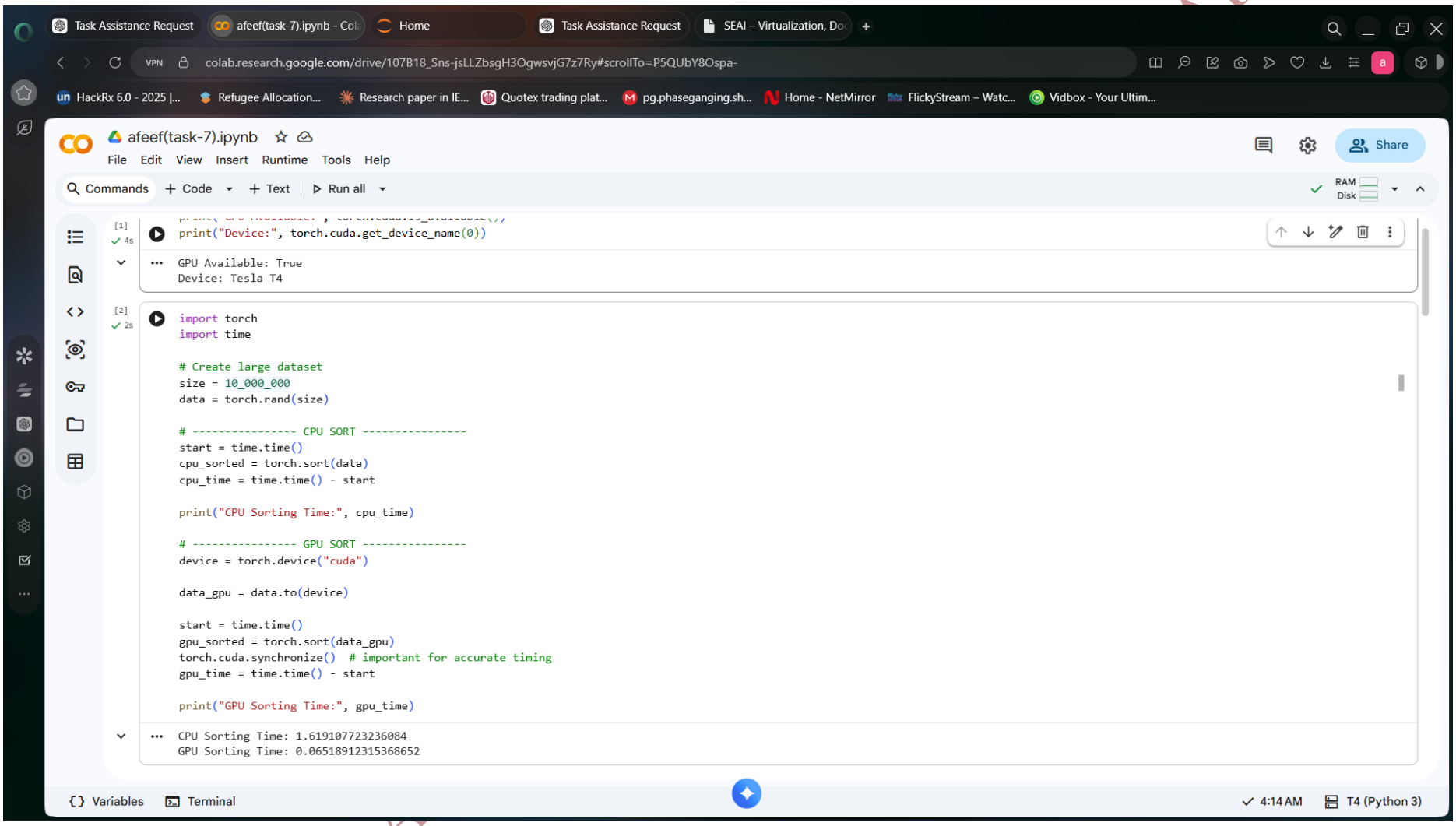


Fig 43:GPU Sorting Output

Then, the same data was moved to the GPU memory, and the sorting operation was again performed to measure runtime.

```
device = torch.device("cuda")

data_gpu = data.to(device)

start = time.time()
torch.sort(data_gpu)
torch.cuda.synchronize()
gpu_time = time.time() - start

print("GPU Time:", gpu_time)
```

As seen from the results, GPU computations were considerably more efficient compared to CPU for large data sets, which demonstrated the benefits of parallel computations.

Task 8 : Testing CUDA-X Tensor Operations, Mixed Precision, and Quantization-Aware Training using PyTorch

Date: 25-04-2026

Procedure

In the current experiment, we explored GPU programming in PyTorch through advanced CUDA tensor operations, mixed precision, and automatic differentiation.

Step-by-Step Explanation

In the following example, the device was selected as either GPU or CPU based on availability.

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using:", device)
```

Large tensors were created on GPU and matrix multiplication was applied to them.

```
A = torch.randn(4096, 4096, device=device)
B = torch.randn(4096, 4096, device=device)

C = A @ B
print("Matrix multiplication completed")
```

Autodiff capabilities in PyTorch were tested next.

```
x = torch.tensor(2.0, requires_grad=True)
y = x**3 + 2*x**2 + x

dy_dx = torch.autograd.grad(y, x, create_graph=True)[0]
d2y_dx2 = torch.autograd.grad(dy_dx, x)[0]

print("First derivative:", dy_dx.item())
print("Second derivative:", d2y_dx2.item())
```

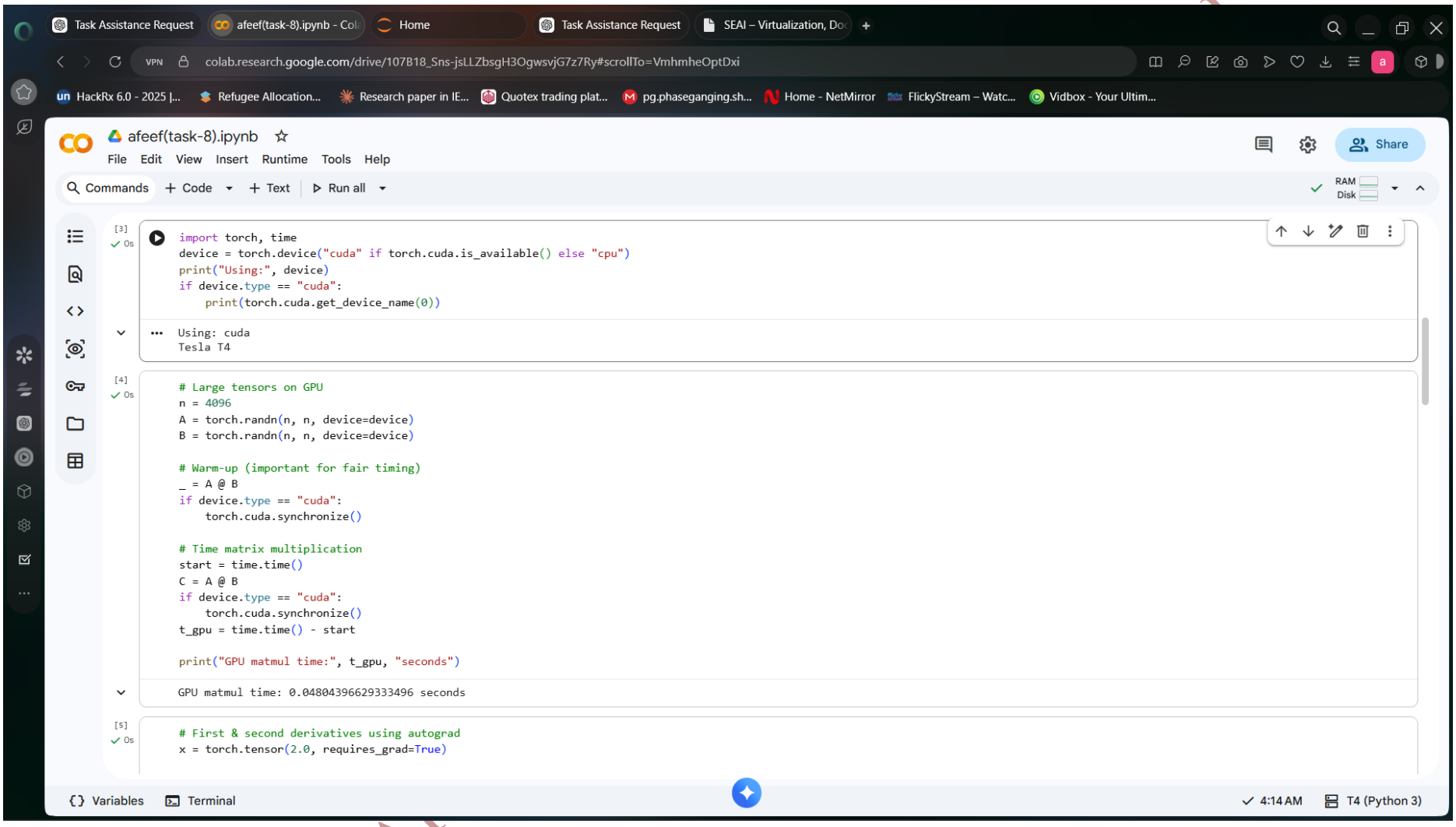


Fig 44:GPU Tensor Operations

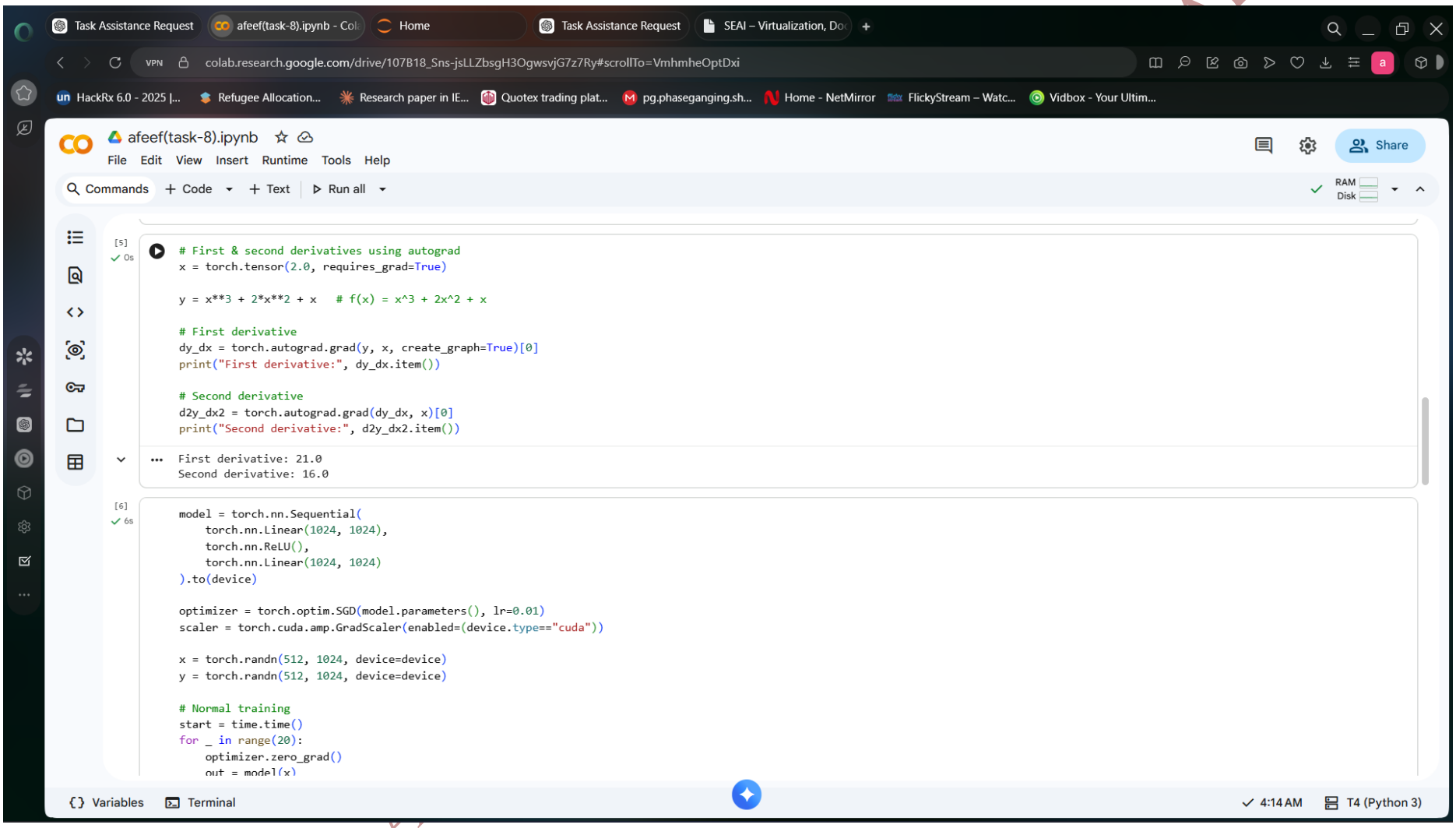


Fig 45:Autograd Derivatives Output

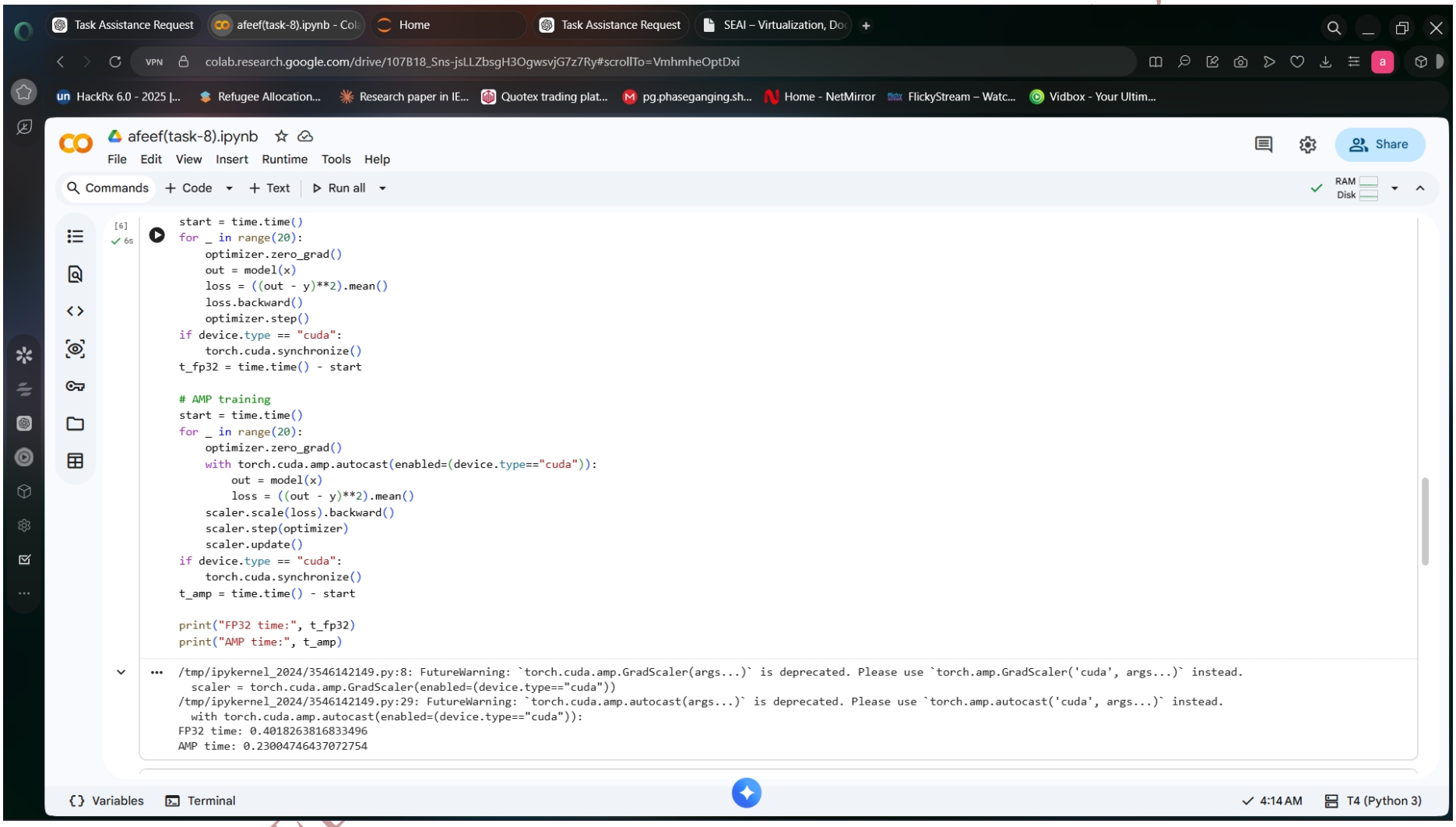


Fig 46:AMP Performance Output

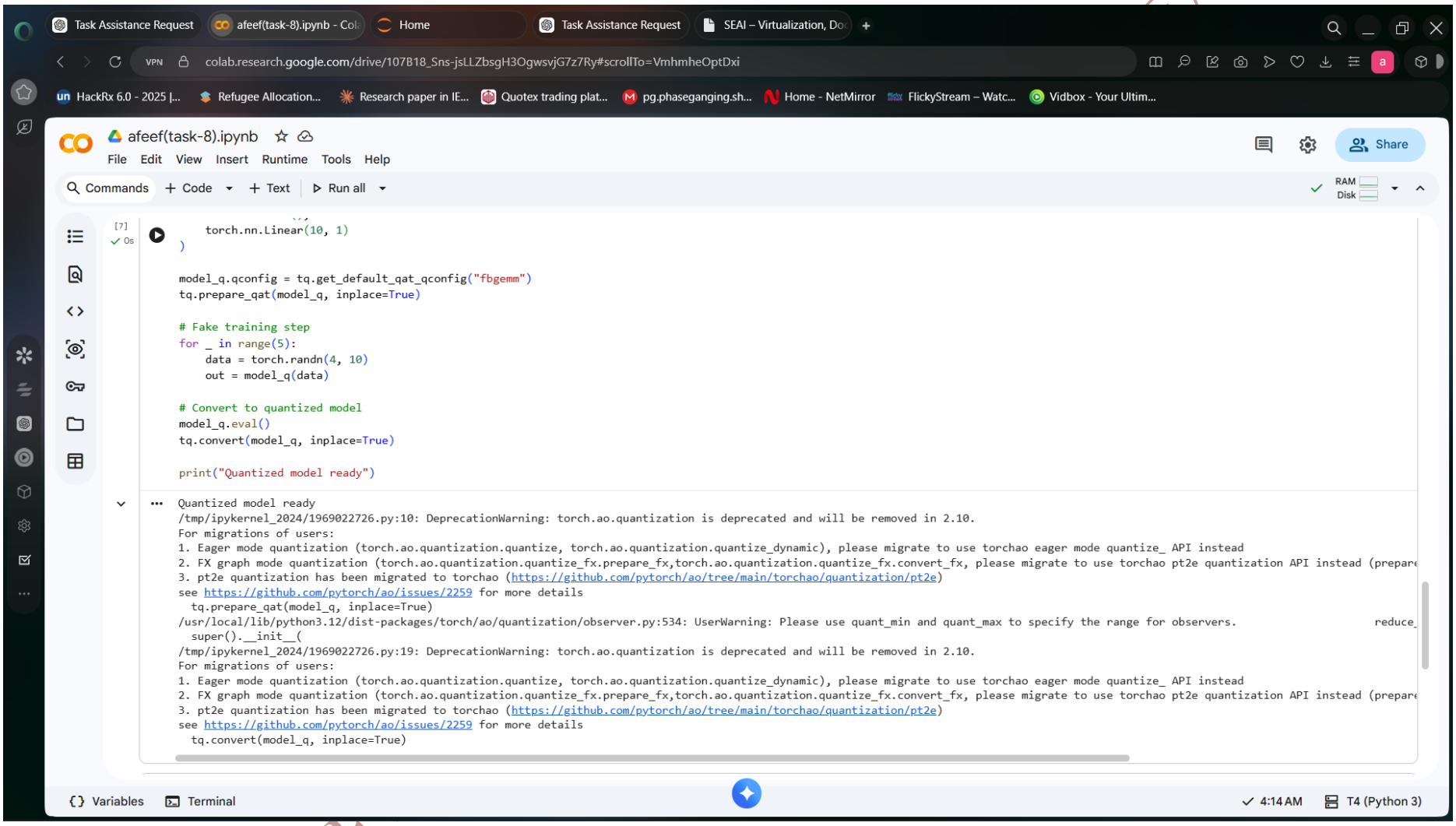


Fig 47:Quantization Output

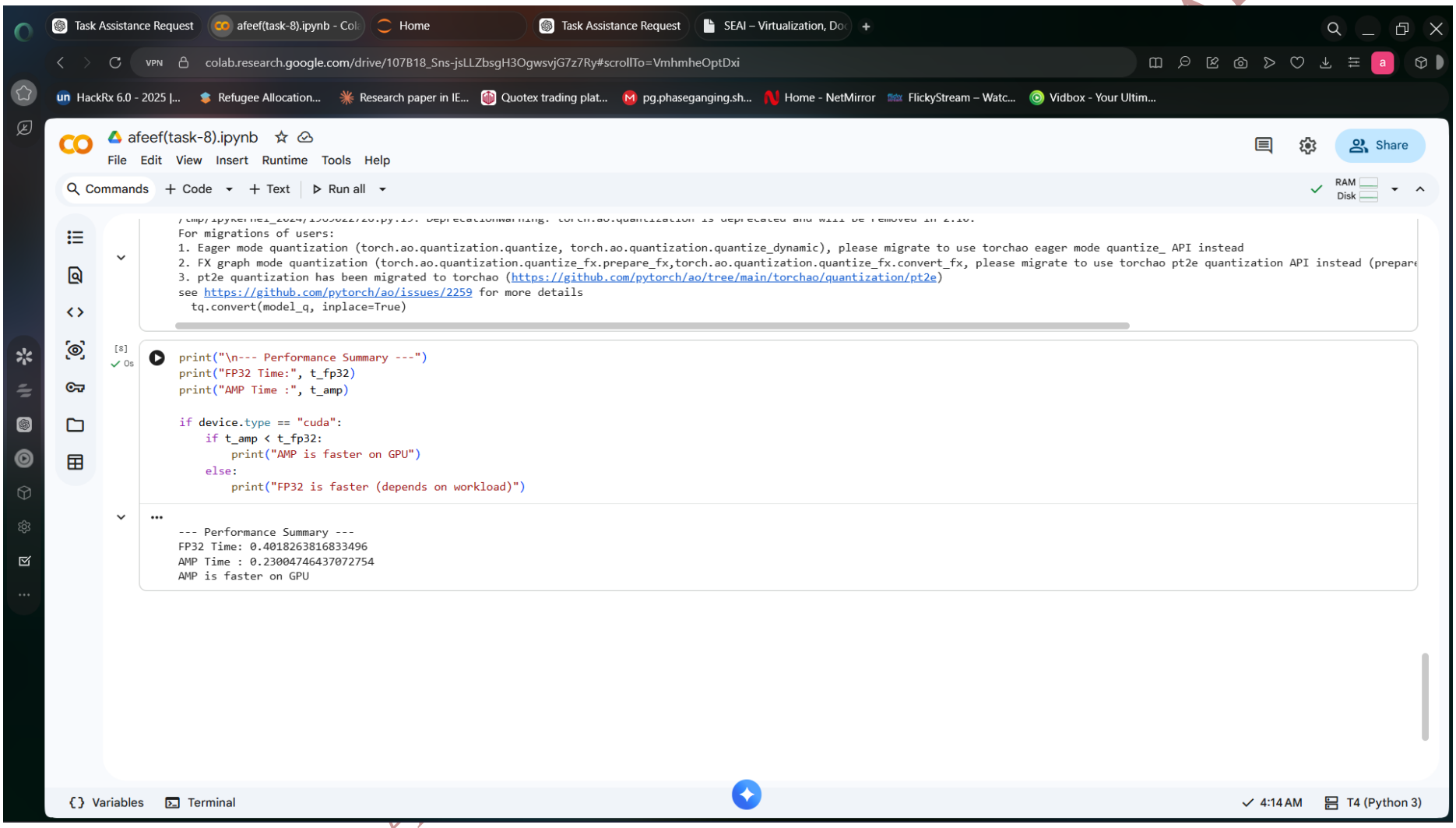


Fig 48:Performance Summary Output

Next, mixed precision using autodiff was tested to increase speed.

```
from torch.cuda.amp import autocast

with autocast():
   result = A @ B

print("AMP computation completed")
```

Finally, quantization-aware training with PyTorch library was tested.

```
import torch.quantization as tq

model = torch.nn.Linear(10, 1)
model.qconfig = tq.get_default_qat_qconfig("fbgemm")

tq.prepare_qat(model, inplace=True)
tq.convert(model, inplace=True)

print("Quantization completed")
```

After this, quantization aware training was demonstrated using PyTorch libraries..

```
import torch.quantization as tq

model_q = torch.nn.Sequential(
    torch.nn.Linear(10, 10),
    torch.nn.ReLU(),
    torch.nn.Linear(10, 1)
)

model_q.qconfig = tq.get_default_qat_qconfig("fbgemm")
tq.prepare_qat(model_q, inplace=True)

for _ in range(5):
    data = torch.randn(4, 10)
    out = model_q(data)

model_q.eval()
tq.convert(model_q, inplace=True)

print("Quantized model ready")
```

There was an improvement in efficiency and reduced computation costs.

Task 9 : Accelerating Neural Network Inferencing: TensorRT & Triton Inference Server

Date: 28-04-2026

Procedure

This experiment was conducted to accelerate neural network inferencing using TensorRT and Triton Inference Server. It involved model conversion, repository creation, inference server deployment, and finally model inferencing.

Step-by-Step Explanation

A model was generated using PyTorch and exported to the ONNX format.

```
import os
os.environ["TORCH_ONNX_USE_DYNAMO"] = "0"

import torch
import torchvision.models as models

model = models.resnet18(pretrained=True)
model.eval()

dummy_input = torch.randn(1,3,224,224)

torch.onnx.export(model, dummy_input, "model.onnx", opset_version=11)
```

This model was then converted to the tensorRT engine.

```
trtexec --onnx=model.onnx --saveEngine=model.plan
```

The next step involved the generation of the model repository and placement of the engine files in the repositor.

```
model_repository/
└── my_model/
    └── 1/
        └── model.plan
```

Configuration for this repository was made where the input/output information were specified.

```
name: "my_model"
platform: "tensorrt_plan"
max_batch_size: 1
```

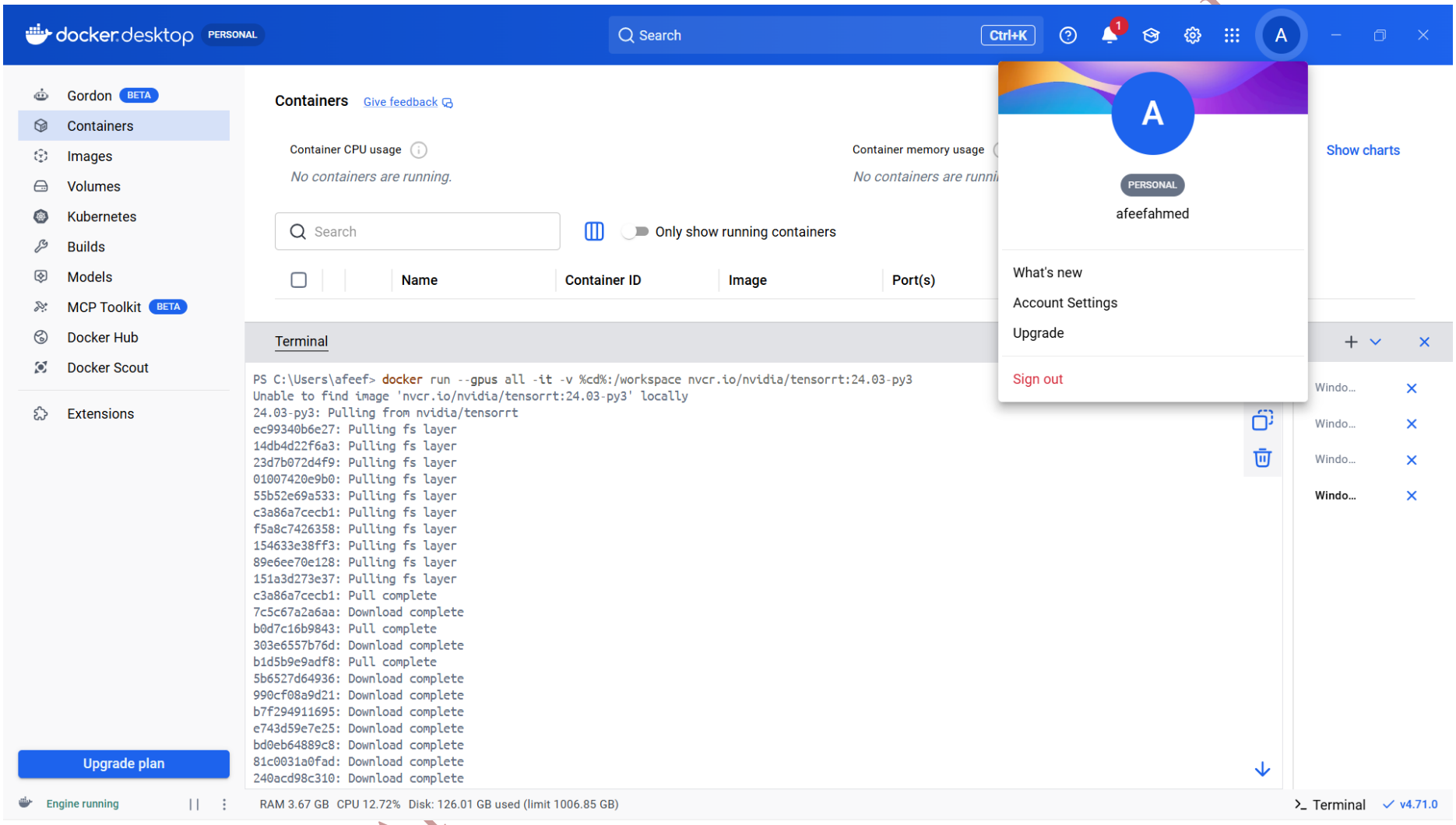


Fig 49:Triton Docker Pull

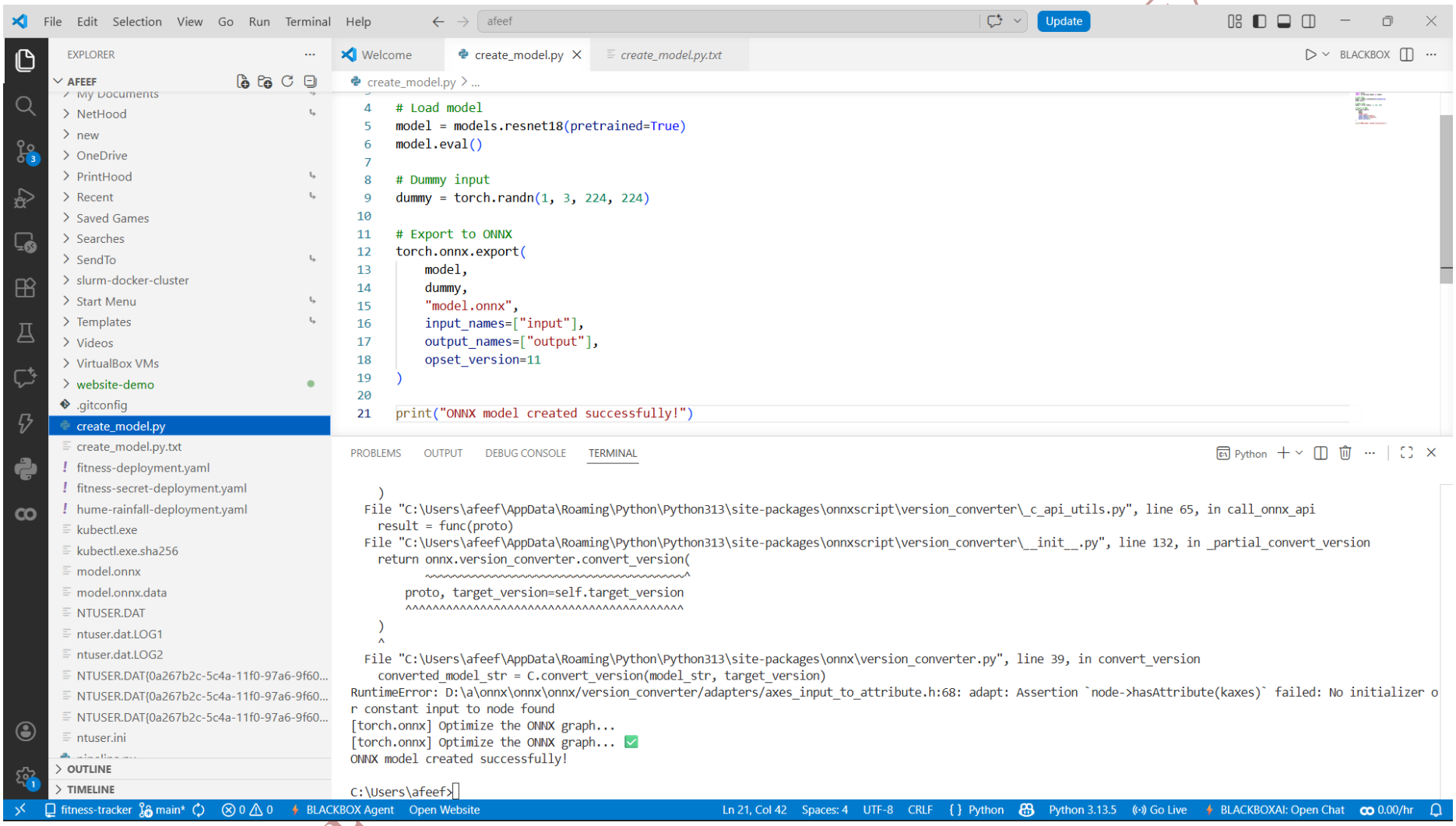


Fig 50:ONNX Model Script

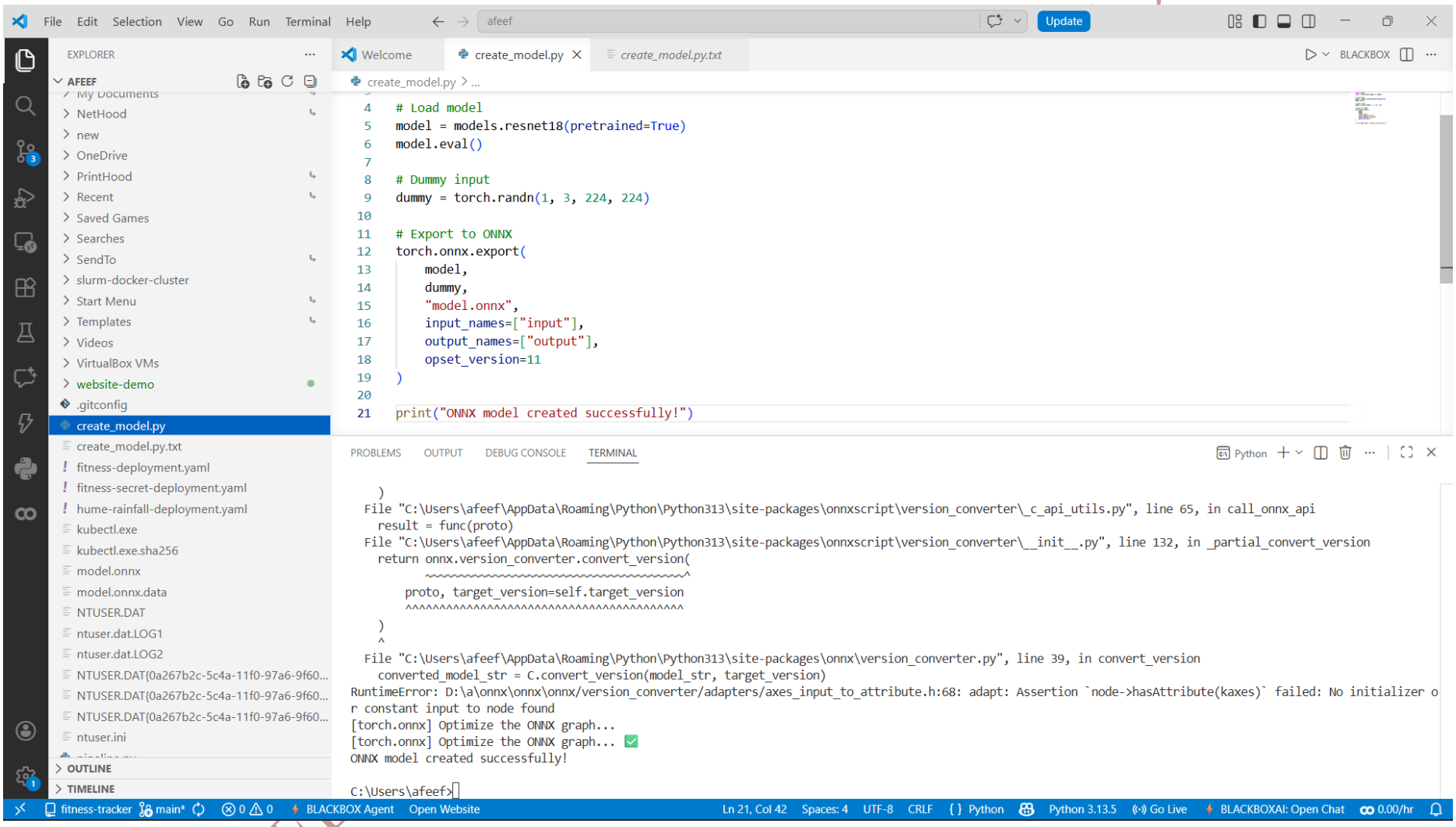


Fig 51:ONNX Model Created

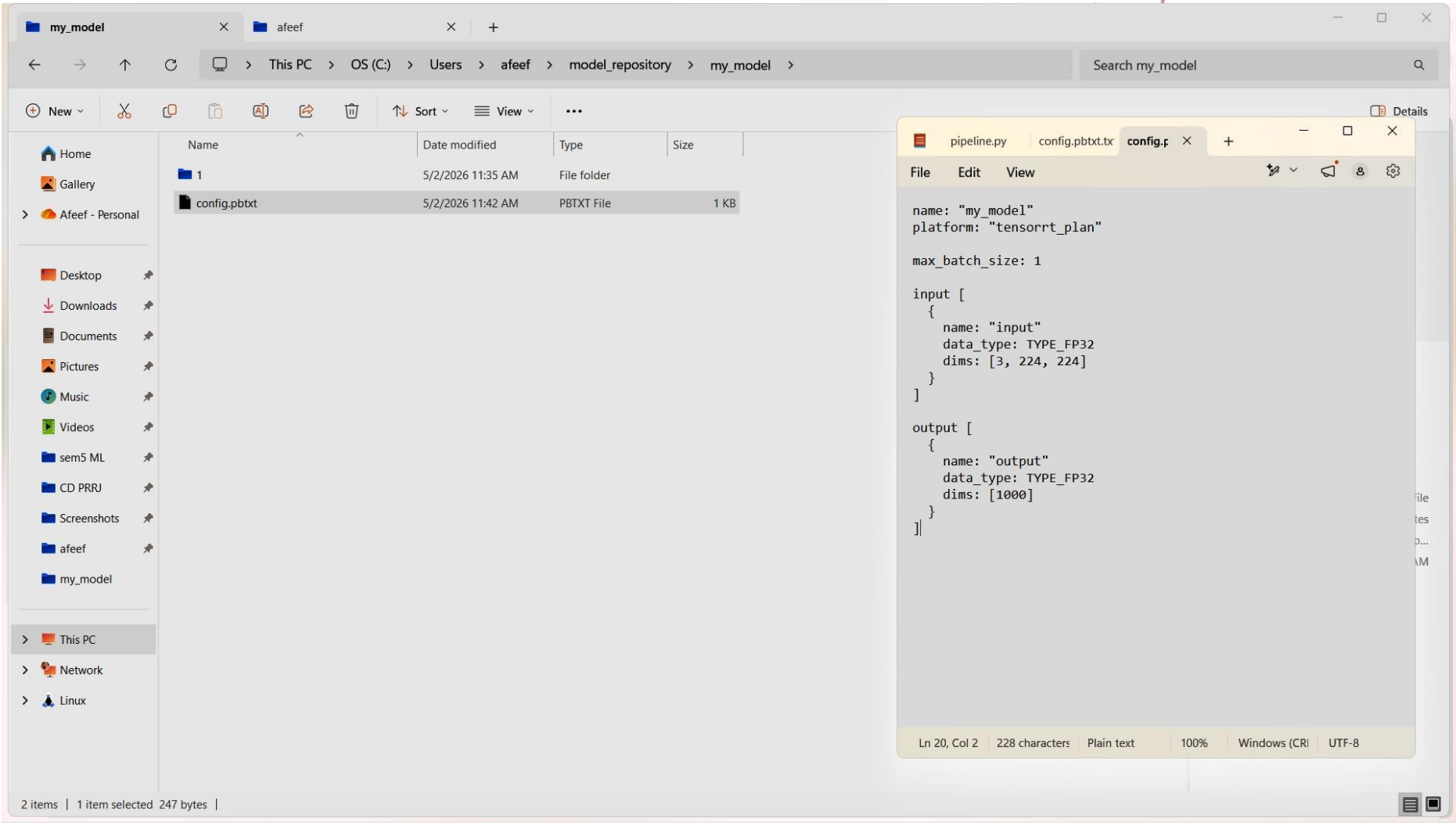


Fig 52:Triton Config File

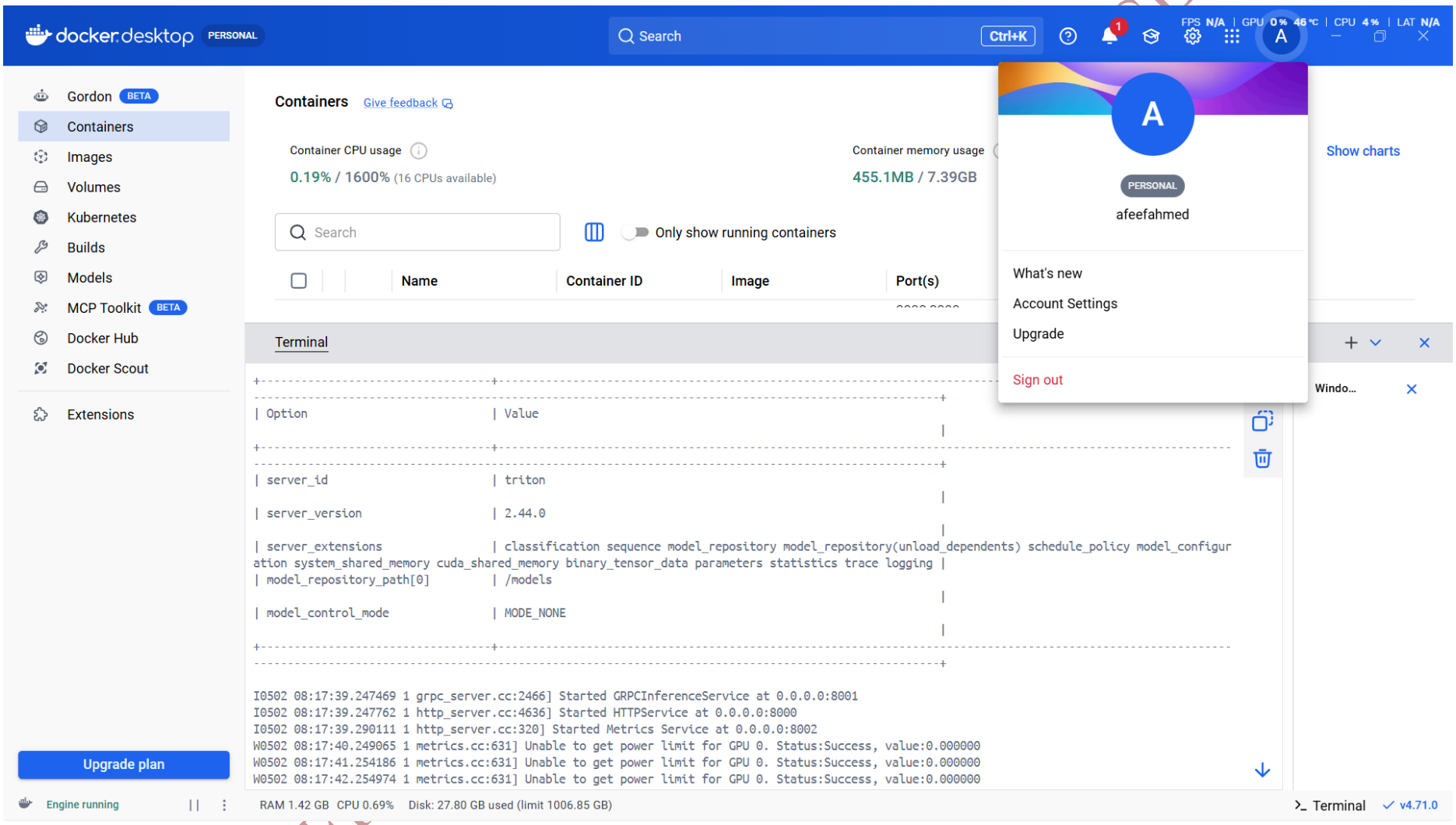


Fig 53:Triton Server Running and status

Finally, the Triton inference server was deployed using the Docker application.

```
docker run --gpus all -p8000:8000 -p8001:8001 -p8002:8002 -v C:\Users\afeef\model_repository:/models
nvcr.io/nvidia/tritonserver:24.03-py3 tritonserver --model-repository=/models
```

Server status checked.

```
curl http://localhost:8000/v2/health/ready -UseBasicParsing
```

Model is successfully deployed and Inference speed is enhanced using GPU.

Task 10 : Monitoring Load Balancers & Schedulers using Docker with Jupyter, Prometheus, and Grafana

Date: 01-5-2026

Procedure

In this experiment, load balancing and scheduling services were deployed using Docker and performance metrics were analyzed using Prometheus and Grafana visualization tools. Computational load was applied using a Jupyter notebook.

Steps Involved

In this experiment, load balancing and scheduling services were deployed using Docker Compose and visualizations were carried out using Prometheus and Grafana visualization tooling. A Jupyter notebook was used for generating computational load.

Step-by-Step Explanation

Project directory was made, and docker-compose.yml was created.

```
mkdir exp10
cd exp10
```

Docker Compose configuration consisted of following components such as Jupyter, Prometheus, Grafana and Node Exporter.

```
version: '3'
services:
  jupyter:
    image: jupyter/base-notebook
    ports: ["8888:8888"]

  prometheus:
    image: prom/prometheus
    ports: ["9090:9090"]

  grafana:
    image: grafana/grafana
    ports: ["3000:3000"]

  node-exporter:
    image: prom/node-exporter
    ports: ["9100:9100"]
```

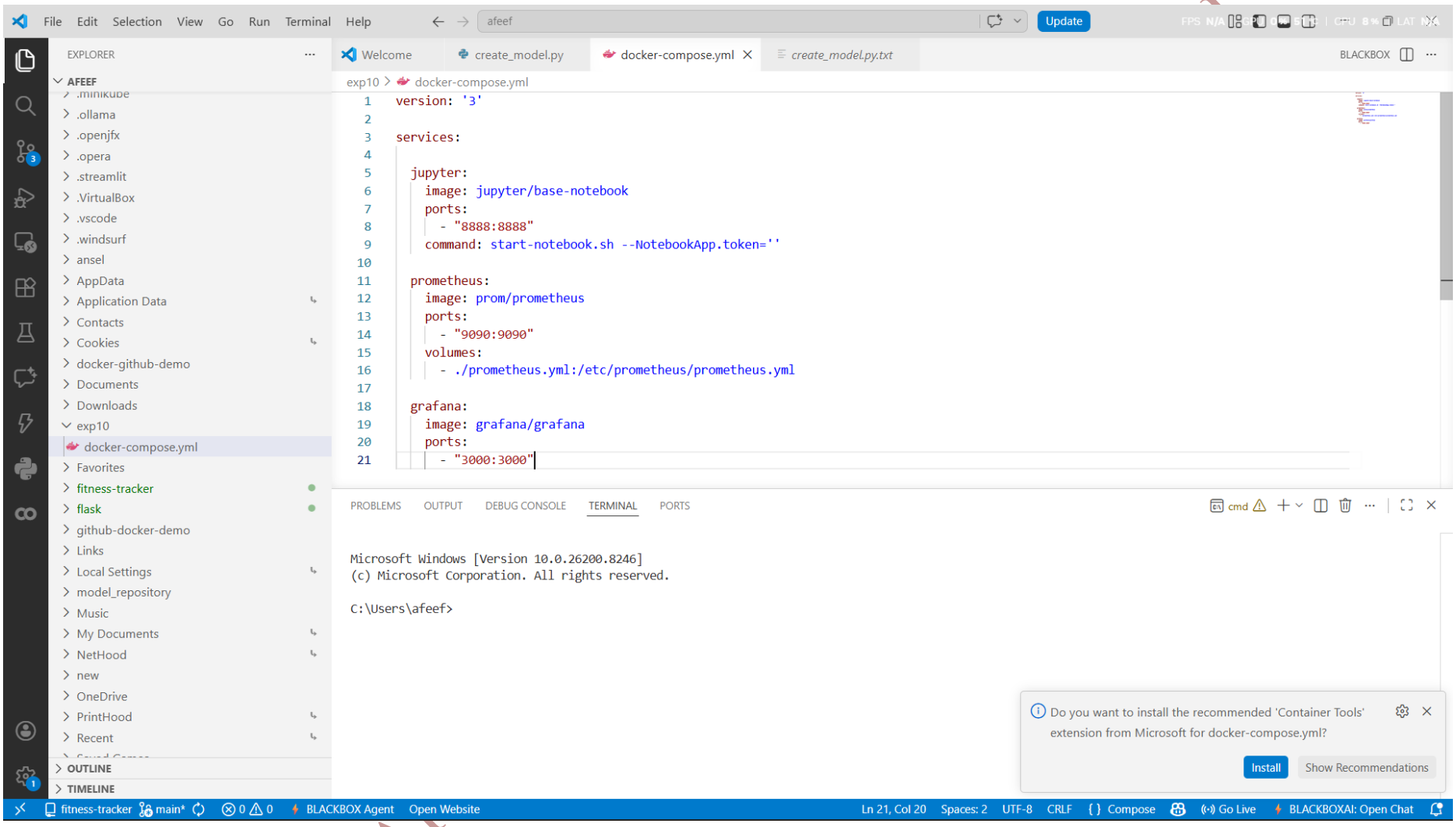


Fig 54:Docker Compose Config

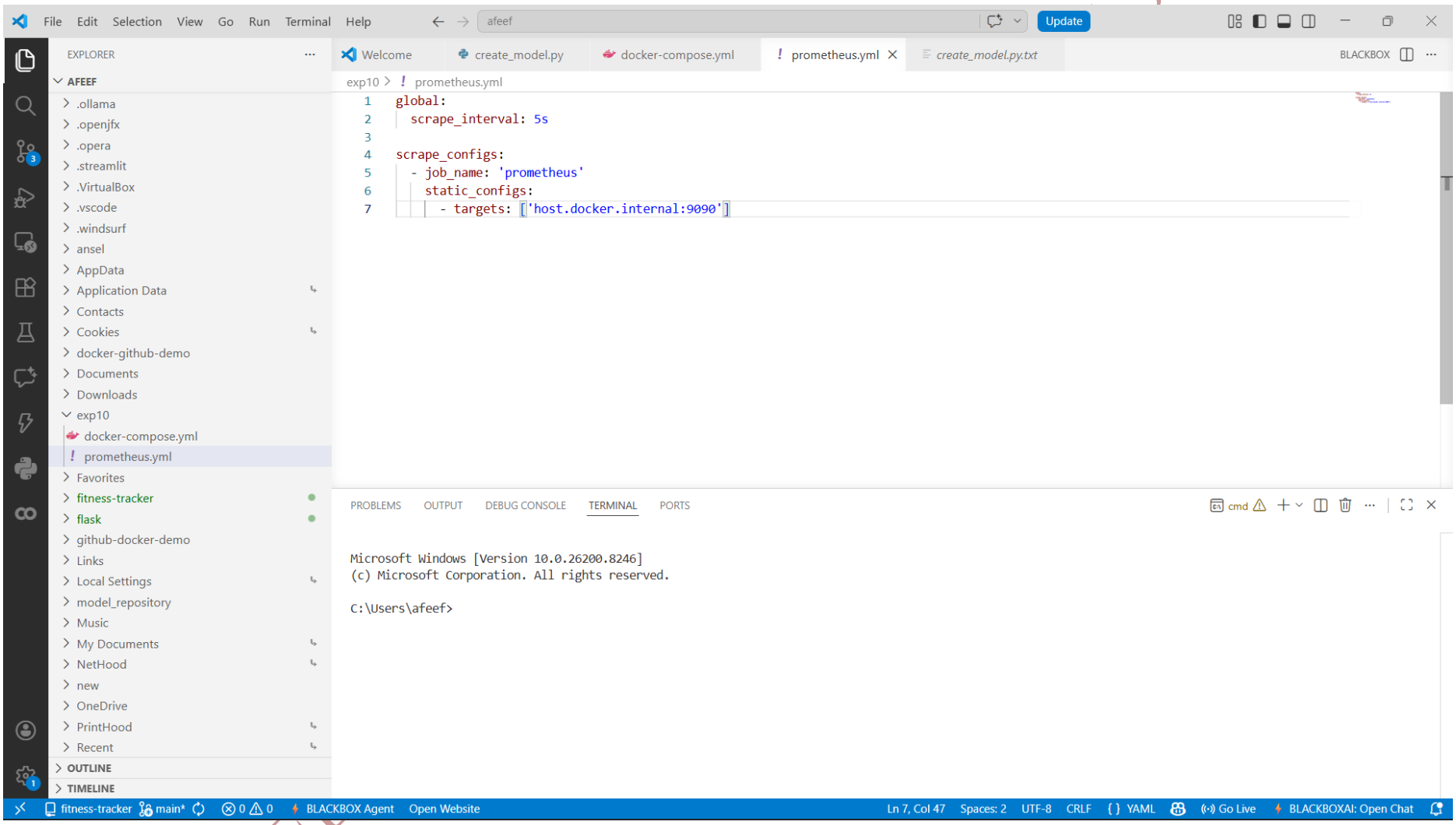


Fig 55:Prometheus Config File

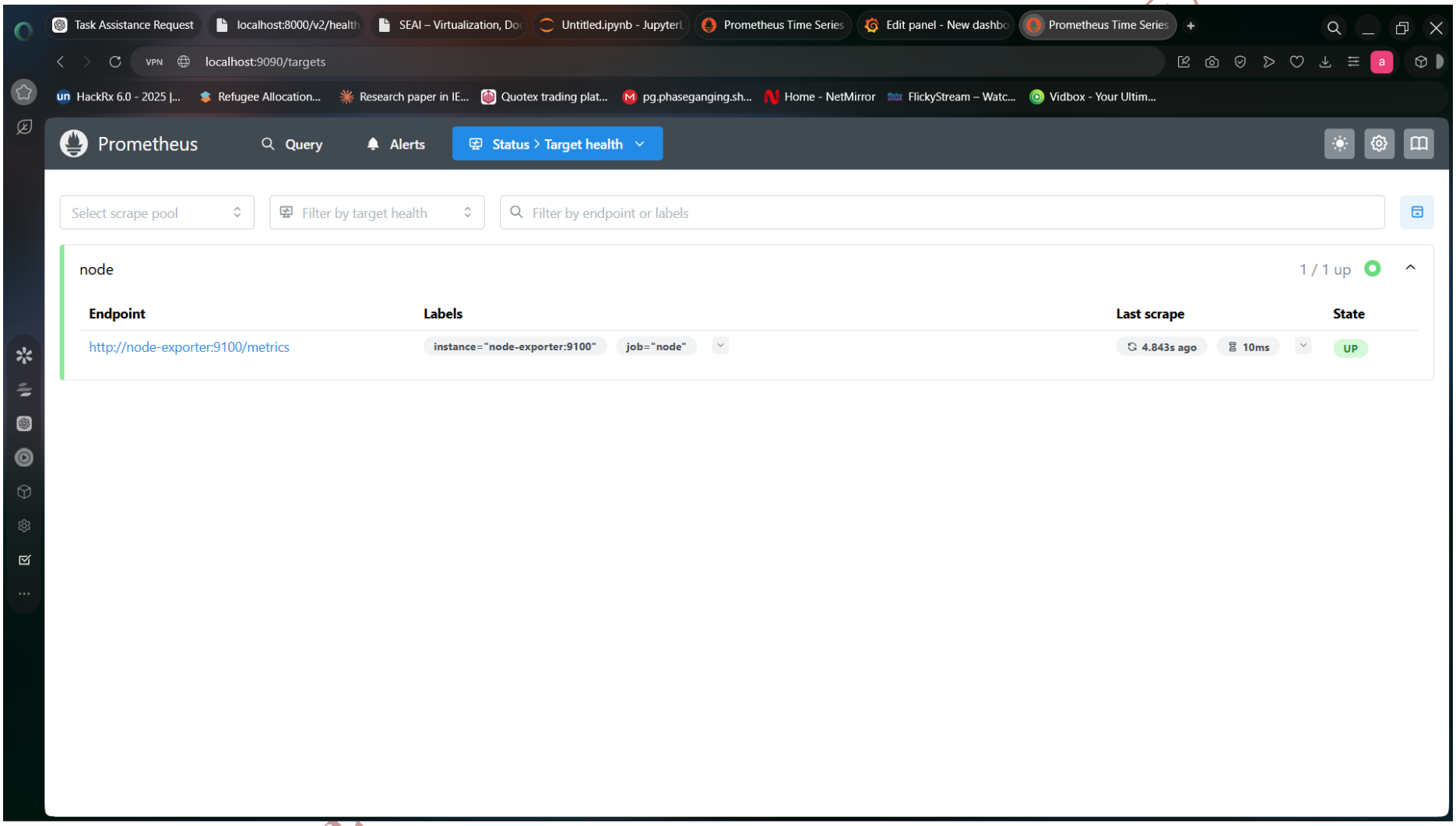


Fig 56:Prometheus Target Health Status

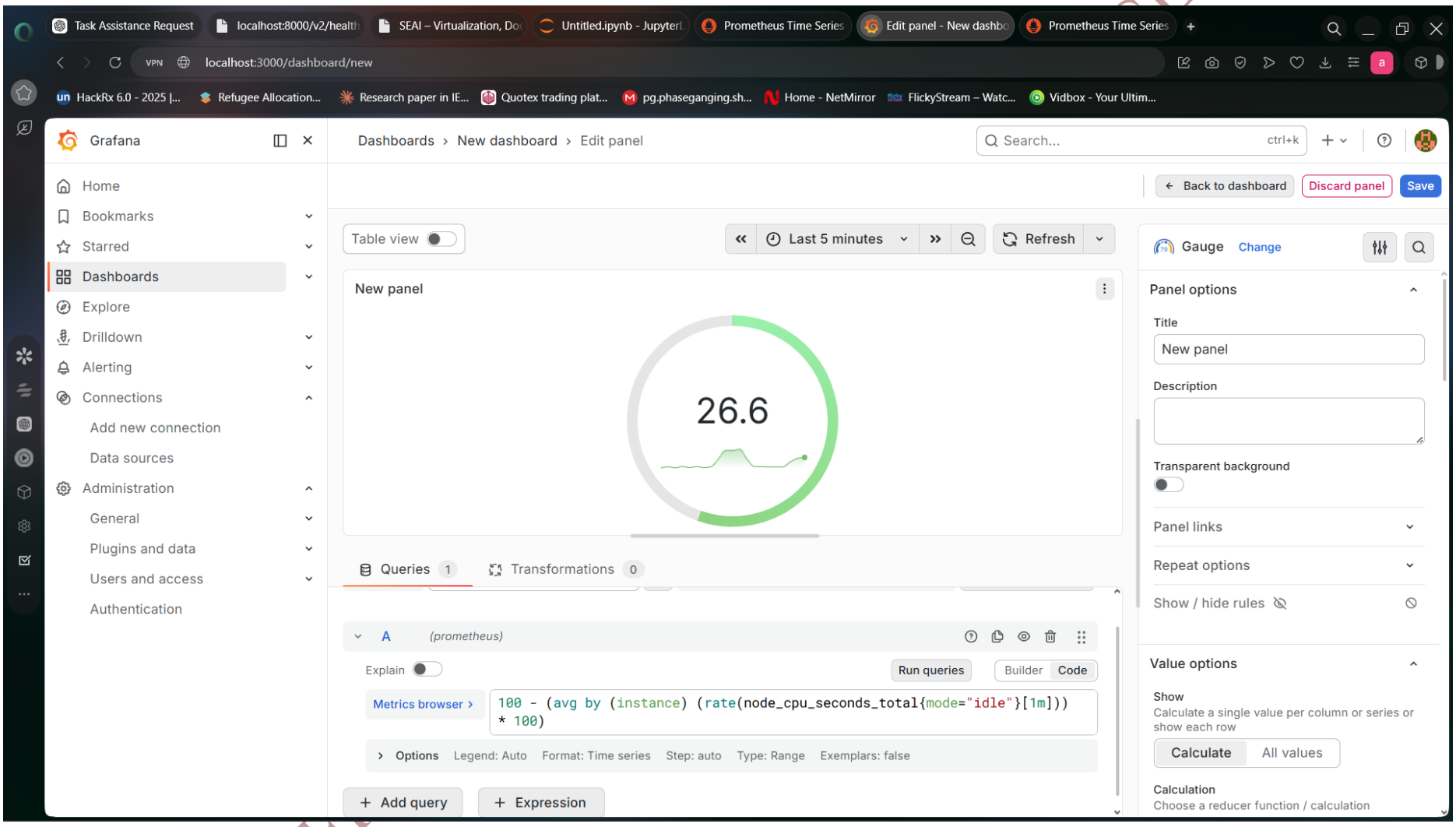


Fig 57:Grafana Monitoring Dashboard

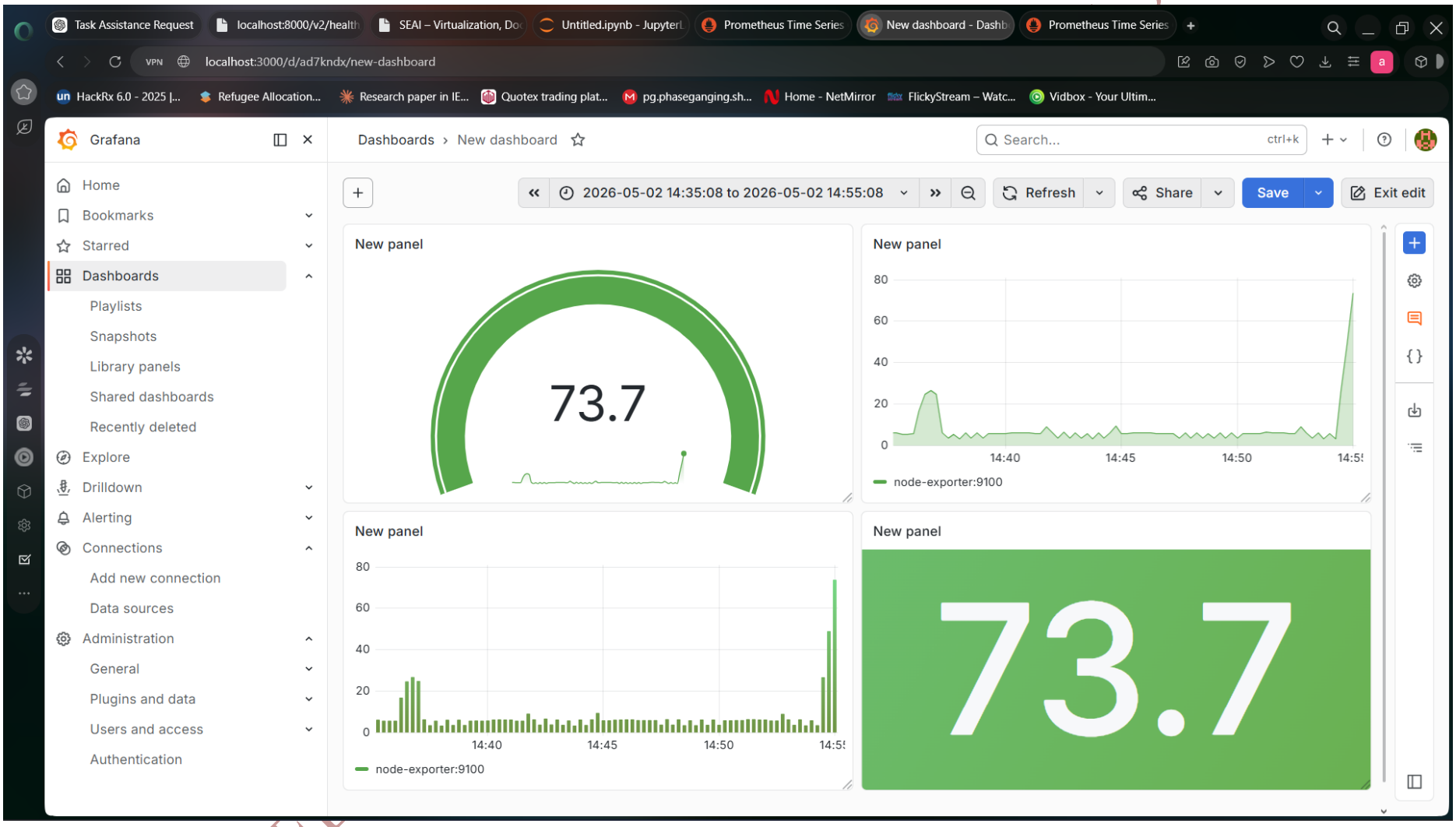


Fig 58:Grafana System Metrics Dashboard

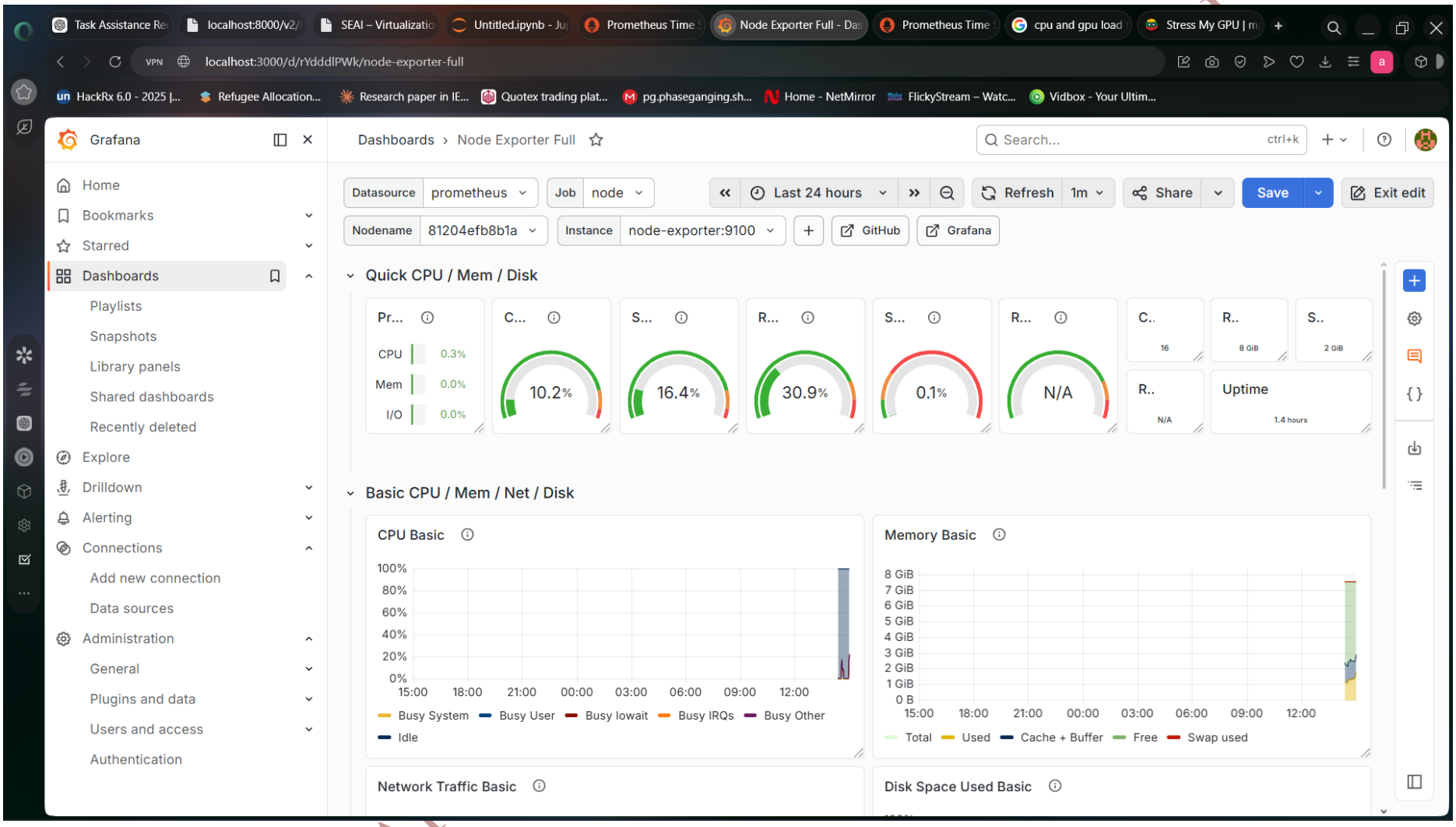


Fig 59:Grafana System Metrics Dashboard

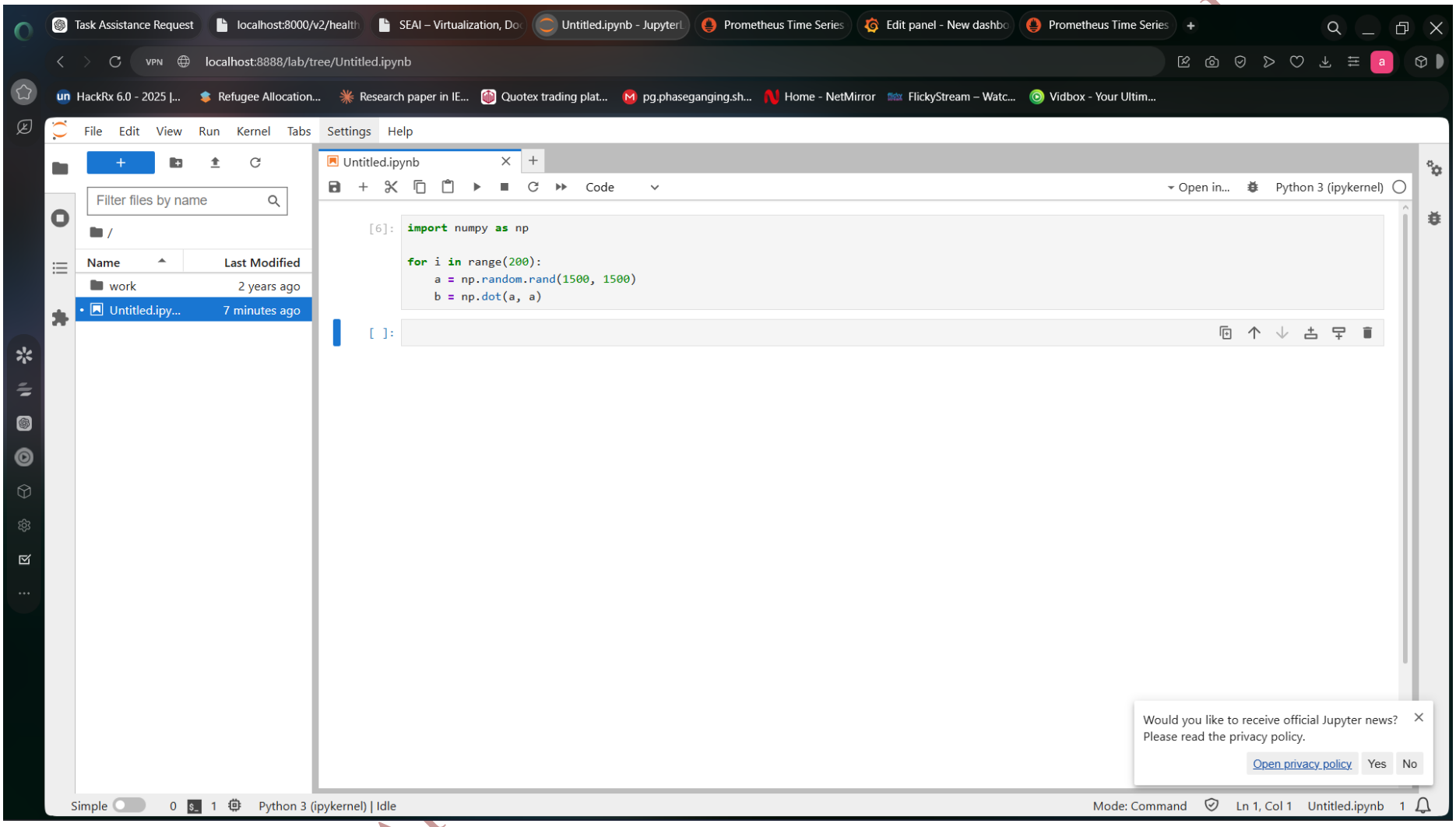


Fig 60:Jupyter Load Test Script

Services were started by using command:

```
docker compose up
```

The services were accessed through:

```
http://localhost:8888
http://localhost:9090
http://localhost:3000
```

And services can be accessed using following URLs.

```
import numpy as np

for i in range(200):
    a = np.random.rand(1500,1500)
    b = np.dot(a,a)
```

CPU usage was monitored in Grafana dashboard.

Product Innovation

Title: COVID-19 Detection System: AI-Powered Clinical Diagnostic Platform for COVID-19 Detection using Deep Learning and Explainable AI

1. Introduction

The presented product introduces an innovative AI-driven solution for the automated diagnosis of COVID-19 and related pulmonology conditions via chest X-ray images. Unlike other machine learning solutions that rely on one predictive algorithm, this system integrates several deep learning models, explainable AI components, and a number of scalable deployment options to ensure maximum interpretability, usability, and adaptability. In addition to delivering accurate predictions, the system's aim is to be reliable and useful to medical specialists when used in their clinical practices.

To train the proposed deep learning model, thousands of chest X-rays with labels were employed in this project. This data helps the model to identify several classes: COVID-19, normal lungs, viral pneumonia, and lung opacity. What makes this innovation different from traditional solutions is that this platform utilizes the power of a multi-model ensemble mechanism that involves the analysis of the same input image by several different models that reach a conclusion.

Along with improving the performance of the predictive process, this feature adds another layer of reliability to the diagnostic process since more than one model has to make the same decision for it to happen. Besides achieving the highest level of prediction accuracy, the system emphasizes the importance of being transparent and explainable. Clinicians expect a detailed explanation for each predicted class to help diagnose the patient and take further action.

A developed implementation of the current work is present at the following link:
https://github.com/ssdafeef/PulmonaryAI

2. Problem Statement

Current AI-based diagnostic solutions have faced some critical issues within the healthcare domain that limit their usage within clinical practice. One of the most prominent problems lies in using a single deep learning model that makes them prone to bias. Considering that any errors that the system makes will directly affect the health and life of patients, it is crucial to develop a reliable solution to deal with such a problem.

In addition, some diagnostic systems are primarily designed for research purposes and thus do not reflect real-life processes of working with medical data and making diagnoses. For example, they lack scalability and do not offer an opportunity to integrate the tool into existing systems and software solutions. In reality, there is a variety of steps required to be taken for diagnosing, such as data validation, collaboration, and structuring.

Another challenge of the current AI diagnostic solution is its inability to manage uncertainty. Current technologies tend to produce overconfident predictions that may influence practitioners'

decisions in a wrong way. Moreover, there is a lack of some practical features, like generating reports, classification of the disease severity level, and multi-language support.

Moreover, such systems do not use any approaches for uncertainty management or providing confidence scores, making predictions highly overconfident. Another important point is the lack of other useful features like automated report generation, severity categorization, or multilinguality, which makes these solutions not usable in practice. Hence, the aim of this project is to develop a system that will have all those shortcomings addressed.

3. Proposed Solution

The proposed system represents a hybrid AI solution incorporating the latest advancements in deep learning, explainable artificial intelligence, and deployment frameworks in order to deliver a full-fledged clinical decision support system for chest X-ray image assessment. As the core component of the system, a convolutional neural network with a special attention module is used. The chosen network structure is ResNet50.

For increased reliability of the solution, a multi-model ensemble method will be utilized. It will consist of several independent neural networks performing predictions based on a provided image. Then, using a certain algorithm, a consensus between the predictions generated by individual networks will be achieved. Thus, only the most confident predictions from different networks will make their way to the final result. This is inspired by the process of collaborative work of physicians in reality, where a team of specialists contributes to a final decision.

The problem of interpretability and explanation of predictions will also be solved using explainable artificial intelligence techniques. Grad-CAM algorithms will allow generating visual explanations represented by the heatmaps showing what parts of the image influenced the decision of the classifier. Moreover, a separate language model will be able to generate a natural-language explanation of a predicted diagnosis. It will be accompanied by the reasoning behind the prediction, its consequences, and possible further actions.

Automated generation of reports with all predictions and explanations will be implemented as another crucial feature. Reports generated by the solution will be in the PDF format, containing all data regarding the analysis of a chest X-ray image, including severity level. Moreover, there will be multilingual capabilities of the solution, along with a severity classification system.

4. System Workflow

The operation of the system involves a pipeline workflow simulation that resembles a realistic diagnostic scenario. The procedure starts by uploading a chest X-ray image through the web application. It is designed to allow easy interaction for both experienced and non-experienced users.

Once the image is uploaded, it is preprocessed in the backend environment. It includes image size rescaling, normalization of the image's pixel values, and conversion to a specific format. Proper preprocessing is vital for achieving consistent and reliable performance of AI models.

Further, the image is subjected to the analysis of several AI models. Each of them independently provides probability outputs related to different classes. Outputs of different models are combined using the technique of voting. If there is consistent behavior between the models, a certain classification result with high confidence level is obtained. However, in case of discrepancy, the

system informs about the inconsistency and suggests additional analysis. After the prediction, the system uses explainable AI and provides heatmaps highlighting the most influential areas in the image. Further, an AI-driven natural language explanation of the results is generated by the language model.

Additionally, the system performs severity classification for COVID-19 infection cases. As a final step, all information is aggregated and provided via the interface. A PDF document containing information related to the diagnosis is prepared automatically and ready for download.

5. Key Innovations

- AttentionResNet50 with a custom attention mechanism for COVID-19 classification
- Hybrid inference approach using NVIDIA Triton Inference Server and local Keras models
- Explainable AI based on the Grad-CAM visualization of every prediction
- Voting between multiple inference branches based on the confidence in prediction
- Ai-powered explanation using a language model (LLM; Google Gemini)
- Automated PDF report generation with multi-language support
- Four-class classification of Chest X-rays: COVID-19, Normal, Viral Pneumonia, Opacity
- Accuracy ~94%, ~0.97 AUC-ROC on 21,165 chest X-ray images
- Severity classification for COVID-19 infections (mild, moderate, severe)
- Fully deployed system using Docker Compose
- Model serving on NVIDIA Triton Inference Server
- End-to-end AI pipeline for uploading images and generating PDF reports

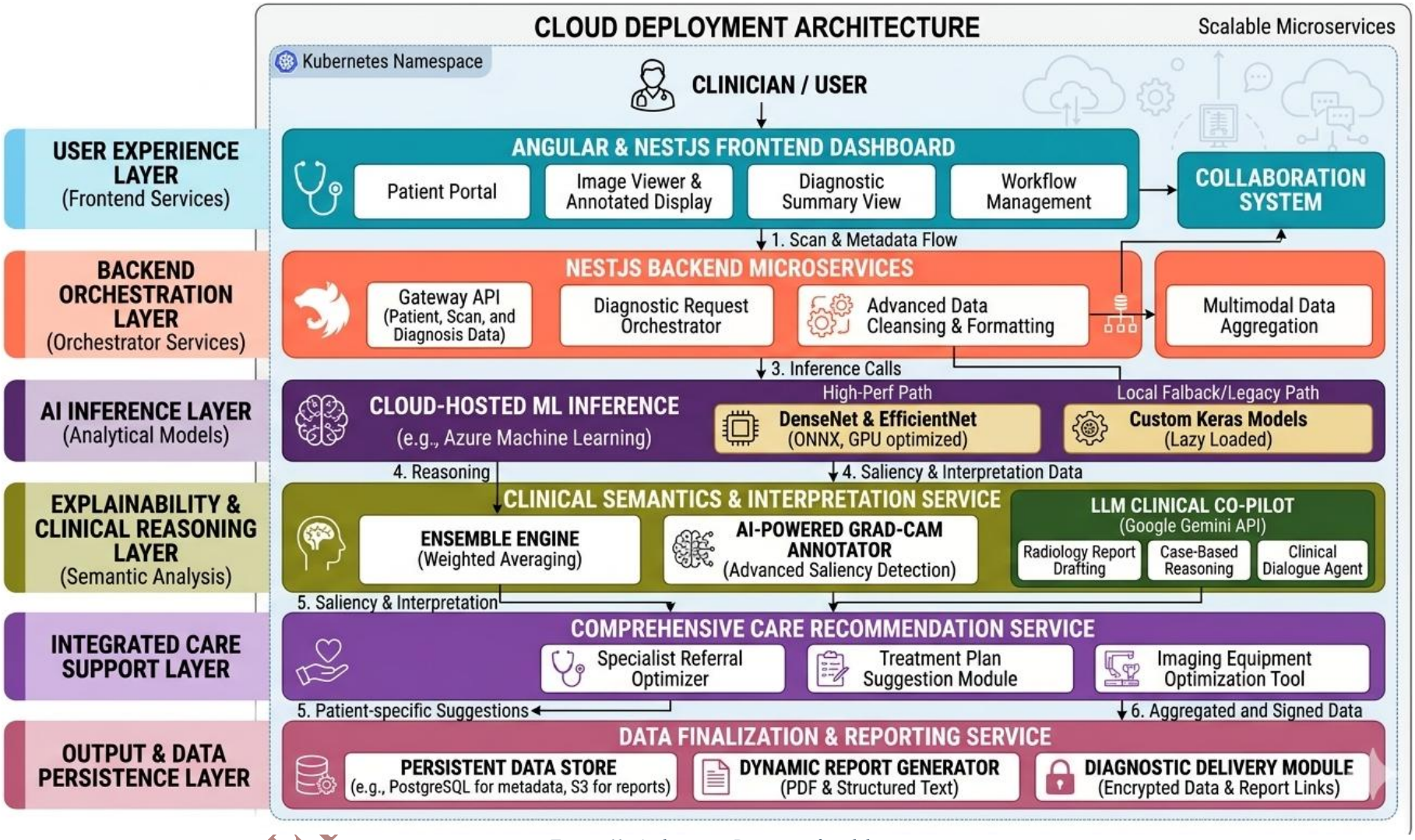


Figure 61: Architecture Diagram of model

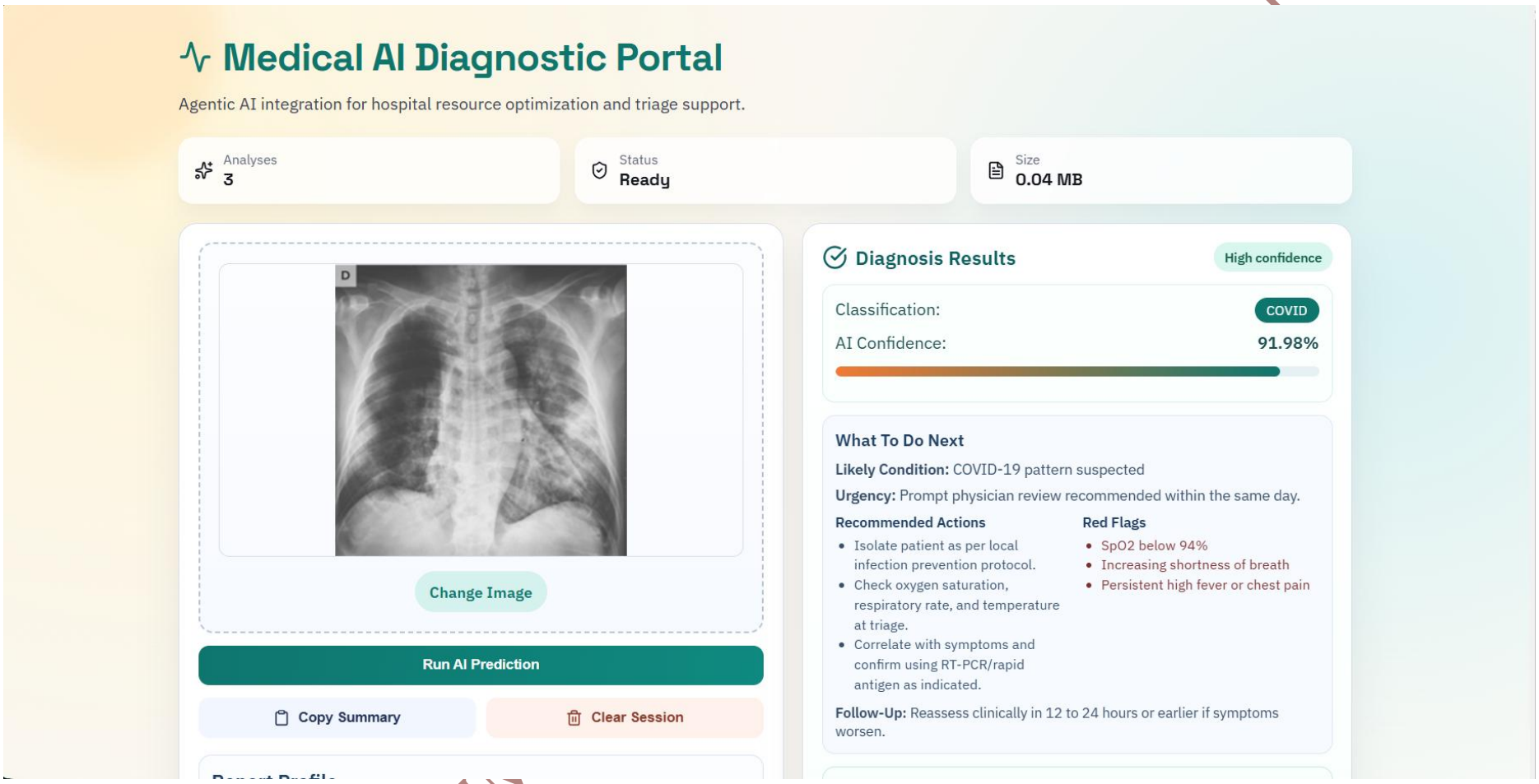


Figure 62: X-ray prediction output showcasing result

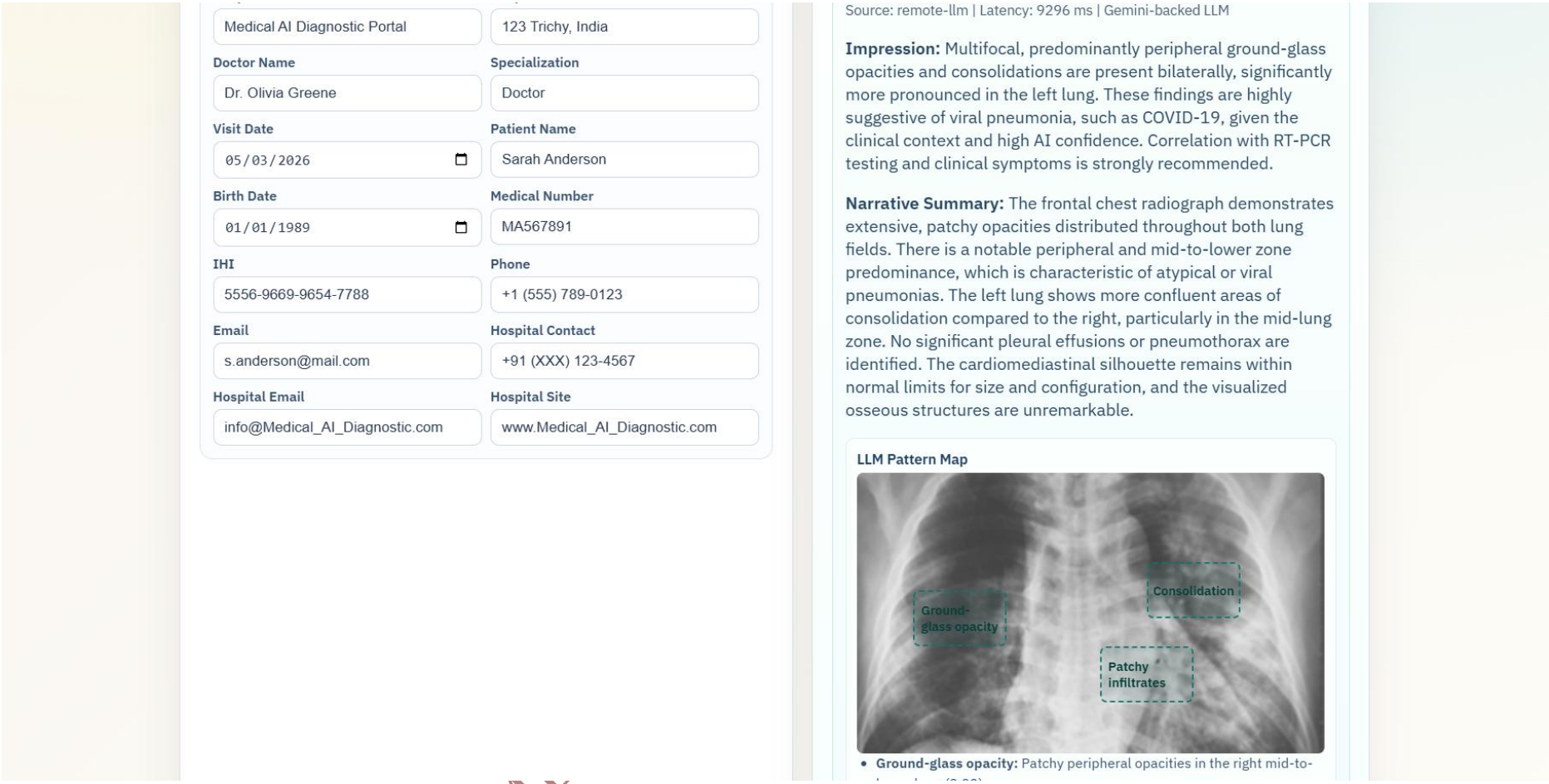


Figure 63: The LLM model showcasing overlay of detected area

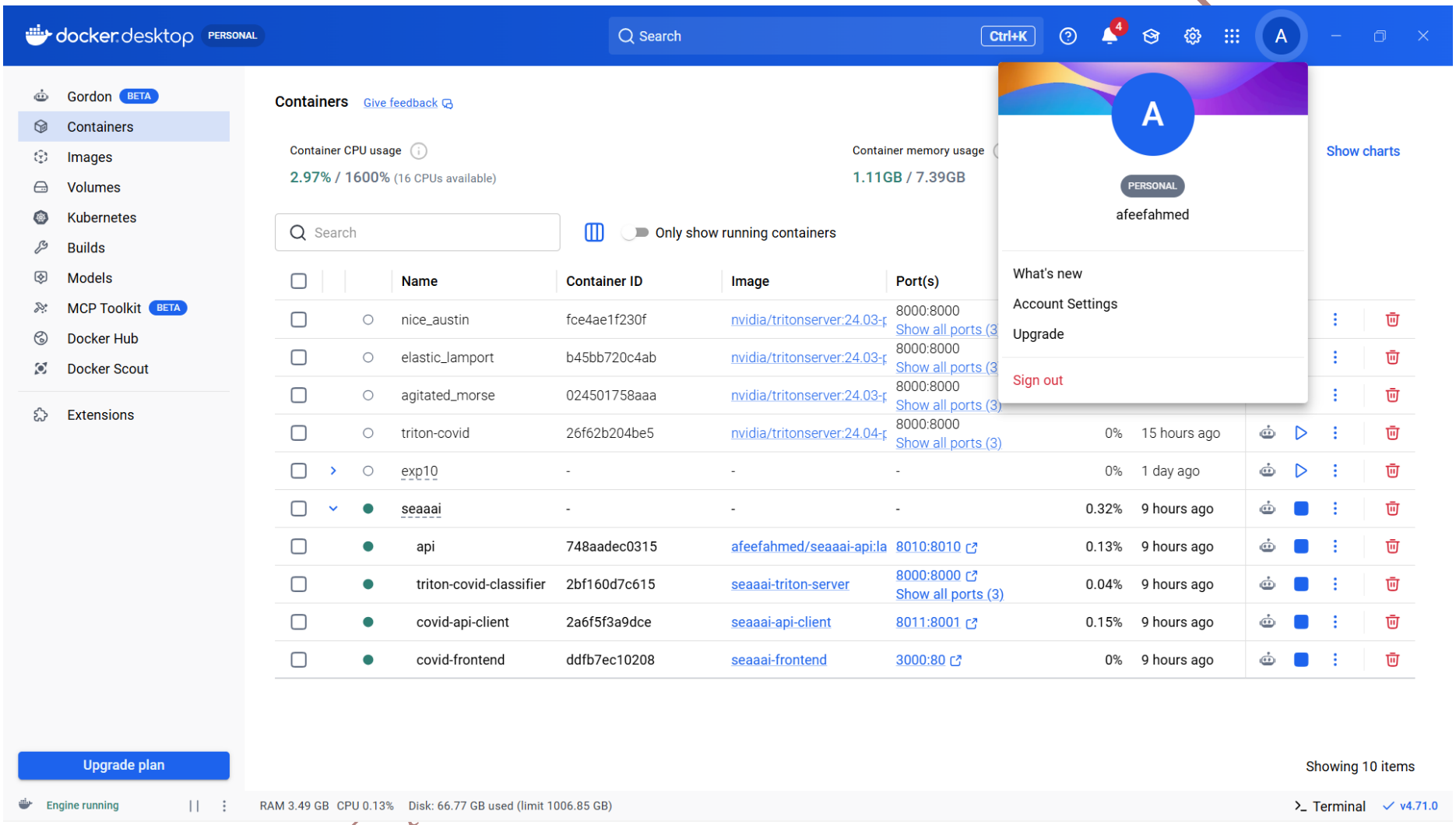


Figure 64: The Whole Project running on docker

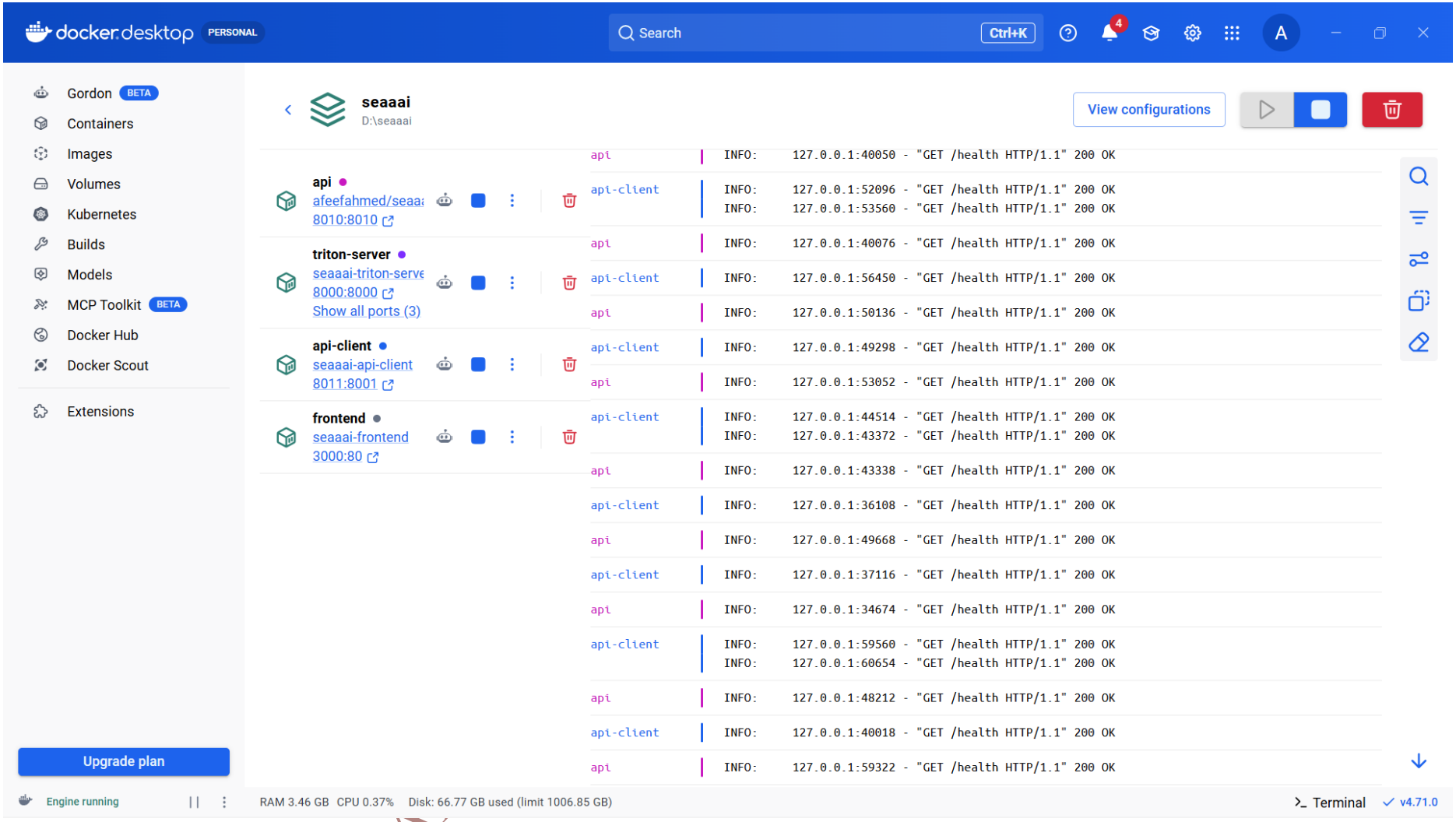


Figure 65: All the container inside docker container

1. Dataset and Training Details

The creation of the proposed COVID-19 Detection System is possible because of the COVID-19 Radiography Database, which is the commonly-used dataset for medical image classification tasks. It includes numerous chest radiography images grouped into several classes: COVID-19, Normal, Viral Pneumonia, and Lung Opacity. The presence of over 21,000 images makes it diverse enough to train the system with deep learning methods.

The dataset has training, validation, and testing samples to ensure the possibility of reliable evaluation of the system's performance. For example, the training subset is utilized to train models and learn necessary patterns, while the validation one monitors the learning process and prevents overfitting. At last, test data is used to assess the results and estimate the performance in real-world conditions.

To enhance the performance of models, multiple preprocessing techniques are applied. They include resizing images into standard input dimensions (224 × 224 pixels) and pixel values normalization. Some other techniques used are rotations, flips, and scaling of images to diversify data. Finally, because of class imbalance, class weighting was applied during the learning stage to ensure fair results.

Deep learning models are trained using TensorFlow library in Google Colab equipped with GPU. Transfer learning is used to pretrain the models on pretrained architectures such as ResNet50 and DenseNet121. The best performance of models trained with transfer learning and without it are compared. Moreover, class weighting and standard training techniques are also explored.
The accuracy score reaches about 94% when using the most productive approach. Thus, the selected combination of techniques yields impressive results and ensures an accurate system.

In general, with an extensive dataset, numerous preprocessing techniques, transfer learning, and GPU utilization, the developed system is highly accurate and reliable.

2. Deployment and Scalability

To provide scalability and portability to the developed system, a number of techniques can be used. They include the creation of self-contained units, packaging of components into containers, and implementing cloud computing capabilities. All mentioned elements are present in the design of the system, making it scalable and portable.

A high-performance inference server is utilized to efficiently process numerous requests and perform in real-time environment. Additionally, because of the hybrid architecture, the system works in environments with or without GPUs. The use of container orchestration makes it easy to scale the system up to handle higher loads.

By implementing such technologies as container orchestration, the system can easily scale to process higher volumes of data. Therefore, the solution can be deployed in hospitals, scientific centers, and even in remote medical centers. Generally speaking, the deployment approach enables the creation of an AI tool that is powerful and pragmatic.

3. Real-World Impact

With its help, the diagnosis of such diseases as coronavirus infection, lung inflammation, and other related conditions can be carried out with precision and accuracy. Furthermore, the AI-driven diagnostic tool will make it easier for specialists to make informed choices since the system offers both the results and explanations. It may serve as an additional aid to those parts of the world where experienced radiologists cannot be found.

The use of such a system can positively affect healthcare through improved speed and quality of diagnostics. Moreover, the possibility of generating structured reports and evaluating the severity of symptoms increases the efficiency of the entire diagnostic process.

4. Conclusion

This paper has offered an insight into developing a powerful system designed to help people get reliable diagnoses. Such a tool combines various modern approaches used in machine learning and can provide useful solutions for medical diagnostics.

The project can be seen as an example of the successful application of artificial intelligence in solving complex problems and creating helpful applications for everyday practice

PulmonaryAI: A Hybrid Deep Learning and LLM-Based System for Multi-Lung Disease Detection from Chest X-Ray Images

Afeef Ahmed Shaik
Department of Computer Science and Engineering
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
Tiruchirappalli, India
afeefahmed.shaik@gmail.com

Abstract

This project proposes PulmonaryAI which is an end-to-end AI-based clinical decision support system focused on classification of chest x-rays into four classes viz. COVID-19, Normal, Viral Pneumonia and Lung Opacity. For prediction, it uses hybrid inference pipeline which includes the use of NVIDIA Triton Inference Server for deploying the pre-trained Keras-based deep learning classifier model locally. Unlike classical inference models which follow single model architecture, PulmonaryAI follows ensemble model where confidence scores of predictions made by each of the branches of inference are combined in order to get the final prediction output. Grad-CAM based visualizations are used in order to identify regions in the image that play key role in determining the prediction output from the model. Besides this, it also incorporates the use of LLMs to generate clinical explanations for the prediction made by model in human readable form. The overall implementation of platform follows microservice based approach where backend is built using FastAPI and frontend is built using React (Vite).

Keywords

Multi-class Lung Disease Classification, Deep Learning, Chest X-ray Analysis, Explainable AI, Grad-CAM, Ensemble Models, Clinical Decision Support Systems

Metadata

Nr	Code metadata description	Metadata
C1	Current code version	v1.0
C2	Github link to code/repository	https://github.com/ssdafeef/PulmonaryAI
C3	Legal code license	MIT
C4	Code versioning system used	Git
C5	Software code languages, tools and services used	Python 3.x, FastAPI, React (Vite), TensorFlow/Keras, NVIDIA Triton Inference Server, Docker, Gemini API
C6	Compilation requirements,	Docker Desktop (12GB+ RAM recommended), Python 3.x, Node.js 18+

	operating environments and dependencies	
C7	Developer documentation/manual	https://github.com/ssdafeef/PulmonaryAI/blob/main/README.md
C8	Support email for questions	afeefahmed.shaik@gmail.com

1. Motivation and Significance

Despite advancements, respiratory diseases, including COVID-19, pneumonia, and lung opacity, remain significant global health concerns, especially in locations where expert radiologists are not available. Chest X-rays are commonly used for diagnosis. However, the process is often lengthy, subjective, and requires clinical expertise.

PulmonaryAI proposes an automated and explainable diagnostic solution that will aid clinicians in decision-making. While existing solutions rely solely on deep learning models for inference, PulmonaryAI proposes a novel hybrid approach using a multi-branch inference strategy that uses Triton served models and locally hosted Keras inference models. The approach will improve the robustness of the inference process and reduce the need to rely solely on a single inference model's results.

The main novelty of the system is the confidence-aware ensemble approach that checks the agreement among all inference sources. If there is a high level of agreement among the predictions, the prediction is marked as having a high level of confidence. Otherwise, if the outputs are different, the prediction is marked as low confidence. The second motivation behind the development is interpretability. Most medical AI models have a black-box nature, limiting the ability of clinicians to validate their reasoning. In addition to providing visual interpretability through Grad-CAM visualization, the model will also integrate the use of a large language model that can automatically generate clinical reasoning.

2. Software Description

2.1 Software Architecture

The software architecture for PulmonaryAI is based on a modular design, with a three-layer approach that includes:

- **Frontend Layer (React + Vite)**
  Offers a user-friendly interface for uploading chest X-ray images, displaying the predictions, interpreting Grad-CAM maps, and downloading the reports.

- **Backend Layer (FastAPI)**
  Serves as the coordination system for handling:
  - Image preprocessing (resizing to 224×224 and normalizing)

- Scheduling model inference
- Ensembling of predictions
- Generation of Grad-CAM heat maps
- LLM-driven report generation

- **Inference Layer (Hybrid Model Serving)**
  - **Triton Inference Server:** Runs the main deep learning model for inference operations
  - **local Keras Model: Serves as the secondary model for inference** reliability

The process starts with users uploading the images, followed by image processing and parallel inference using Triton and Local Models.

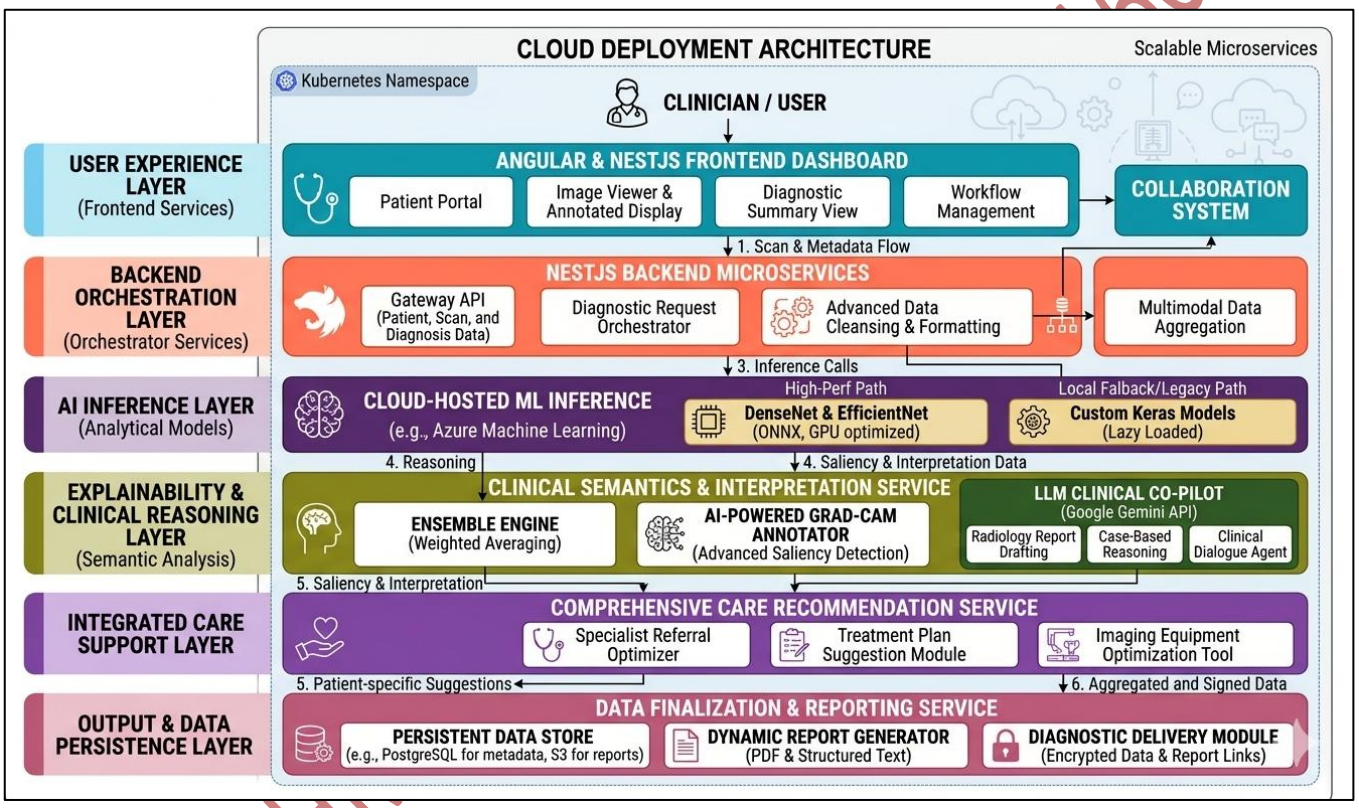


Figure 1: PulmonaryAI system architecture depicting the end-to-end pipeline from image input to diagnosis, explanation, and clinical report generation.

2.2 Software Functionalities

Some of the advanced features offered by PulmonaryAI are:

- **"Multi-Class Disease Classification"**

  Classification of chest x-rays into:
  - COVID-19
  - Normal
  - Viral Pneumonia
  - Lung Opacity
- **"Hybrid Ensemble Prediction"**

Prediction of classification by combining the results of Triton and local prediction models with confidence and agreement.

- **“Explainability via Grad-CAM”**
  Heatmap generation to identify regions affecting model decisions.
- **“Explainability via Grad-CAM”**
  Conversion of numerical results into medical conclusions such as:
  - Diagnosis summary
  - Evidence points
  - Differential diagnosis
  - Actionable suggestions
- **“Image Quality and Severity Estimation”**
  Analysis of input image quality and severity estimation through areas of attention.
- **“Interactive Visualization”**
  Displays predictions, heatmaps, and ai-generated insights in real time.

2.3 Implementation Details

n the PulmonaryAI system, deep learning is coupled with the web framework and cloud deployment capabilities for the creation of the production-grade application. The backend implementation involves FastAPI with asynchronous support to allow processing of multiple users' input at once. Preprocessing of the chest X-rays images includes resizing to 224 × 224 pixels, normalization of pixel values, and conversion into the format suitable for the neural network. The classification model implemented uses the architecture of ResNet and includes dense layers with four output classes. Hybrid inference strategy has been adopted in PulmonaryAI, with the first being hosted within NVIDIA Triton Inference Server and the second being the local Keras model. HTTP requests are used to transmit data between the backend and Triton server; specifically, requests sent from the backend include tensors of preprocessed images, whereas the responses received from the inference contain probabilities of four predictions that will be further processed to determine a class label with a certain level of confidence.

The confidence-aware ensemble method is employed in the system to make a final diagnosis based on the agreement of two inference sources and assigning predictions to high confidence and further verification categories. Explainability of AI-based predictions is ensured through the integration of Grad-CAM, which provides heatmap visualization of predictions by computing gradients in the final convolutional layer and mapping them onto the original image. Severity estimation and anomaly identification can be done by analyzing such heatmaps. To provide structured reports of AI-assisted diagnosis, PulmonaryAI implements LLM through an external API. Frontend of the app is created using React and Vite and is capable of interacting with the backend via REST API. The entire application is containerized using Docker and deployed using Docker Compose tool.

3. Illustrative Example

In order to showcase the functioning of PulmonaryAI, an image taken through a chest X-Ray scan is uploaded through the web-based interface. Then, the backend process processes the image and sends it simultaneously to the Triton server and the locally installed Keras model. Both models come up with the respective predictions as well as their respective confidence scores. Ensemble methods are applied on both the predictions and accordingly the final result is determined.

For instance, in case of similar predictions by both the models of presence of COVID-19 with high level of confidence, the final result comes up as 'COVID-19 (High Confidence)' and Grad-CAM visualization is used to generate heat maps indicating the severity of condition.

Lastly, the LLM creates a clinical report including:

- Diagnosis report
- evidence
- Differential diagnosis
- Recommendations for Clinical Action

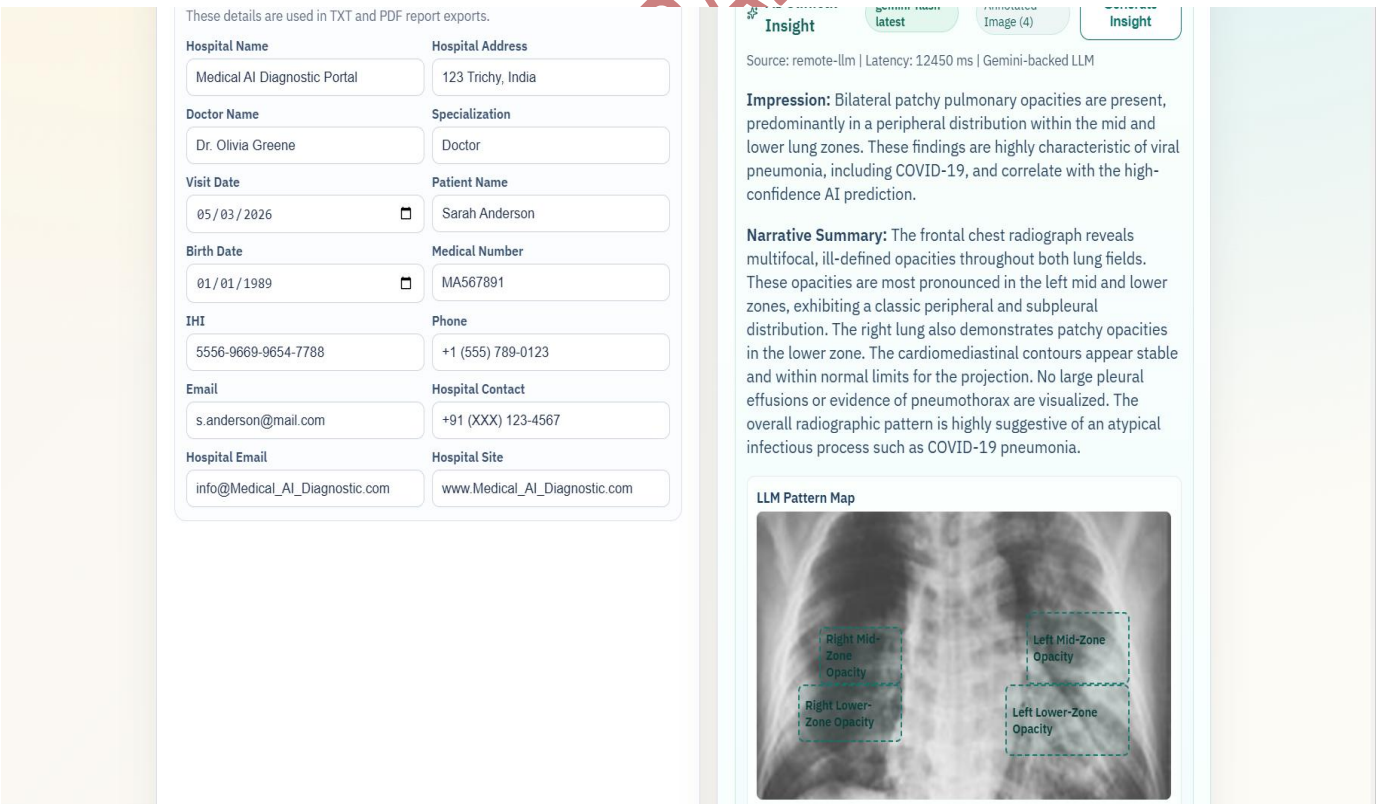


Figure 2: PulmonaryAI output detecting COVID and showcasing Virus being detected using LLM model

4. Impact

There are several promising directions of research associated with the implementation of PulmonaryAI in the domain of medical imaging and artificial intelligence-based decision support systems. The first important novelty of this approach involves the implementation of hybrid inference architecture based on the combination of a centralized inference framework, like NVIDIA Triton, and local models for ensuring redundant and reliable operation. This hybrid inference design allows the use of a number of tradeoffs between latency, scalability, and interpretability, thus allowing application in real clinical environment. At the same time, the utilization of Grad-CAM visualization in the context of multi-models ensemble raises important questions about the consistency of generated explanations, their validity and credibility. Another important innovation relates to the integration of the large language model allowing for automatic generation of structured patient-friendly explanations of findings made by the algorithm.

This project is significantly better than existing algorithms, as it is designed to operate under confidence-aware ensemble and provide for greater generalization of results and enhanced robustness. It addresses one of the main challenges that academic AI projects face – lack of practical implementations due to scalability issues. This system is designed to allow for fast preliminary assessment of the chest x-rays with the help of the generated heatmap and automatic clinical report, assisting clinicians in making better-informed decisions, particularly in under-resourced areas. The design of a modular and containerized system ensures easy deployment and usage across organizations. Moreover, there are multiple opportunities for developing commercial products using this solution in the future, including telemedicine services and other AI-based diagnostic tools.

5. Conclusions

The PulmonaryAI solution provides a scalable and explainable artificial intelligence-based tool for multi-class pulmonary disease classification based on chest X-ray images using an inference approach that includes hybrid inference. Using the NVIDIA Triton Inference Server to serve a locally trained deep learning model, the solution offers better stability and robustness via ensemble-based predictions. The combination of Grad-CAM helps provide visual interpretability, whereas the introduction of a large language model contributes towards improved explanation generation.

The use of Docker container technology for the construction of PulmonaryAI allows for easy deployment of the system and its portability across different environments. Thus, PulmonaryAI shows how recent advances in areas such as deep learning, inference, and explainable artificial intelligence can be effectively integrated to create a practical tool contributing to the development of computer-aided diagnosis.

6. Acknowledgements

The author gratefully acknowledges the valuable inputs and constant guidance received from the faculty and mentors in developing the current study. The author wishes to thank the Department of Computer Science for the facilities provided to carry out the research. The author also wishes to acknowledge the role of the open source community in helping develop this study through the use of various resources.

7. References

[1] COVID-19 Radiography Database, Kaggle Dataset. Available: https://www.kaggle.com/datasets/tawsifurrahman/covid19-radiography-database

[2] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE CVPR, 2016.

[3] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," in Proc. IEEE ICCV, 2017.

[4] NVIDIA Corporation, "Triton Inference Server," 2023. Available: https://developer.nvidia.com/triton-inference-server

[5] TensorFlow Developers, "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. Available: https://www.tensorflow.org/

[6] FastAPI Contributors, "FastAPI Framework," 2023. Available: https://fastapi.tiangolo.com/

[7] Google, "Gemini API Documentation," 2024. Available: https://ai.google.dev/

[8] Docker Inc., "Docker Documentation," 2023. Available: https://docs.docker.com/

Demo Video as per Guidelines

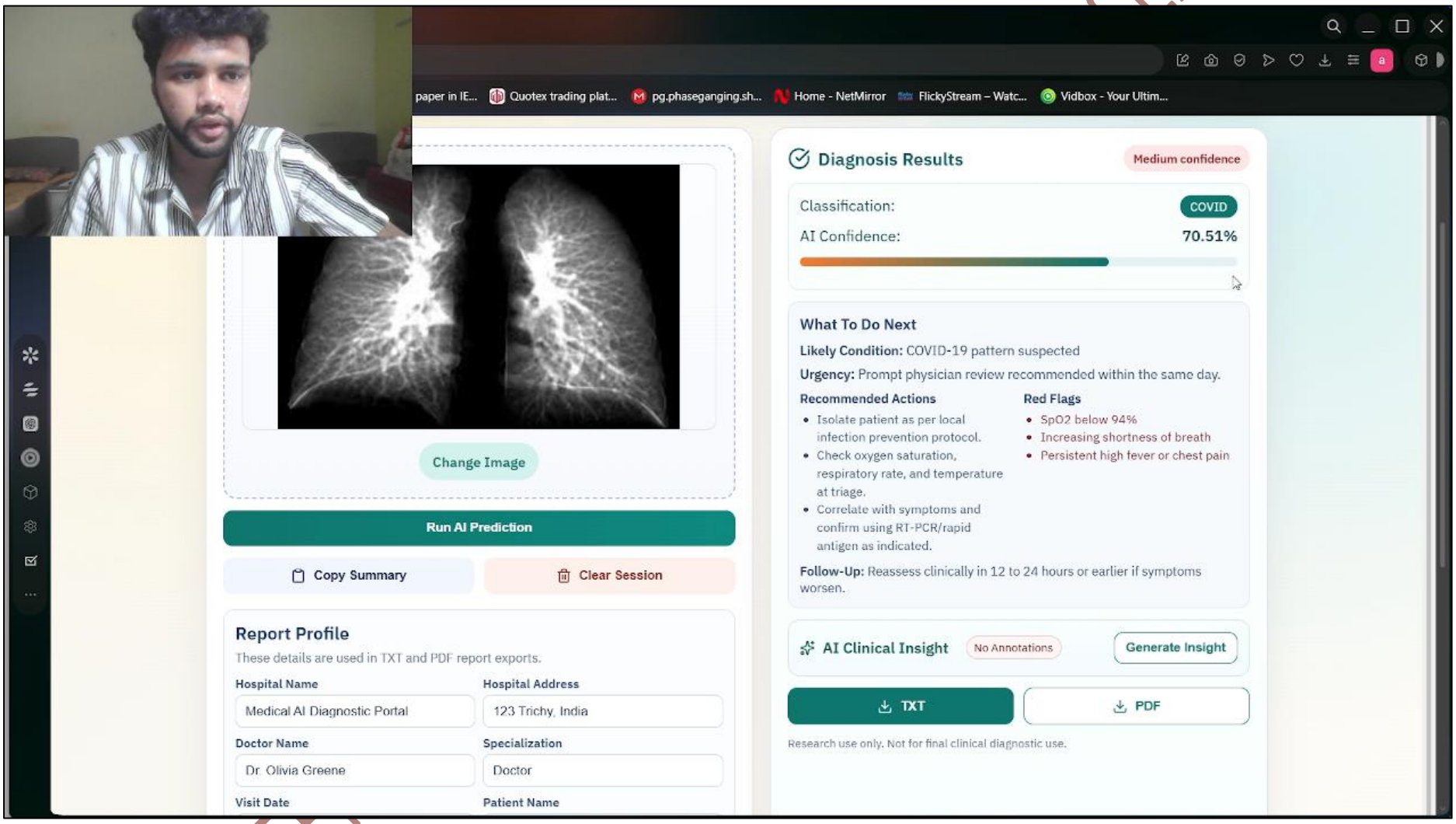


Fig 66 : Vidio Drive Link : https://drive.google.com/drive/folders/1HYollCRsFegY67pfavab6J0K13X4mKf?usp=sharing

Github Repo and Fork

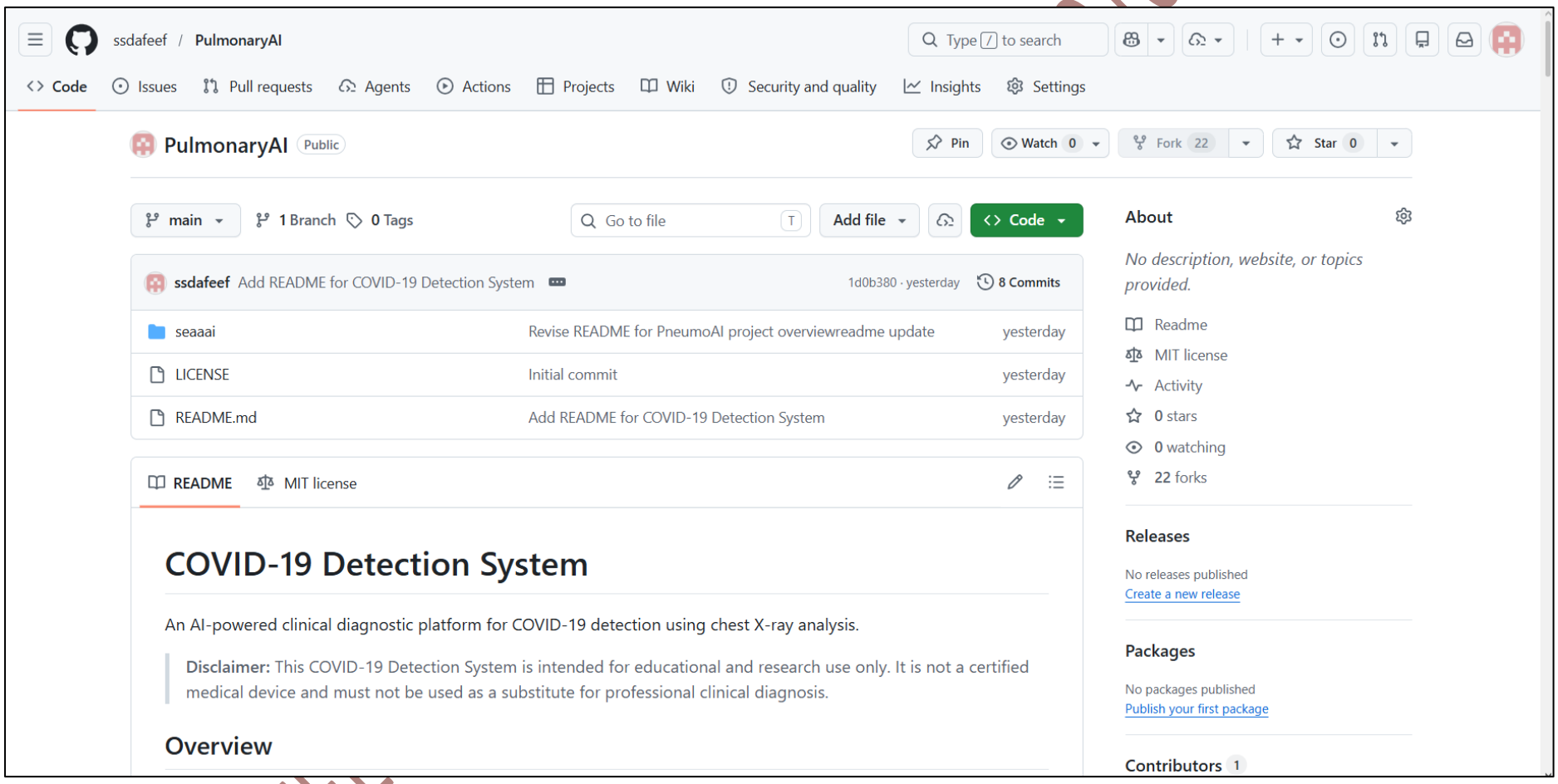


Fig 67: GitHub Repository Overview (PneumoAI Project) — https://github.com/ssdafeef/PulmonaryAI

Review 1: Quality of the Module Developed

First of all, an essential aspect of the developed module is the presence of a structure in the system and the successful use of several advanced technologies within the model. It should be mentioned that the process of creating the model was performed on the basis of a rather rich COVID-19 Radiography Database, which includes many samples of chest radiograph images. The interaction with such a resource allowed considering important aspects, such as class distributions, data preparation, and training processes, contributing greatly to the creation of the proposed system.

Another crucial feature of the module is high computational efficiency due to the possibility to work with GPU. Training models in Google Colab using GPU resources allowed performing faster tests and avoiding problems related to the necessity to process big amounts of data and train complicated structures of deep learning models.
Moreover, it is important to note that another aspect of the quality of the module concerns its architecture. In the context of the described project, one can observe how the system successfully combines such important components as data preprocessing, model prediction, explaining AI, and report generation. Moreover, each component performs its function while improving the performance of the overall system.

In addition to the previous aspects, it is vital to note the presence of the feature of the generated explanations by the system. The ability to see visual explanations of results and use the information provided by AI about the disease increases the quality of the tool significantly.

Overall, it should be noted that the created project demonstrates the high quality of its modules since theoretical ideas are combined with the implementation of the solution..

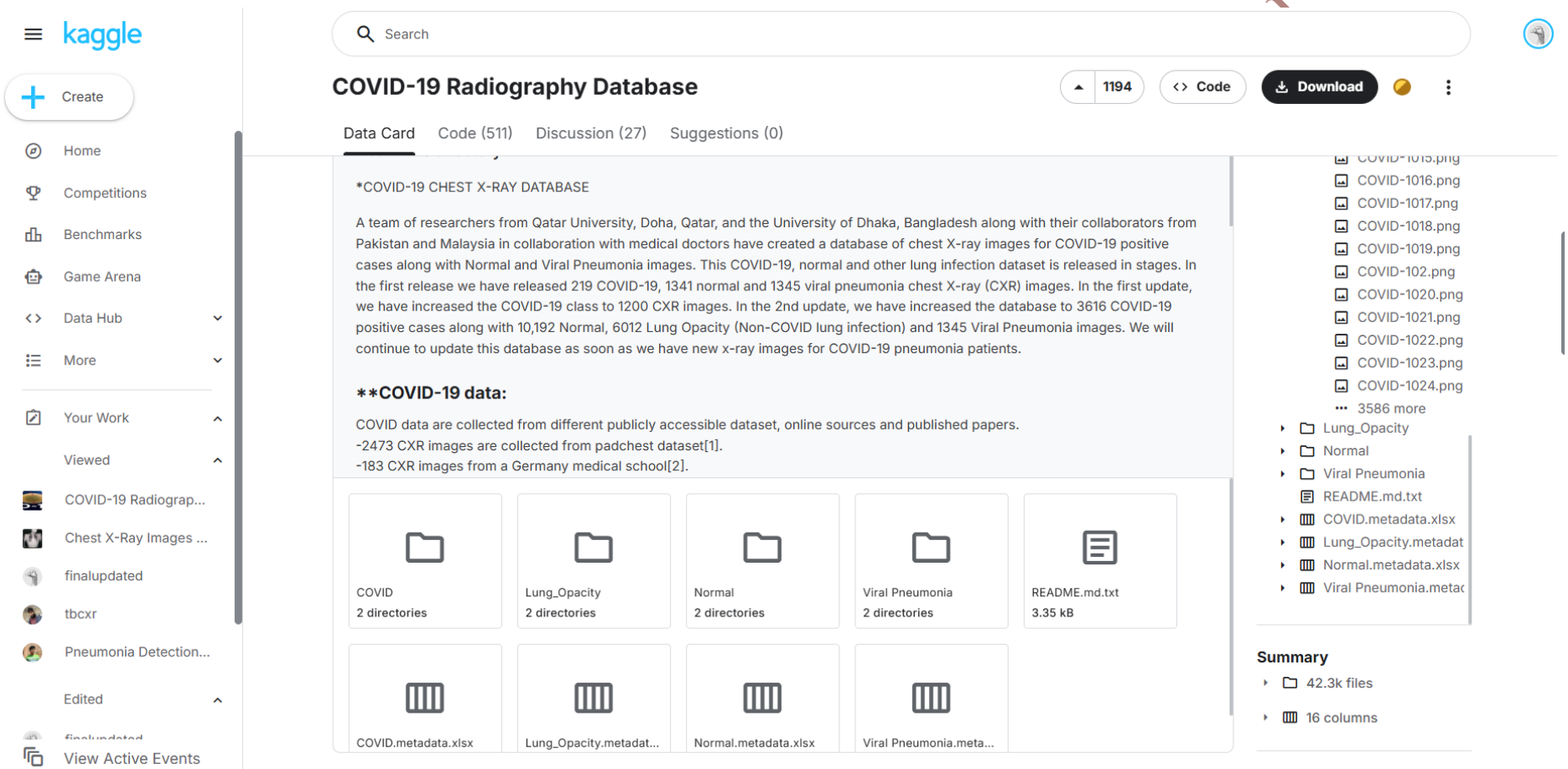


Figure 18: All the different dataset images

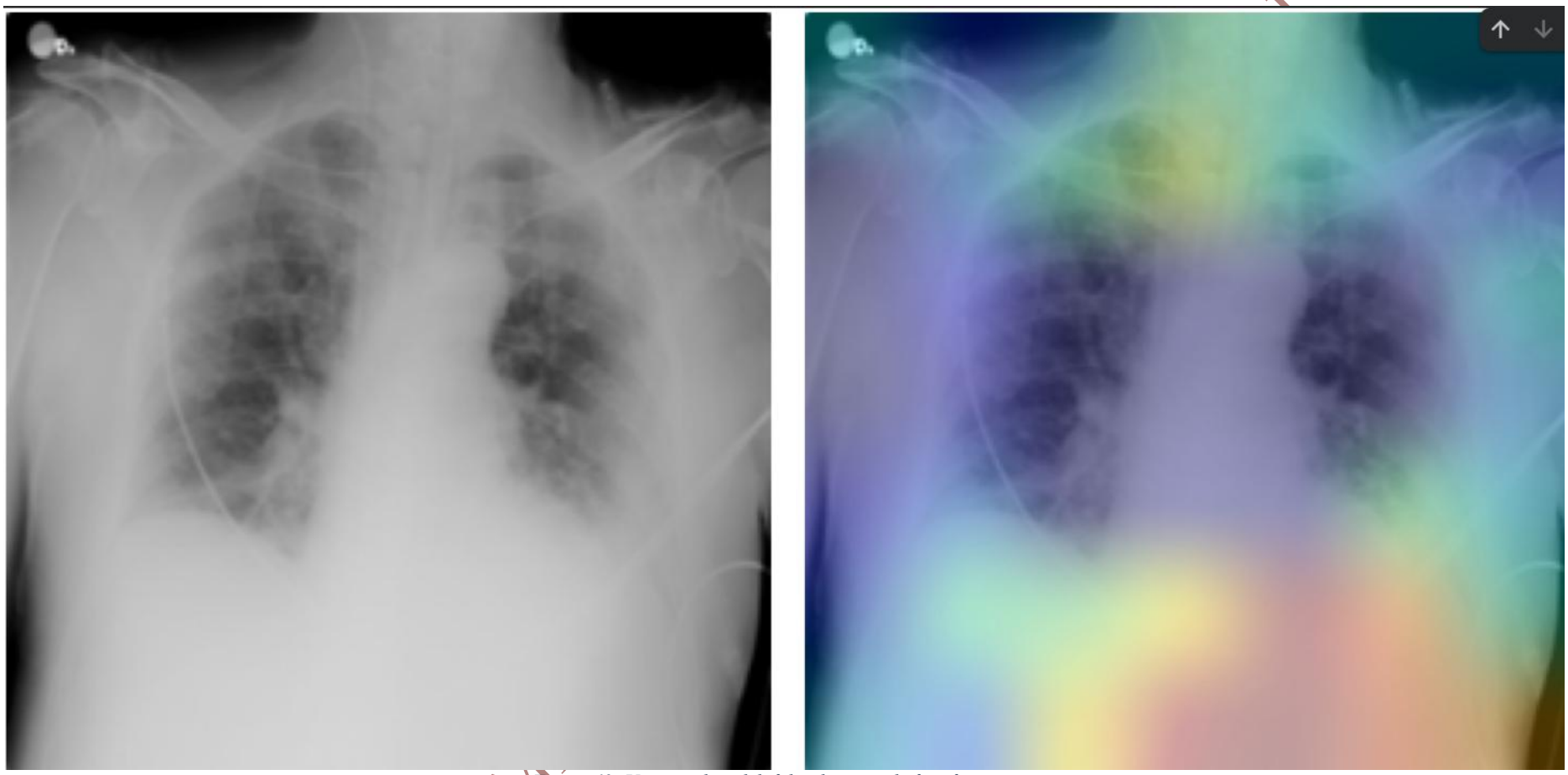


Figure 69: Untrained model false detection before fine tuning

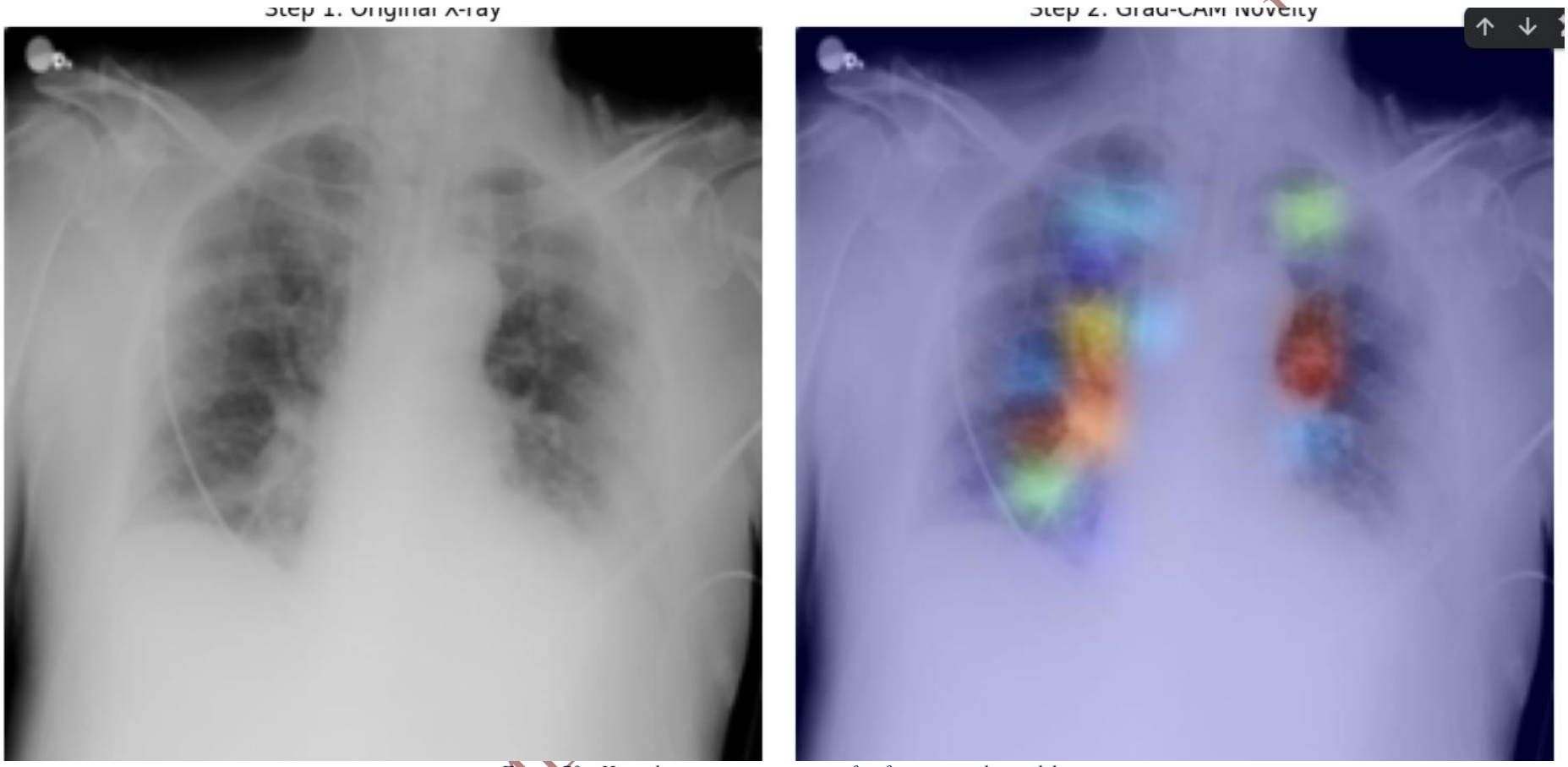


Figure 70:: Huge detection improvement after fine-tuning the model

Review 2: Containerization and Deployment

One of the most important features of the developed COVID-19 Detection System is the presence of clear mechanisms for containerization and deployment which make the system easily reproducible, portable and fit for productive use.

During the process of system creation, a special focus was put on following modern software architecture and deployment trends. Therefore, containerization was used in accordance with the modern principles based on Docker and Docker Compose technologies.

System Architecture:

The developed COVID-19 Detection System can be viewed as a multi-component architecture including the following parts:

- **Frontend**: React app that uses Vite for development and optimized building
- **Backend**: FastAPI server for images preprocessing, orchestration and API management
- **Model Inference**: NVIDIA Triton Inference Server for efficient, high-performance inference of models
- **Local Model**: As a fallback, Keras model can be used to ensure more reliable results

All of these components were packaged separately into containers by means of Docker technology. It guarantees that no external dependencies exist, and all necessary libraries are packed.

Docker Compose Orchestration

In order to facilitate the launch of multiple services, Docker Compose technology was used. The file docker-compose.yml describes services, their configuration, dependencies and networking. The system can be started with the following command:

```
docker compose up -d
```

This command does the following:

- Building and launching of Triton Inference Server with COVID classifier model loaded
- Starting a FastAPI server with correct health check setup and dependencies
- Launching React application
- Setting up inter-service communication and volumes for model loading

NVIDIA Triton Model Serving

When deploying a system, production-ready NVIDIA Triton Inference Server is used for efficient inference of pre-trained models. For setting up the service, the corresponding Docker image from NVIDIA repository is downloaded first:

```
docker pull nvcr.io/nvidia/tritonserver:23.10-py3
```

Service Endpoints

After deployment, all service endpoints can be accessed using the following URLs:

- **Frontend**: http://localhost:3000— React Vite application for uploading X-rays and visualizing results
- **Backend API**: http://localhost:8010/docs — FastAPI documentation with prediction endpoint
- **Triton Server**: http://localhost:8001— NVIDIA Triton Inference Server for models inference (HTTP), http://localhost:8002 (gRPC

Benefits of Deployment

The following are the key benefits of this deployment solution:

- **Consistency:** Standardized execution in development, testing, and production stages
- **Mobility:** All services can be executed on any device that supports Docker and Docker Compose
- **Scaleability:** Individual scaling of services based on demand (front-end, back-end, inference)
- **Resilience:** Health checks guarantee the availability of services; fault tolerance is provided by the mixed inference pipeline using Triton and local Keras
- **Deployment Ready:** Enterprise-level orchestration that is capable of being deployed to cloud-based systems or on-site infrastructure

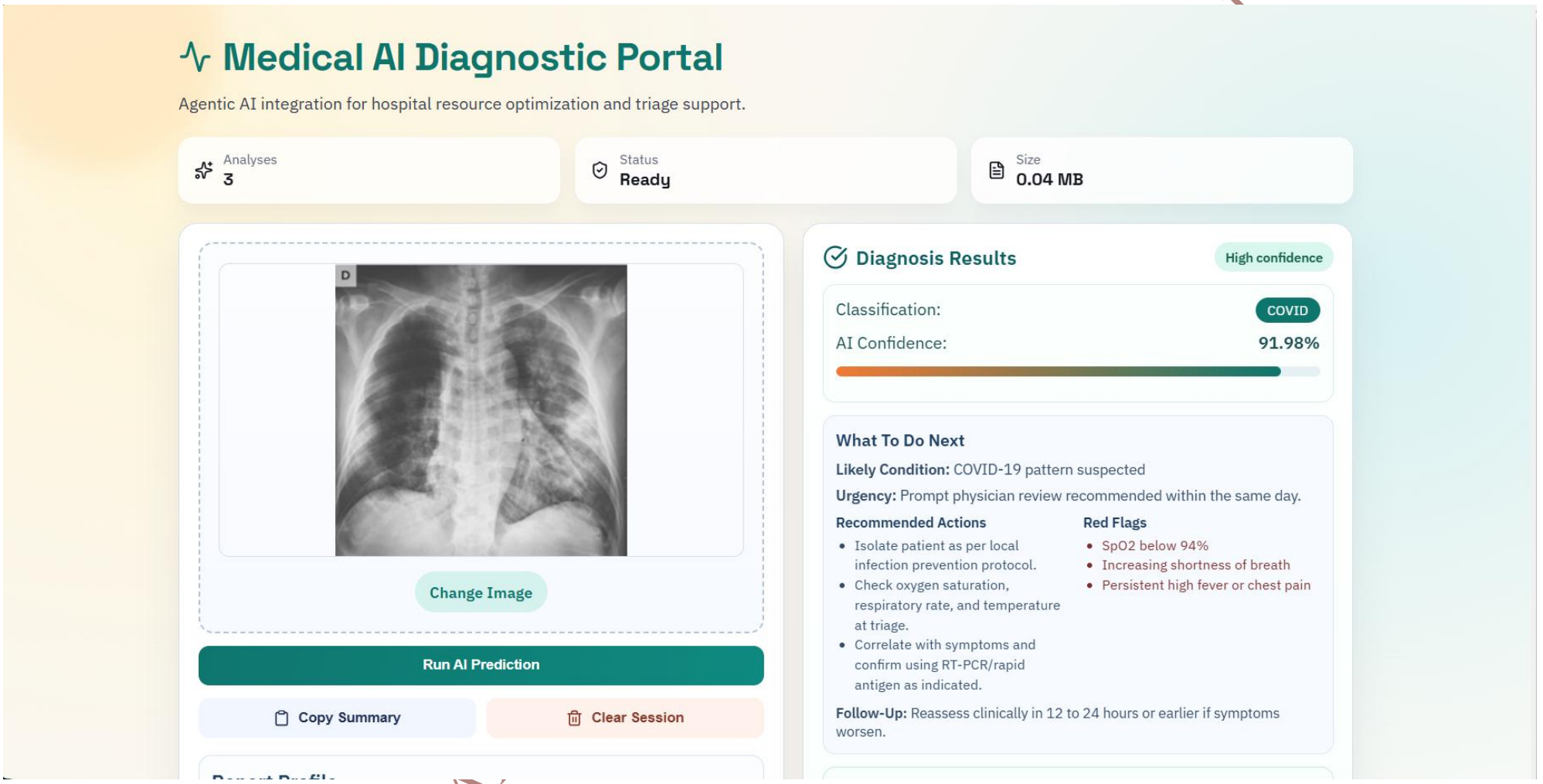


Figure 71: Main Dashboard

Medical AI Diagnostic Portal | 123 Trichy, India

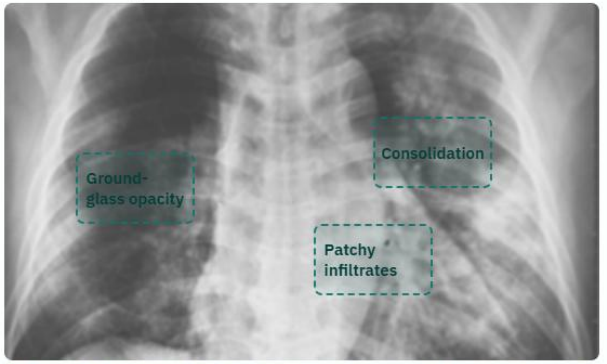
Doctor Name: Dr. Olivia Greene
Specialization: Doctor
Visit Date: 05/03/2026
Patient Name: Sarah Anderson
Birth Date: 01/01/1989
Medical Number: MA567891
IHI: 5556-9669-9654-7788
Phone: +1 (555) 789-0123
Email: s.anderson@mail.com
Hospital Contact: +91 (XXX) 123-4567
Hospital Email: info@Medical_AI_Diagnostic.com
Hospital Site: www.Medical_AI_Diagnostic.com

Source: remote-llm | Latency: 9296 ms | Gemini-backed LLM

Impression: Multifocal, predominantly peripheral ground-glass opacities and consolidations are present bilaterally, significantly more pronounced in the left lung. These findings are highly suggestive of viral pneumonia, such as COVID-19, given the clinical context and high AI confidence. Correlation with RT-PCR testing and clinical symptoms is strongly recommended.

Narrative Summary: The frontal chest radiograph demonstrates extensive, patchy opacities distributed throughout both lung fields. There is a notable peripheral and mid-to-lower zone predominance, which is characteristic of atypical or viral pneumonias. The left lung shows more confluent areas of consolidation compared to the right, particularly in the mid-lung zone. No significant pleural effusions or pneumothorax are identified. The cardiomediastinal silhouette remains within normal limits for size and configuration, and the visualized osseous structures are unremarkable.

LLM Pattern Map



- **Ground-glass opacity:** Patchy peripheral opacities in the right mid-to-

Figure 72: LLM model Insights

LLM Annotated Image

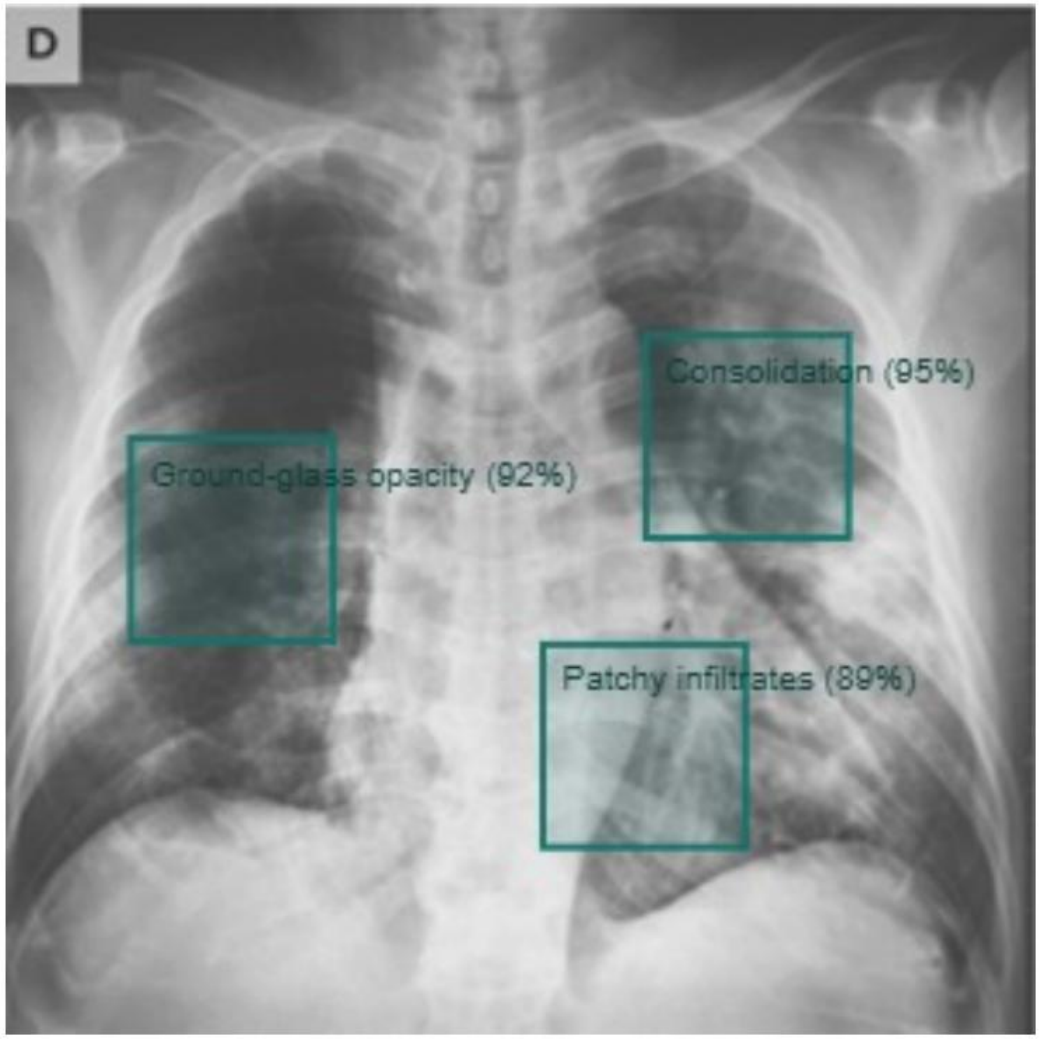


Figure 73: Model Overly of Detected virus

:

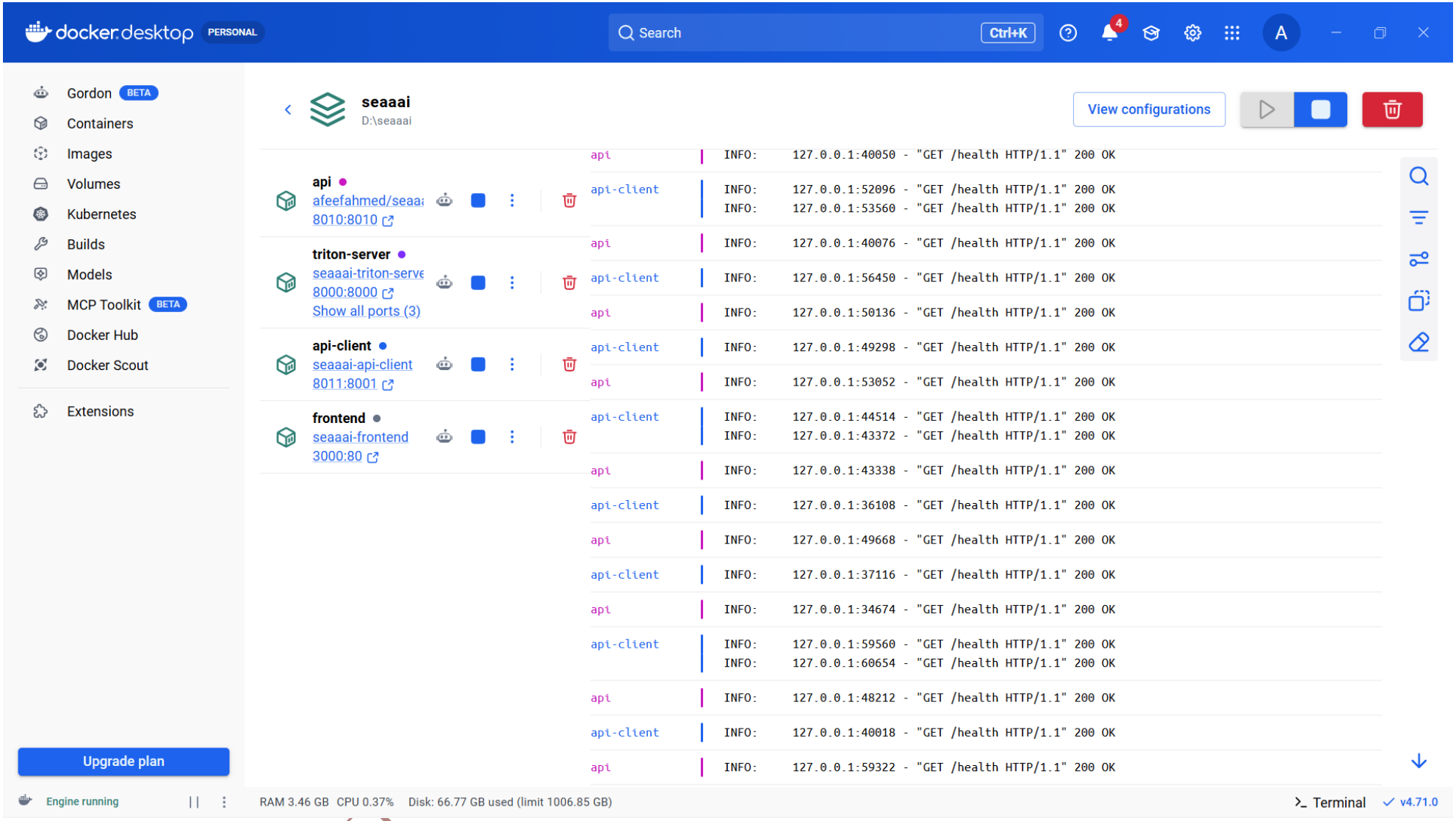


Figure 74: All containers running in docker

COURSE CERTIFICATE

Apr 13, 2026

AFEEF SHAIK

has successfully completed

Software Engineering in AI

an online course authorized by SRM IST Trichy and offered through Coursera

Balaji Ganesh R
Associate Professor
School of Computing

Verify at:
https://coursera.org/verify/WN3CDE6DIKRI

Coursera has confirmed the identity of this individual and their participation in the course.

The course contents are generated using AI and aggregated from various sources available in Coursera.